

```

    } catch (e) {
      console.error("[whiteboard] Failed to load state:", e);
      return null;
    }
  }
}

let lastSaveWasDegraded = false;

/**
 * Fire a storage-health event without ever throwing. saveState's whole job on
 * the failure path is to report trouble instead of propagating it; if the
 * reporting itself threw (no CustomEvent, no dispatchEvent, a listener that
 * raises) the exception would escape saveState and reach the autosave timer –
 * the exact crash the quota handling exists to prevent.
 */
function notify(event: string): void {
  try {
    window.dispatchEvent(new CustomEvent(event));
  } catch {
    /* reporting is best-effort; never let it break a save */
  }
}

/**
 * Ordered save queue. IndexedDB writes are async, so two rapid saves could
 * otherwise interleave and an OLDER snapshot could land after a newer one.
 * Chaining keeps last-write-wins: every save waits for the previous one.
 * Rejections are consumed here (each failure is already reported via the
 * storage-health events below), so the chain itself never breaks.
 */
let saveChain: Promise<void> = Promise.resolve();
function queueWorkspaceSave(task: () => Promise<void>): Promise<void> {
  saveChain = saveChain.then(task, task);
  saveChain.catch(() => undefined);
  return saveChain;
}

/**
 * Tiny cross-tab heartbeat. The workspace itself moved to IndexedDB (which
 * fires no cross-tab events), so every successful save also stamps ~50 bytes
 * into localStorage – just enough to keep the `storage`-event based
 * multi-tab guard in `watchExternalSaves` working. Best-effort by design: a
 * full localStorage must never be able to fail the real save.
 */
let heartbeatRev = 0;
function writeHeartbeat(): void {
  try {
    heartbeatRev += 1;
    window.localStorage.setItem(HEARTBEAT_KEY, JSON.stringify({ rev: heartbeatRev, at: Date.now() }));
  } catch {
    /* heartbeat is expendable; the save already succeeded */
  }
}

let lastPressureCheck = 0;
/**
 * Early warning BEFORE writes start failing: if the origin's pooled storage
 * is past the pressure line, tell the user to back up while saves still work.
 * Throttled – estimate() is cheap but there is no reason to ask twice a second.
 */
function checkPressureThrottled(): void {
  const now = Date.now();
  if (now - lastPressureCheck < 30_000) return;
  lastPressureCheck = now;
  void measureStoragePressure().then((p) => {
    if (p && p.ratio >= STORAGE_PRESSURE_RATIO) notify("wb:storage-pressure");
  });
}

/**
 * Primary save. Where the bytes land depends on capability, not preference:
 *
 * - IndexedDB present (every real browser, the APK, Electron): the snapshot
 *   is gzipped past 1 MB and stored there, where the quota is a share of free
 *   disk instead of localStorage's ~5 MB hard cap.
 * - IndexedDB absent (unit tests, SSR, exotic WebViews): the legacy
 *   synchronous localStorage write, byte-for-byte the old behavior – the E2E
 *   suites exercise exactly this path.

```

```

*
* Failure reporting is identical on both paths: `wb:save-degraded` when only
* the image-stripped retry fit, `wb:save-failed` when nothing could be
* written, `wb:save-recovered` when writes succeed again.
*/
export function saveState(state: PersistedState): void | Promise<void> {
  if (typeof window === "undefined") return;
  if (!workspaceDbAvailable()) {
    saveStateLocalStorage(state);
    return;
  }
  return queueWorkspaceSave(() => saveWorkspaceToBackend(state, createIdbWorkspaceBackend()).then(() => undefi
    ned));
}

/**
* The IndexedDB save itself, backend-injected so unit tests can prove the
* quota story with an in-memory store: returns how the write landed.
* Production always passes the real IndexedDB backend via saveState above.
*/
export async function saveWorkspaceToBackend(
  state: PersistedState,
  backend: WorkspaceBackend
): Promise<"ok" | "degraded" | "failed"> {
  let json: string;
  try {
    json = JSON.stringify({ ...state, version: SCHEMA_VERSION });
  } catch (e) {
    // Unserializable state (e.g. a cycle smuggled into an element). This
    // used to escape saveState and kill the autosave timer; reporting a
    // failed save keeps the app alive and the banner truthful.
    console.error("[whiteboard] Failed to serialize state:", e);
    notify("wb:save-failed");
    return "failed";
  }
  try {
    await backend.save(await encodeWorkspace(json));
  } catch (e) {
    console.error("[whiteboard] Failed to save state:", e);
    // Degraded retry: pasted image data-URLs are the heaviest single
    // payloads. Structure, strokes and text survive; images don't.
    try {
      await backend.save(await encodeWorkspace(JSON.stringify(withoutImagePayloads(state))));
    } catch {
      notify("wb:save-failed");
      return "failed";
    }
    lastSaveWasDegraded = true;
    notify("wb:save-degraded");
    return "degraded";
  }
  if (lastSaveWasDegraded) {
    lastSaveWasDegraded = false;
    notify("wb:save-recovered");
  }
  writeHeartbeat();
  checkPressureThrottled();
  return "ok";
}

/** The legacy synchronous localStorage write. Still the whole story where IndexedDB is absent. */
function saveStateLocalStorage(state: PersistedState): void {
  const payload = JSON.stringify({ ...state, version: SCHEMA_VERSION });
  try {
    window.localStorage.setItem(STORAGE_KEY, payload);
  } catch (e) {
    console.error("[whiteboard] Failed to save state:", e);
    return saveDegraded(state);
  }
  // The write succeeded. Announcing recovery must not be able to undo that by
  // throwing into the catch above, so it happens outside the guarded block.
  if (lastSaveWasDegraded) {
    lastSaveWasDegraded = false;
    notify("wb:save-recovered");
  }
  writeHeartbeat();
}

```

```

/**
 * Workspace without the heaviest payloads – pasted image data-URLs, from BOTH
 * places they live. This used to touch only `b.elements` – the canvas field –
 * so a notebook, whose images sit in `b.pages[i].elements`, shed nothing, the
 * payload stayed over quota, the retry failed too, and NOTHING was saved.
 * Dropping image pixels is bad; silently saving nothing is worse.
 */
function withoutImagePayloads(state: PersistedState): PersistedState {
  const stripImages = (els: WhiteboardElement[] | undefined) =>
    (els ?? []).map((el) => (el.type === "image" ? { ...el, src: "" } : el));
  return {
    ...state,
    boards: state.boards.map((b) => ({
      ...b,
      elements: stripImages(b.elements),
      ...(Array.isArray(b.pages)
        ? { pages: b.pages.map((p) => ({ ...p, elements: stripImages(p.elements ?? []) })) }
        : {}),
    })),
  };
}

/**
 * Quota (or any write failure) on the localStorage path: retry once without
 * the heaviest payloads. Either way the UI is notified so the user knows
 * saving is impaired.
 */
function saveDegraded(state: PersistedState): void {
  let written = false;
  try {
    const degraded = withoutImagePayloads(state);
    window.localStorage.setItem(STORAGE_KEY, JSON.stringify({ ...degraded, version: SCHEMA_VERSION }));
    written = true;
  } catch {
    written = false;
  }
  // Notifications live outside the try so a throwing listener cannot flip a
  // successful degraded write into a reported failure (or escape the function).
  if (written) {
    lastSaveWasDegraded = true;
    notify("wb:save-degraded");
  } else {
    notify("wb:save-failed");
  }
}

export interface WorkspaceLoadResult {
  state: PersistedState | null;
  /** True when the state was adopted from the legacy localStorage copy. */
  migratedFromLs: boolean;
}

/**
 * Primary load. Prefers the IndexedDB record; adopts the legacy localStorage
 * copy (and moves it over) when there is none; returns null when nothing
 * readable exists anywhere. A corrupt IndexedDB record falls back to
 * localStorage rather than bricking the app – a bad record costs convenience,
 * never boards.
 *
 * The localStorage copy is deleted only AFTER the migrated record is safely
 * stored – a crash mid-migration keeps the old copy readable. Downgrades are
 * not supported: an older build cannot read the IndexedDB record, so moving
 * back to an older APK requires a backup round-trip.
 */
export async function loadWorkspace(backend?: WorkspaceBackend): Promise<WorkspaceLoadResult> {
  const store = backend ?? (workspaceDbAvailable() ? createIdbWorkspaceBackend() : null);
  if (!store) return { state: loadState(), migratedFromLs: false };
  try {
    const rec = await store.load();
    if (rec) {
      return { state: migrate(JSON.parse(await decodeWorkspace(rec)) as PersistedState), migratedFromLs: false };
    }
  } catch (e) {
    console.error("[whiteboard] Workspace record unreadable, falling back:", e);
  }
  const legacy = loadState();
  if (!legacy) return { state: null, migratedFromLs: false };
}

```

```

    try {
      await store.save(await encodeWorkspace(JSON.stringify({ ...legacy, version: SCHEMA_VERSION })));
      try {
        window.localStorage.removeItem(STORAGE_KEY);
      } catch {
        /* origin full; the stale copy is harmless, the live one is in IndexedDB */
      }
      return { state: legacy, migratedFromLs: true };
    } catch (e) {
      console.error("[whiteboard] Could not migrate workspace to IndexedDB:", e);
      return { state: legacy, migratedFromLs: false };
    }
  }

export function clearState(): void {
  if (typeof window === "undefined") return;
  window.localStorage.removeItem(STORAGE_KEY);
  // The workspace itself lives in IndexedDB now; the localStorage key is only
  // the legacy copy / heartbeat. Clearing must remove both, or a reload would
  // resurrect the "deleted" boards from the other store.
  if (workspaceDbAvailable()) {
    createIdbWorkspaceBackend().clear().catch(() => undefined);
  }
}

/**
 * Multi-tab guard: the `storage` event fires in every OTHER tab when this
 * tab writes. Each successful workspace save stamps a tiny heartbeat key, so
 * a write from another tab means this tab's in-memory copy is now stale –
 * listeners can warn the user instead of silently losing work on the next save.
 */
export function watchExternalSaves(onExternal: () => void): () => void {
  if (typeof window === "undefined") return () => {};
  const handler = (ev: StorageEvent) => {
    if (ev.key === STORAGE_KEY) onExternal();
  };
  window.addEventListener("storage", handler);
  return () => window.removeEventListener("storage", handler);
}

/**
 * Validate + backfill an arbitrary saved payload. Exported so that data coming
 * from OUTSIDE localStorage – an imported backup file, or a workspace pulled
 * from OneDrive – goes through exactly the same hardening as a local load.
 */
export function migrateState(old: PersistedState): PersistedState {
  return migrate(old);
}

function migrate(old: PersistedState): PersistedState {
  // Harden against corrupt / partial / hand-edited saves: a board whose
  // `elements` is undefined used to pass through and crash every consumer
  // (`for..of` throws), bricking the app on each load because the bad state
  // was re-saved. Everything is validated and backfilled here.
  const rawBoards = Array.isArray(old.boards) ? old.boards : [];
  const rawFolders = Array.isArray(old.folders) ? old.folders : [];

  const folders: Folder[] = rawFolders
    .filter((f) => f && typeof f.id === "string")
    .map((f) => ({
      ...f,
      name: typeof f.name === "string" ? f.name : "Untitled folder",
      parentId: typeof f.parentId === "string" ? f.parentId : null,
      expanded: f.expanded !== false,
      createdAt: typeof f.createdAt === "number" ? f.createdAt : Date.now(),
      color: typeof f.color === "string" ? f.color : "#f59e0b",
    }));

  // Elements are sanitised identically wherever they live – the canvas field
  // or a notebook page – so a stroke cannot survive on one and be dropped on
  // the other.
  const cleanElements = (raw: unknown): WhiteboardElement[] =>
    (Array.isArray(raw) ? raw : [])
      .filter(
        (el) =>
          el && typeof (el as { id?: unknown }).id === "string" &&
            typeof (el as { type?: unknown }).type === "string"
      )

```

```

// Text boxes started and never written in. Older builds never removed
// them, so they piled up as invisible elements that only showed a "Type
// thoughts..." placeholder when selected. Lossless to drop: they hold no
// text, and a box styled as a shape (fill / border) is kept.
.filter((el) => !isAbandonedTextBox(el as WhiteboardElement))
// Drop the legacy cached stroke `path`. It is derived data that froze
// each stroke's geometry at draw time and cost ~143% of the points'
// size; dropping it reclaims that space and lets existing strokes pick
// up rendering fixes.
.map((el) => {
  const e = el as Record<string, unknown>;
  if (e.type === "stroke" && "path" in e) {
    const { path: _legacyPath, ...rest } = e;
    return rest;
  }
  return e;
}) as unknown as WhiteboardElement[];

const cleanBackground = (bg: unknown): BoardBackground | undefined => {
  const b = bg as { pattern?: string; size?: unknown } | undefined;
  if (!b || !VALID_BOARD_PATTERNS.includes(b.pattern as never)) return undefined;
  return {
    ...b,
    size:
      typeof b.size === "number" && Number.isFinite(b.size)
        ? Math.min(BOARD_PATTERN_MAX_SIZE, Math.max(BOARD_PATTERN_MIN_SIZE, b.size))
        : undefined,
  } as BoardBackground;
};

const boards = rawBoards
  .filter((b) => b && typeof b.id === "string" && typeof b.name === "string")
  .map((b) => ({
    ...b,
    elements: cleanElements(b.elements),
    camera:
      b.camera &&
      typeof b.camera.x === "number" &&
      typeof b.camera.y === "number" &&
      typeof b.camera.zoom === "number" &&
      Number.isFinite(b.camera.zoom) &&
      b.camera.zoom > 0
        ? b.camera
        : { x: 0, y: 0, zoom: 1 },
    // Board paper: drop an unknown pattern rather than rendering nothing, and
    // clamp the spacing. A size of 0 or NaN would make the SVG pattern tile
    // infinitely small and lock the renderer.
    background: cleanBackground(b.background),

    // ---- Notebooks ----
    // A board with no `kind` is a canvas, so everything saved before
    // notebooks existed loads unchanged with no migration.
    ...(b.kind === "notebook"
      ? (() => {
          const rawPages: unknown[] = Array.isArray(b.pages) ? (b.pages as unknown[]) : [];
          const pages: NotebookPage[] = rawPages
            .filter((p) => p && typeof (p as { id?: unknown }).id === "string")
            .map((raw) => {
              const p = raw as Record<string, unknown>;
              return {
                id: p.id as string,
                elements: cleanElements(p.elements),
                background: cleanBackground(p.background),
                bookmarked: p.bookmarked === true,
                // A PDF reference is only usable if both halves survived.
                // A half-written one would render a blank sheet forever.
                pdf:
                  p.pdf &&
                  typeof (p.pdf as { docId?: unknown }).docId === "string" &&
                  Number.isInteger((p.pdf as { pageNumber?: unknown }).pageNumber) &&
                  (p.pdf as { pageNumber: number }).pageNumber >= 1
                    ? (p.pdf as { docId: string; pageNumber: number })
                    : undefined,
                createdAt: typeof p.createdAt === "number" ? p.createdAt : Date.now(),
                updatedAt: typeof p.updatedAt === "number" ? p.updatedAt : Date.now(),
              };
            });
        }) : {});
  });
};

```

```

// A notebook must always have at least one page to draw on, and
// `activePageId` must name a page that exists – a stale id would
// otherwise leave the document apparently blank.
const safePages: NotebookPage[] =
  pages.length > 0
    ? pages
    : [
      {
        id: `pg_recovered_${b.id}`,
        elements: [],
        createdAt: Date.now(),
        updatedAt: Date.now(),
      },
    ];
const activePageId =
  typeof b.activePageId === "string" && safePages.some((p) => p.id === b.activePageId)
    ? b.activePageId
    : safePages[0].id;

return {
  kind: "notebook" as const,
  pages: safePages,
  activePageId,
  pageSize: (VALID_PAGE_SIZES.includes(b.pageSize as string)
    ? b.pageSize
    : DEFAULT_PAGE_SIZE) as PageSizeId,
  // A zero or negative sheet size would collapse every page to
  // nothing, with no way back from the UI.
  customPageSize:
    b.customPageSize &&
    typeof (b.customPageSize as { width?: unknown }).width === "number" &&
    typeof (b.customPageSize as { height?: unknown }).height === "number" &&
    (b.customPageSize as { width: number }).width > 0 &&
    (b.customPageSize as { height: number }).height > 0
    ? (b.customPageSize as { width: number; height: number })
    : undefined,
  pdfDocIds: Array.isArray(b.pdfDocIds)
    ? (b.pdfDocIds as unknown[]).filter((x): x is string => typeof x === "string")
    : undefined,
};
})();
: { kind: undefined, pages: undefined, activePageId: undefined, pageSize: undefined },
tags: Array.isArray(b.tags) ? b.tags : [],
starred: typeof b.starred === "boolean" ? b.starred : false,
accessedAt: typeof b.accessedAt === "number" ? b.accessedAt : b.createdAt ?? Date.now(),
color: typeof b.color === "string" ? b.color : "#3b82f6",
});
return {
  ...old,
  version: SCHEMA_VERSION,
  folders,
  boards: boards as unknown as Board[],
  activeBoardId: typeof old.activeBoardId === "string" ? old.activeBoardId : null,
  activeFolderId: typeof old.activeFolderId === "string" ? old.activeFolderId : null,
  // Templates carry ink, so they get the same sanitising as a page – a
  // malformed one would otherwise crash the picker that renders it.
  pageTemplates: (Array.isArray(old.pageTemplates) ? (old.pageTemplates as unknown[]) : [])
    .filter(
      (t) =>
        t &&
        typeof (t as { id?: unknown }).id === "string" &&
        typeof (t as { name?: unknown }).name === "string"
    )
    .map((raw) => {
      const t = raw as Record<string, unknown>;
      return {
        id: t.id as string,
        name: t.name as string,
        createdAt: typeof t.createdAt === "number" ? t.createdAt : Date.now(),
        width: typeof t.width === "number" && t.width > 0 ? t.width : 794,
        height: typeof t.height === "number" && t.height > 0 ? t.height : 1123,
        background: cleanBackground(t.background),
        elements: cleanElements(t.elements),
      };
    }),
  // A split pointing at a board that has since been deleted would render
  // an empty pane with no obvious way to close it.

```

```

splitBoardId:
  typeof old.splitBoardId === "string" &&
  (Array.isArray(old.boards) ? old.boards : []).some(
    (b: { id?: unknown }) => b && b.id === old.splitBoardId
  )
  ? old.splitBoardId
  : null,
splitRatio:
  typeof old.splitRatio === "number" && Number.isFinite(old.splitRatio)
  ? Math.max(0.2, Math.min(0.8, old.splitRatio))
  : 0.5,
splitSide: old.splitSide === "right" ? "right" : "left",
splitOrientation: old.splitOrientation === "stacked" ? "stacked" : "side",
// An unknown dock edge would position the toolbar nowhere; fall back.
toolbarPosition: VALID_DOCKS.includes(old.toolbarPosition as string)
  ? old.toolbarPosition
  : "bottom",
customLinkTypes: Array.isArray(old.customLinkTypes) ? old.customLinkTypes : [],
// Favourite pens: drop anything that is not a well-formed pin rather than
// letting a corrupt entry render as an unusable swatch.
pinnedPens: Array.isArray(old.pinnedPens)
  ? old.pinnedPens.filter(
    (p): p is PinnedPen =>
      !!p &&
      typeof p.id === "string" &&
      VALID_PIN_TOOLS.includes(p.tool) &&
      typeof p.color === "string" &&
      Number.isFinite(p.size) &&
      p.size > 0
  )
  : [],
// Preferences. Each is validated on its own so one bad field cannot throw
// away the rest; anything invalid falls back to the default at init().
style: objectOrUndefined<Partial<StyleState>>(old.style),
touchSettings: objectOrUndefined<Partial<TouchSettings>>(old.touchSettings),
tool: typeof old.tool === "string" && VALID_TOOLS.includes(old.tool) ? old.tool : undefined,
snapToGrid: boolOrUndefined(old.snapToGrid),
minimapOpen: boolOrUndefined(old.minimapOpen),
legendOpen: boolOrUndefined(old.legendOpen),
zoomDockOpen: boolOrUndefined(old.zoomDockOpen),
shapeRecognition: boolOrUndefined(old.shapeRecognition),
// App default paper for future boards/pages. Validated like every other
// background; garbage falls back to plain at init().
defaultPaper: cleanBackground(old.defaultPaper) ?? { pattern: "plain" },
pinnedBarOpen: boolOrUndefined(old.pinnedBarOpen),
propsPanelCollapsed: boolOrUndefined(old.propsPanelCollapsed),
toolPropsBarCollapsed: boolOrUndefined(old.toolPropsBarCollapsed),
toolbarTools: cleanToolbarItems(old.toolbarTools),
toolbarHidden: Array.isArray(old.toolbarHidden) ? cleanToolbarItems(old.toolbarHidden) ?? [] : undefined,
recentColors: Array.isArray(old.recentColors)
  ? old.recentColors.filter((c): c is string => typeof c === "string").slice(0, 8)
  : [],
pinnedBarPos:
  old.pinnedBarPos &&
  Number.isFinite(old.pinnedBarPos.x) &&
  Number.isFinite(old.pinnedBarPos.y)
  ? { x: old.pinnedBarPos.x, y: old.pinnedBarPos.y }
  : null,
};
}

// =====
// Debounced autosave
// =====

let saveTimer: ReturnType<typeof setTimeout> | null = null;
const SAVE_DEBOUNCE_MS = 500;

/**
 * While the pen is writing, background saves wait. A save serialises the
 * WHOLE workspace on the main thread (9 MB for a 20-notebook workspace:
 * ~75 ms on a Galaxy Tab S9), and with a 500 ms debounce it landed in the
 * short pauses between words - exactly when the next word began, so its
 * first stroke started late. Saves now run once the pen has been quiet a
 * moment. Leaving the app still saves at once (flushSave / flushHistorySave
 * on visibilitychange, pagehide and beforeunload), so nothing is put at risk.
 */
let quietUntil = 0;

```

```

export function deferSavesWhileWriting(ms = 1500): void {
  quietUntil = Math.max(quietUntil, Date.now() + ms);
}

/**
 * setTimeout that, when due, waits on while the pen is still writing.
 * `setHandle` always holds the live timer, so a flush can still cancel it.
 */
function whenQuiet(
  delay: number,
  fn: () => void,
  setHandle: (t: ReturnType<typeof setTimeout>) => void
): void {
  const run = () => {
    const wait = quietUntil - Date.now();
    if (wait > 0) {
      setHandle(setTimeout(run, wait));
      return;
    }
    fn();
  };
  setHandle(setTimeout(run, delay));
}

/**
 * Schedule a debounced save. Multiple rapid updates (e.g., during drawing)
 * will collapse into a single write at the end of activity.
 */
export function scheduleSave(getState: () => PersistedState): void {
  if (saveTimer) clearTimeout(saveTimer);
  whenQuiet(
    SAVE_DEBOUNCE_MS,
    () => {
      saveTimer = null;
      saveState(getState());
    },
    (t) => {
      saveTimer = t;
    }
  );
}

/** Force an immediate save (e.g., before unload) */
export function flushSave(getState: () => PersistedState): void {
  if (saveTimer) {
    clearTimeout(saveTimer);
    saveTimer = null;
  }
  saveState(getState());
}

// =====
// Undo history persistence – a SEPARATE key, on purpose
// =====
//
// Undo used to die with the session: close the app and every step was gone.
// It is now remembered per notebook (last 100 undo steps) across restarts.
//
// The separation is the safety design, not an accident:
// - the workspace save must never fail because history grew – a quota error
//   on the workspace key can lose boards, while a quota error here only
//   forgets old undo steps (oldest first);
// - history entries are dropped, never migrated: a corrupt entry costs one
//   undo step, and anything structurally wrong is rejected at load.

const HISTORY_KEY = "z-whiteboard-history-v1";
/** Undo steps remembered per the feature request. */
export const HISTORY_PAST_MAX = 100;
export const HISTORY_FUTURE_MAX = 100;
/** History may never threaten the ~5 MB workspace budget. */
const HISTORY_MAX_BYTES = 1_500_000;

export interface SavedHistoryEntry {
  boardId: string;
  /** Document that was live when recorded; absent in older saves (= boardId). */
  activeAt?: string;
  pageId?: string;
  elements: WhiteboardElement[];
}

```

```

    pages?: NotebookPage[];
    /** Epoch ms; orders entries back into one list on load. */
    at?: number;
}

export interface SavedHistory {
    past: SavedHistoryEntry[];
    future: SavedHistoryEntry[];
}

/** A structurally valid entry whose elements cannot crash a restore. */
export function cleanHistoryEntry(e: unknown): SavedHistoryEntry | null {
    if (!e || typeof e !== "object") return null;
    const r = e as Record<string, unknown>;
    if (typeof r.boardId !== "string") return null;

    const hasElements = Array.isArray(r.elements);
    const hasPages = Array.isArray(r.pages);
    if (!hasElements && !hasPages) return null;

    const elements: WhiteboardElement[] = [];
    if (hasElements) {
        for (const el of r.elements as unknown[]) {
            if (
                !el ||
                typeof (el as { id?: unknown }).id !== "string" ||
                typeof (el as { type?: unknown }).type !== "string"
            ) {
                return null;
            }
            elements.push(el as WhiteboardElement);
        }
    }

    let pages: NotebookPage[] | undefined = undefined;
    if (hasPages) {
        const cleanedPages: NotebookPage[] = [];
        for (const p of r.pages as unknown[]) {
            if (
                !p ||
                typeof (p as { id?: unknown }).id !== "string" ||
                !Array.isArray((p as { elements?: unknown }).elements)
            ) {
                return null;
            }
            const pageElements: WhiteboardElement[] = [];
            for (const el of (p as { elements: unknown[] }).elements) {
                if (
                    !el ||
                    typeof (el as { id?: unknown }).id !== "string" ||
                    typeof (el as { type?: unknown }).type !== "string"
                ) {
                    return null;
                }
                pageElements.push(el as WhiteboardElement);
            }
            cleanedPages.push({
                ...(p as NotebookPage),
                elements: pageElements,
            });
        }
        pages = cleanedPages;
    }

    return {
        boardId: r.boardId,
        activeAt: typeof r.activeAt === "string" ? r.activeAt : undefined,
        pageId: typeof r.pageId === "string" ? r.pageId : undefined,
        elements,
        pages,
        at: typeof r.at === "number" ? r.at : 0,
    };
}

/**
 * Write history, oldest-first shrinking until it fits. Never throws into the
 * caller: the worst outcome is forgotten undo steps, never lost boards.
 */

```

```

export function saveHistory(history: SavedHistory): void {
  if (typeof window === "undefined") return;
  let past = history.past.slice(-HISTORY_PAST_MAX);
  let future = history.future.slice(-HISTORY_FUTURE_MAX);
  for (let i = 0; i < 6; i++) {
    let payload: string;
    try {
      payload = JSON.stringify({ version: 1, past, future });
    } catch {
      return; // unserializable – drop it all rather than half a history
    }
    if (payload.length <= HISTORY_MAX_BYTES) {
      try {
        window.localStorage.setItem(HISTORY_KEY, payload);
      } catch {
        // Nearly full (the workspace owns most of the quota): shrink and retry.
        past = past.slice(Math.ceil(past.length / 2));
        future = future.slice(Math.floor(future.length / 2));
        continue;
      }
      return;
    }
    if (past.length <= 1 && future.length === 0) return;
    past = past.slice(Math.ceil(past.length / 2));
    future = future.slice(Math.floor(future.length / 2));
  }
}

export function loadHistory(): SavedHistory | null {
  if (typeof window === "undefined") return null;
  try {
    const raw = window.localStorage.getItem(HISTORY_KEY);
    if (!raw) return null;
    const parsed = JSON.parse(raw) as { past?: unknown; future?: unknown };
    const clean = (list: unknown): SavedHistoryEntry[] =>
      (Array.isArray(list) ? list : [])
        .map(cleanHistoryEntry)
        .filter((e): e is SavedHistoryEntry => e !== null);
    return { past: clean(parsed.past), future: clean(parsed.future) };
  } catch {
    return null;
  }
}

/**
 * What survives a restart: entries for boards that still exist, oldest first,
 * capped to what the session keeps. Pure, so it is unit-tested directly.
 */
export function restorableHistory(
  boardIds: string[],
  loaded: SavedHistory | null
): SavedHistory {
  if (!loaded) return { past: [], future: [] };
  const alive = new Set(boardIds);
  const byTime = (a: SavedHistoryEntry, b: SavedHistoryEntry) => (a.at ?? 0) - (b.at ?? 0);
  return {
    past: loaded.past
      .filter((e) => alive.has(e.boardId))
      .sort(byTime)
      .slice(-HISTORY_PAST_MAX),
    future: loaded.future
      .filter((e) => alive.has(e.boardId))
      .sort(byTime)
      .slice(-HISTORY_FUTURE_MAX),
  };
}

let historyTimer: ReturnType<typeof setTimeout> | null = null;
const HISTORY_DEBOUNCE_MS = 800;

/** Debounced history write after commit/undo/redo. */
export function scheduleHistorySave(getHistory: () => SavedHistory): void {
  if (historyTimer) clearTimeout(historyTimer);
  whenQuiet(
    HISTORY_DEBOUNCE_MS,
    () => {
      historyTimer = null;
      saveHistory(getHistory());
    }
  );
}

```

```

    },
    (t) => {
      historyTimer = t;
    }
  );
}

/** Force an immediate history write (e.g., before unload). */
export function flushHistorySave(getHistory: () => SavedHistory): void {
  if (historyTimer) {
    clearTimeout(historyTimer);
    historyTimer = null;
  }
  saveHistory(getHistory());
}

=====
// === END FILE: src/lib/whiteboard/persistence.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/placement.ts ===
// File 117 of 194 | Lines: 279 | Size: 10183 bytes
=====

/**
 * Where a newly inserted image goes, and how big it is.
 *
 * Before this, the toolbar placed an image at `camera.x/y` at up to 500 units
 * wide with no idea a page existed. In a notebook that meant an image could
 * land half off the sheet, straddling the gap between two pages, or partly
 * outside the part of the page you were looking at – "added to a random
 * place". GoodNotes and Notability both drop a new image in the middle of what
 * you can see, sized to fit comfortably, and never off the paper.
 *
 * So the rule here:
 * 1. The target region is what is VISIBLE – and in a notebook, the visible
 *    part of the active sheet (the sheet sits at the world origin).
 * 2. The image fits inside 60% of that region, keeps its aspect ratio, and
 *    is never scaled UP past its own pixels.
 * 3. It is centred on the point asked for (a paste at the pointer) or on the
 *    region, then pulled fully onto the sheet.
 */

import type { WhiteboardElement } from "../types";
import { getUnionBounds } from "../geometry";

export interface PlaceImageInput {
  naturalWidth: number;
  naturalHeight: number;
  camera: { x: number; y: number; zoom: number };
  viewport: { width: number; height: number };
  /** The active notebook sheet, at the world origin. Null for a canvas. */
  page?: { width: number; height: number } | null;
  /** Preferred centre in world units (e.g. where the pointer is). */
  at?: { x: number; y: number } | null;
}

export interface PlacedRect {
  x: number;
  y: number;
  width: number;
  height: number;
}

/** Fraction of the target region a new image may occupy at most. */
export const IMAGE_FIT_FRACTION = 0.6;
/** Smallest a new image is made, so a tiny icon is still grabbable. */
export const IMAGE_MIN_SIDE = 48;

export function placeImage(input: PlaceImageInput): PlacedRect {
  const { camera, viewport, page } = input;
  const natW = Math.max(1, input.naturalWidth || 1);
  const natH = Math.max(1, input.naturalHeight || 1);
  const zoom = camera.zoom > 0 ? camera.zoom : 1;

  // What the viewport shows, in world units.

```

```

const halfW = viewport.width / 2 / zoom;
const halfH = viewport.height / 2 / zoom;
let region = {
  x: camera.x - halfW,
  y: camera.y - halfH,
  w: halfW * 2,
  h: halfH * 2,
};

if (page) {
  // Only the part of the sheet you can actually see.
  const x0 = Math.max(region.x, 0);
  const y0 = Math.max(region.y, 0);
  const x1 = Math.min(region.x + region.w, page.width);
  const y1 = Math.min(region.y + region.h, page.height);
  region =
    x1 - x0 > 1 && y1 - y0 > 1
      ? { x: x0, y: y0, w: x1 - x0, h: y1 - y0 }
      : { x: 0, y: 0, w: page.width, h: page.height }; // sheet off screen: use all of it
}

// Fit inside 60% of the region, never enlarge past the image's own pixels.
const fit = Math.min(
  (region.w * IMAGE_FIT_FRACTION) / natW,
  (region.h * IMAGE_FIT_FRACTION) / natH,
  1
);
let width = natW * fit;
let height = natH * fit;

// A tiny image is enlarged to a grabbable minimum, aspect preserved, but
// never past the page.
const shortSide = Math.min(width, height);
if (shortSide < IMAGE_MIN_SIDE) {
  const up = IMAGE_MIN_SIDE / shortSide;
  width *= up;
  height *= up;
}
if (page) {
  const cap = Math.min(page.width / width, page.height / height, 1);
  width *= cap;
  height *= cap;
}

const cx = input.at ? input.at.x : region.x + region.w / 2;
const cy = input.at ? input.at.y : region.y + region.h / 2;
let x = cx - width / 2;
let y = cy - height / 2;

if (page) {
  // Fully on the paper — translate, never shrink again.
  x = Math.min(Math.max(x, 0), page.width - width);
  y = Math.min(Math.max(y, 0), page.height - height);
}

return { x, y, width, height };
}

/**
 * Aspect-locked resize for a corner handle.
 *
 * A photo stretched by its corner looks broken, so images keep their
 * proportions: the larger of the two proposed scale factors wins, which makes
 * the corner track the pointer along whichever axis you moved most.
 */
export function lockAspect(
  origW: number,
  origH: number,
  proposedW: number,
  proposedH: number,
  minSide = 20
): { width: number; height: number } {
  const s = Math.max(proposedW / origW, proposedH / origH);
  const floor = minSide / Math.min(origW, origH);
  const scale = Math.max(s, floor);
  return { width: origW * scale, height: origH * scale };
}

```

```

/** Rotation snaps to the nearest right angle within this many radians (~4°). */
export const ROTATE_SNAP = (4 * Math.PI) / 180;

// =====
// Favorite pens bar: where it parks before you have dragged it.
// =====

/** Gap between the bar and the edge of the canvas pane. */
export const PENS_BAR_EDGE_GAP = 16;
/** Height guess used before first layout, when the bar cannot be measured. */
export const PENS_BAR_HEIGHT_ESTIMATE = 300;

export interface CanvasAreaRect {
  left: number;
  top: number;
  right: number;
  bottom: number;
}

/**
 * The bar's home: left edge of the canvas pane, vertically centred.
 *
 * One global spot shared by every board (canvases and notebooks alike) – the
 * position is a single persisted value, not per-board – and a saved drag
 * always wins over this. It only ever applies when there is no saved position
 * (a fresh workspace), so existing placements are left untouched.
 */
export function defaultPensBarPosition(
  area: CanvasAreaRect,
  barHeight: number
): { x: number; y: number } {
  const h =
    Number.isFinite(barHeight) && barHeight > 0 ? barHeight : PENS_BAR_HEIGHT_ESTIMATE;
  const areaH = area.bottom - area.top;
  const y = areaH > h + 16 ? area.top + (areaH - h) / 2 : area.top + 8;
  return { x: area.left + PENS_BAR_EDGE_GAP, y };
}

export function snapRotation(rad: number): number {
  const quarter = Math.PI / 2;
  const nearest = Math.round(rad / quarter) * quarter;
  const snapped = Math.abs(rad - nearest) <= ROTATE_SNAP ? nearest : rad;
  const twoPi = Math.PI * 2;
  return ((snapped % twoPi) + twoPi) % twoPi;
}

// =====
// Shape tools: what a tap (no drag) inserts.
// =====

/**
 * Default tap-inserted shape size, in world units – the same house style as
 * text (220x48) and sticky notes (200x160), which are also fixed world sizes.
 */
export const TAP_SHAPE_WIDTH = 160;
export const TAP_SHAPE_HEIGHT = 120;
/** A tapped line/arrow is this long, horizontal. */
export const TAP_LINE_LENGTH = 160;

/**
 * Drag smaller than this is a tap, not a shape: fingers always wobble a few
 * pixels on contact, and committing the wobble produced thin slivers nobody
 * asked for. Same ballpark as the long-press slop for the same reason.
 */
export const TAP_DRAG_SLOP_PX = 12;

export interface TapShapeRect {
  x: number;
  y: number;
  width: number;
  height: number;
}

/**
 * Where a tapped (not dragged) shape lands, and how big it is.
 *
 * A tap means "put one HERE": the shape is centred on the tap point, never
 * larger than the sheet (capped, then translated fully onto it – the same
 * translate-don't-reshape rule as clampBoxToSheet), and optionally snapped.
 * Before this, a tap created a 2x2 draft that pointer-up discarded, so every

```

```

* shape on a tablet required a finger drag – and finger drags overshoot,
* which is where the "shapes come out huge" reports came from.
*/
export function tapShapeRect(
  tapX: number,
  tapY: number,
  opts: {
    snap?: (v: number) => number;
    /** The active notebook sheet. Null for a canvas (nothing to stay inside). */
    page?: { width: number; height: number } | null;
  } = {}
): TapShapeRect {
  let width = TAP_SHAPE_WIDTH;
  let height = TAP_SHAPE_HEIGHT;
  if (opts.page && opts.page.width > 0 && opts.page.height > 0) {
    // A shape must never be born bigger than the page it lands on.
    const cap = Math.min(1, opts.page.width / width, opts.page.height / height);
    width *= cap;
    height *= cap;
  }
  let x = tapX - width / 2;
  let y = tapY - height / 2;
  if (opts.snap) {
    x = opts.snap(x);
    y = opts.snap(y);
  }
  if (opts.page && opts.page.width > 0 && opts.page.height > 0) {
    x = Math.min(Math.max(x, 0), Math.max(0, opts.page.width - width));
    y = Math.min(Math.max(y, 0), Math.max(0, opts.page.height - height));
  }
  return { x, y, width, height };
}

// -----
// Where a capture lands when it is added to a document
// -----

/**
* On a fresh page: at the top, centred, at its own size - it is a clipping of
* something you read, and blowing it up to fill the sheet would change what
* it looks like. Only shrunk, never enlarged, to fit inside the margins.
*/
export function placeCaptureOnPage(
  page: { width: number; height: number },
  size: { width: number; height: number },
  margin = 48
): PlacedRect {
  const room = {
    width: Math.max(1, page.width - margin * 2),
    height: Math.max(1, page.height - margin * 2),
  };
  const fit = Math.min(1, room.width / Math.max(1, size.width), room.height / Math.max(1, size.height));
  const width = size.width * fit;
  const height = size.height * fit;
  return { x: (page.width - width) / 2, y: Math.min(margin, (page.height - height) / 2), width, height };
}

/**
* On a whiteboard: to the right of everything already there, top-aligned
* with it, so adding to a board you are not looking at never covers its
* contents. An empty board gets it at the origin.
*/
export function placeCaptureBeside(
  elements: WhiteboardElement[],
  size: { width: number; height: number },
  gap = 80
): PlacedRect {
  const b = getUnionBounds(elements.filter((el) => el.type !== "connection"));
  if (!b) return { x: -size.width / 2, y: -size.height / 2, width: size.width, height: size.height };
  return { x: b.x + b.width + gap, y: b.y, width: size.width, height: size.height };
}

=====
// === END FILE: src/lib/whiteboard/placement.ts ===
=====
=====

```

```
// === START FILE: src/lib/whiteboard/pointerHandoff.ts ===
// File 118 of 194 | Lines: 90 | Size: 3723 bytes
=====

/**
 * Split-view pointer hand-off
 * =====
 * When a press lands on the non-live pane, that pane becomes live and the
 * press is replayed on the canvas that replaces it a couple of frames later.
 * Moved here (out of SplitPane.tsx) unchanged so the press/release pairing
 * can be unit-tested: it is exactly the pairing that decides whether a stale
 * finger poisons later gestures.
 */

export interface PressInit extends PointerEventInit {
  pointerId: number;
}

/**
 * Hands a PRESS to the canvas that replaced this pane - and keeps the press
 * and its release paired.
 *
 * The canvas counts fingers: one goes into its map on pointerdown and comes
 * out on pointerup. The replayed pointerdown arrives a few frames late, so a
 * quick tap had usually lifted already - its real release reached the canvas
 * FIRST and removed nothing, then the late replay added a finger that was
 * never lifted. Every later one-finger touch then counted as the second finger
 * of a pinch: scrolling zoomed, drawing and typing did nothing, and closing the
 * split did not help. Measured: a one-finger drag after one quick tap changed
 * the zoom from 0.5 to 0.355; the same drag on a fresh page changed nothing.
 *
 * So: a pointer already released is not replayed at all, and if it lifts after
 * the replay but its release lands outside the canvas, the release is
 * delivered too. Every replayed press gets exactly one release.
 */
export function handOffPress(init: PressInit, clientX: number, clientY: number) {
  let released = false;
  let replayTarget: Element | null = null;
  const stop = () => {
    window.removeEventListener("pointerup", onEnd, true);
    window.removeEventListener("pointercancel", onEnd, true);
  };
  function onEnd(e: PointerEvent) {
    if (e.pointerId !== init.pointerId) return;
    stop();
    released = true;
    if (!replayTarget) return; // lifted before the canvas existed: nothing to pair
    const canvas = replayTarget.closest("[data-wb-canvas-area]") ?? replayTarget;
    if (e.target instanceof Node && canvas.contains(e.target)) return; // it got the real one
    replayTarget.dispatchEvent(
      new PointerEvent(e.type, {
        ...init,
        clientX: e.clientX,
        clientY: e.clientY,
        buttons: 0,
        pressure: 0,
        bubbles: true,
        cancelable: true,
        composed: true,
      })
    );
  }
  window.addEventListener("pointerup", onEnd, true);
  window.addEventListener("pointercancel", onEnd, true);
  // A pointer whose end is never reported must not leave listeners forever.
  setTimeout(stop, 30_000);

  handOff(() => {
    if (released) return null;
    replayTarget = document.elementFromPoint(clientX, clientY);
    return new PointerEvent("pointerdown", { ...init, bubbles: true, cancelable: true, composed: true });
  }, clientX, clientY);
}

/**
 * Replays an event on whatever is now under it, two frames after a focus
 * switch - by then React has swapped this pane for the real canvas, which
 * never saw the original event. `make` may return null to cancel.
 */
```

```

*/
export function handOff(make: () => Event | null, clientX: number, clientY: number) {
  // Exactly once, whichever comes first: two frames (React has certainly
  // swapped the panes by then) or a short timer. requestAnimationFrame alone is
  // paused while the page is hidden, and a replay that never arrives means the
  // stroke you started simply does not happen.
  let done = false;
  const fire = () => {
    if (done) return;
    done = true;
    const ev = make();
    if (ev) document.elementFromPoint(clientX, clientY)?.dispatchEvent(ev);
  };
  requestAnimationFrame(() => requestAnimationFrame(fire));
  setTimeout(fire, 50);
}

```

```

=====
// === END FILE: src/lib/whiteboard/pointerHandoff.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/search.ts ===
// File 119 of 194 | Lines: 678 | Size: 24167 bytes
=====

```

```

/**
 * Search: what is searchable, what is in scope, and in what order it is shown.
 * =====
 * Pure (no React, no store) so the ranking can be unit-tested.
 *
 * What is searchable - and nothing else, so the panel can say so honestly:
 * - note (board / notebook) names, folder names and note tags
 * - text boxes, sticky notes and connection labels, on every notebook page
 * - document text of PDF pages, supplied separately by pdfTextIndex.ts
 * Handwriting is ink with no recognition behind it, shapes carry no text and
 * pictures carry no name, so none of those can be found.
 *
 * Ranking in the wider scopes is by NEARNESS first: the note you are in, then
 * its folder, then the folders around it (sub-folders and the parent chain),
 * then everything else. Someone searching "lecture 3" from inside Physics means
 * Physics' lecture 3, not the one in an old course that happens to match
 * better. Within a group, match quality then recency decide.
 */

import type { Board, Folder, WhiteboardElement } from "../types";
import { getBoardElements, getPages, isNotebook } from "../notebook";
import { getElementBounds, type BoundingBox } from "../geometry";

export type SearchScope = "note" | "folder" | "everywhere";

export type SearchKind = "board" | "folder" | "tag" | "text" | "sticky" | "connection" | "pdf";

/** How close a result is to where you are. Also the group it is shown under. */
export type Nearness = "note" | "folder" | "near" | "elsewhere";

export const NEARNESS_ORDER: Nearness[] = ["note", "folder", "near", "elsewhere"];

/** One searchable string. */
export interface SearchEntry {
  /** Stable across rebuilds, so a highlighted row survives a re-index. */
  id: string;
  kind: SearchKind;
  /** Plain, single-line display text (markdown markers removed). */
  text: string;
  /** `foldText(text)` - what queries are matched against. */
  folded: string;
  boardId: string | null;
  /** The folder it lives in; for a folder entry, the folder itself. */
  folderId: string | null;
  elementId?: string;
  pageId?: string;
  /** 0-based page index in a notebook. */
  pageIndex?: number;
  /** Document text: which match on its page this is (0 = the first). */
  ordinal?: number;
  /** Position on its page, for find-in-page order (top to bottom). */
}

```

```

    y?: number;
    x?: number;
    updatedAt: number;
  }

export interface SearchHit {
  entry: SearchEntry;
  score: number;
  nearness: Nearness;
  /** Match ranges in `entry.folded` coordinates. */
  ranges: Array<number, number>;
  /** Ranges already in display coordinates, when the source supplied them. */
  displayRanges?: Array<number, number>;
}

export interface SearchContext {
  activeBoardId: string | null;
  /**
   * The folder you are "in": the open note's folder, or the sidebar's folder
   * when no note is open. Null is the top level.
   */
  folderId: string | null;
  folders: Folder[];
}

// -----
// Text normalisation
// -----

const COMBINING = /\p{M}/u;
const WORD_CHAR = /\p{L}\p{N}_/u;

/**
 * Case- and accent-insensitive form of a string. Combining marks (accents,
 * Hebrew niqqud and cantillation) are dropped, so "■■■■■■■■" is found by "■■■■"
 * and "café" by "cafe". Returns the map back to the original indices so a
 * match can be highlighted in the text as written.
 */
export function foldWithMap(s: string): { folded: string; map: number[] | null } {
  // Fast path: plain ASCII keeps its length, so no map is needed.
  if (!/[^\x00-\x7f]/.test(s)) return { folded: s.toLowerCase(), map: null };
  let folded = "";
  const map: number[] = [];
  for (let i = 0; i < s.length; i++) {
    const code = s.charCodeAt(i);
    if (code < 0x80) {
      folded += s[i].toLowerCase();
      map.push(i);
      continue;
    }
    // Keep a surrogate pair together (emoji etc.); it has no case or accent.
    if (code >= 0xd800 && code <= 0xdbff && i + 1 < s.length) {
      folded += s[i] + s[i + 1];
      map.push(i, i + 1);
      i++;
      continue;
    }
    const ch = s[i];
    if (COMBINING.test(ch)) continue;
    let f = ch.normalize("NFD").replace(/\p{M}/gu, "").toLowerCase();
    // A few letters lowercase to two characters (e.g. "■"); keep the length so
    // the map stays one-to-one.
    if (f.length !== 1) f = ch.toLowerCase().length === 1 ? ch.toLowerCase() : ch;
    folded += f;
    map.push(i);
  }
  return { folded, map };
}

export function foldText(s: string): string {
  return foldWithMap(s).folded;
}

/**
 * Text boxes and sticky notes hold markdown. Search the text as the canvas
 * SHOWS it - "***Exam** date" must be found by "exam date" - and on one line,
 * because a result row is one line.
 */

```

```

export function plainText(markdown: string): string {
  return markdown
    .replace(/<\?u>/gi, "")
    .replace(/^\\s{0,3}#{1,6}\\s+/gm, "")
    .replace(/\\s*[-*]\\s*\\[\\ xX\\]\\s*/gm, "")
    .replace(/\\s*[-*+]\\s+/gm, "")
    .replace(/\\s*>\\s?/gm, "")
    .replace(/\\s*|_|~|=|`/g, "")
    .replace(/\\[\\([\\^\\]]*)\\:(?\\[\\^\\]]*)\\]\\]/g, (_, target: string, alias?: string) => alias || target)
    .replace(/\\s+/g, " ")
    .trim();
}

// -----
// Index
// -----

function entry(partial: Omit<SearchEntry, "folded">): SearchEntry {
  return { ...partial, folded: foldText(partial.text) };
}

/** The searchable text of one element, or null for ink, shapes and pictures. */
function elementText(el: WhiteboardElement): { kind: SearchKind; text: string } | null {
  if (el.type === "text" || el.type === "sticky") {
    const text = plainText(typeof el.text === "string" ? el.text : "");
    return text ? { kind: el.type, text } : null;
  }
  if (el.type === "connection") {
    const text = typeof el.label === "string" ? el.label.replace(/\\s+/g, " ").trim() : "";
    return text ? { kind: "connection", text } : null;
  }
  return null;
}

function elementEntries(
  board: Board,
  elements: WhiteboardElement[],
  page?: { id: string; index: number }
): SearchEntry[] {
  const out: SearchEntry[] = [];
  for (const el of elements ?? []) {
    const t = elementText(el);
    if (!t) continue;
    const b = focusBounds(el, elements);
    out.push(
      entry({
        id: `${board.id}:${el.id}`,
        kind: t.kind,
        text: t.text,
        boardId: board.id,
        folderId: board.folderId ?? null,
        elementId: el.id,
        pageId: page?.id,
        pageIndex: page?.index,
        x: Number.isFinite(b.x) ? b.x : 0,
        y: Number.isFinite(b.y) ? b.y : 0,
        updatedAt: el.updatedAt || board.updatedAt || 0,
      })
    );
  }
  return out;
}

// Re-indexing only what changed. Boards, pages and element arrays are replaced
// (never mutated) on every edit, so an unchanged one keeps its identity and
// its cached entries - with the panel open, a stroke on one page re-reads that
// page, not 900 of them.
const pageCache = new WeakMap<object, SearchEntry[]>();
const boardCache = new WeakMap<Board, SearchEntry[]>();

function boardEntries(board: Board): SearchEntry[] {
  const cached = boardCache.get(board);
  if (cached) return cached;
  const out: SearchEntry[] = [];
  const recency = board.accessedAt || board.updatedAt || board.createdAt || 0;
  out.push(
    entry({
      id: `board:${board.id}`,

```

```

    kind: "board",
    text: board.name || "Untitled",
    boardId: board.id,
    folderId: board.folderId ?? null,
    updatedAt: recency,
  })
);
for (const tag of new Set(board.tags ?? [])) {
  if (typeof tag !== "string" || !tag.trim()) continue;
  out.push(
    entry({
      id: `tag:${board.id}:${tag}`,
      kind: "tag",
      text: `#${tag.replace(/^#/,"")}`,
      boardId: board.id,
      folderId: board.folderId ?? null,
      updatedAt: recency,
    })
  );
}
if (isNotebook(board)) {
  getPages(board).forEach((page, index) => {
    // The cache key is the page object, but the entries carry the index -
    // so a page moved to a new position must not reuse them.
    const key = page.elements ?? page;
    let pageEntries = pageCache.get(key);
    if (!pageEntries || pageEntries.some((e) => e.pageIndex !== index || e.folderId !== (board.folderId ?? null) || e.pageId !== page.id)) {
      pageEntries = elementEntries(board, page.elements ?? [], { id: page.id, index });
      pageCache.set(key, pageEntries);
    }
    out.push(...pageEntries);
  });
} else {
  out.push(...elementEntries(board, getBoardElements(board)));
}
boardCache.set(board, out);
return out;
}

export function buildSearchIndex(boards: Board[], folders: Folder[]): SearchEntry[] {
  const out: SearchEntry[] = [];
  for (const f of folders) {
    out.push(
      entry({
        id: `folder:${f.id}`,
        kind: "folder",
        text: f.name || "Untitled folder",
        boardId: null,
        folderId: f.id,
        updatedAt: f.createdAt || 0,
      })
    );
  }
  for (const b of boards) out.push(...boardEntries(b));
  return out;
}

/**
 * Where to point the camera for an element. A connection's own x/y is not
 * where it is drawn - it runs between its two ends - so aim at the middle of
 * the line, which is where its label sits.
 */
export function focusBounds(el: WhiteboardElement, siblings: WhiteboardElement[]): BoundingBox {
  if (el.type === "connection") {
    const a = siblings.find((s) => s.id === el.source?.elementId);
    const b = siblings.find((s) => s.id === el.target?.elementId);
    if (a && b) {
      const ba = getElementBounds(a);
      const bb = getElementBounds(b);
      const cx = (ba.x + ba.width / 2 + bb.x + bb.width / 2) / 2;
      const cy = (ba.y + ba.height / 2 + bb.y + bb.height / 2) / 2;
      return { x: cx - 60, y: cy - 16, width: 120, height: 32 };
    }
  }
  return getElementBounds(el);
}

```

```
// -----
// Folders
// -----

/** Every folder below `folderId`, at any depth. Cycle-safe. */
export function descendantFolderIds(folderId: string | null, folders: Folder[]): Set<string> {
  const out = new Set<string>();
  const children = new Map<string | null, string[]>();
  for (const f of folders) {
    const list = children.get(f.parentId ?? null) ?? [];
    list.push(f.id);
    children.set(f.parentId ?? null, list);
  }
  const stack = [...(children.get(folderId) ?? [])];
  while (stack.length) {
    const id = stack.pop() as string;
    if (out.has(id) || id === folderId) continue;
    out.add(id);
    stack.push(...(children.get(id) ?? []));
  }
  return out;
}

/** The folders above `folderId`, nearest first. Cycle-safe. */
export function ancestorFolderIds(folderId: string | null, folders: Folder[]): string[] {
  const byId = new Map(folders.map((f) => [f.id, f]));
  const out: string[] = [];
  const seen = new Set<string>(folderId ? [folderId] : []);
  let cursor = folderId ? byId.get(folderId)?.parentId ?? null : null;
  while (cursor && !seen.has(cursor)) {
    seen.add(cursor);
    out.push(cursor);
    cursor = byId.get(cursor)?.parentId ?? null;
  }
  return out;
}

/** Folder names from the top down to `folderId`, e.g. ["Uni", "Physics"]. */
export function folderPath(folderId: string | null, folders: Folder[]): string[] {
  if (!folderId) return [];
  const byId = new Map(folders.map((f) => [f.id, f]));
  const own = byId.get(folderId);
  if (!own) return [];
  return [
    ...ancestorFolderIds(folderId, folders)
      .reverse()
      .map((id) => byId.get(id)?.name ?? ""),
    own.name,
  ];
}

// -----
// Scope and nearness
// -----

const CONTENT_KINDS: ReadonlySet<SearchKind> = new Set(["text", "sticky", "connection", "pdf"]);

/**
 * Precomputed folder relations for one search, so a 20 000-entry scan does
 * not walk the folder tree per entry.
 */
export interface ScopeInfo {
  ctx: SearchContext;
  below: Set<string>;
  above: Set<string>;
}

export function scopeInfo(ctx: SearchContext): ScopeInfo {
  return {
    ctx,
    below: descendantFolderIds(ctx.folderId, ctx.folders),
    above: new Set(ancestorFolderIds(ctx.folderId, ctx.folders)),
  };
}

export function nearnessOf(e: SearchEntry, info: ScopeInfo): Nearness {
  const { ctx } = info;
  if (e.kind !== "folder" && ctx.activeBoardId && e.boardId === ctx.activeBoardId) return "note";
}
```

```

    if (e.folderId === ctx.folderId) return "folder";
    // At the top level every folder is "below", which would make everything
    // near; there, only the top level itself counts as close.
    if (ctx.folderId === null) return "elsewhere";
    if (e.folderId && (info.below.has(e.folderId) || info.above.has(e.folderId))) return "near";
    return "elsewhere";
}

export function inScope(e: SearchEntry, scope: SearchScope, info: ScopeInfo): boolean {
    const { ctx } = info;
    if (scope === "everywhere") return true;
    if (scope === "note") {
        // Find-in-note: the note's content. Its own name and tags are not "in" it.
        return !!ctx.activeBoardId && e.boardId === ctx.activeBoardId && CONTENT_KINDS.has(e.kind);
    }
    // This folder, sub-folders included - the way Finder and Explorer search a
    // folder. The folder's own name is the scope, not a result.
    if (e.kind === "folder") return e.folderId !== null && info.below.has(e.folderId) && ctx.folderId !== null;
    if (e.folderId === ctx.folderId) return true;
    // The top level has everything below it; "this folder" there means the
    // loose notes at the top, otherwise it would just be "Everywhere".
    return ctx.folderId !== null && e.folderId !== null && info.below.has(e.folderId);
}

/** Whether a board's content is in scope - used to decide whose PDF text to search. */
export function boardInScope(board: Board, scope: SearchScope, info: ScopeInfo): boolean {
    return inScope(
        { id: "", kind: "pdf", text: "", folded: "", boardId: board.id, folderId: board.folderId ?? null, updatedA
          t: 0 },
        scope,
        info
    );
}

// -----
// Matching
// -----

/** Folded query words. Quotes are ignored rather than half-supported. */
export function parseQuery(query: string): string[] {
    return foldText(query.replace(/["']/g, " "))
        .split(/\s+/)
        .filter(Boolean);
}

/** Match strength of one occurrence: 3 whole word, 2 word start, 1 inside a word. */
function occurrenceQuality(text: string, at: number, len: number): number {
    const startOk = at === 0 || !WORD_CHAR.test(text[at - 1]);
    if (!startOk) return 1;
    const end = at + len;
    const endOk = end >= text.length || !WORD_CHAR.test(text[end]);
    return endOk ? 3 : 2;
}

/** The best occurrence of `term` in `text`, or null. */
function bestOccurrence(text: string, term: string): { at: number; quality: number } | null {
    let best: { at: number; quality: number } | null = null;
    let from = 0;
    // A handful of occurrences is enough to find a whole-word one; scanning
    // every "e" in a long note would cost more than it could change.
    for (let n = 0; n < 24; n++) {
        const at = text.indexOf(term, from);
        if (at === -1) break;
        const quality = occurrenceQuality(text, at, term.length);
        if (!best || quality > best.quality) best = { at, quality };
        if (quality === 3) break;
        from = at + 1;
    }
    return best;
}

/** Title-like results outrank content: finding a note by its name is the usual intent. */
const KIND_WEIGHT: Record<SearchKind, number> = {
    board: 20,
    folder: 20,
    tag: 10,
    text: 0,
    sticky: 0,

```

```

    connection: 0,
    pdf: 0,
  };

/**
 * Scores an entry against the query words. Every word must occur (AND); the
 * weakest word decides the quality, so "exam physics" is not ranked as a
 * whole-word hit because "exam" was one. The whole phrase occurring in order,
 * and the text being exactly the query, both add a little on top.
 */
export function matchEntry(
  e: SearchEntry,
  terms: string[]
): { score: number; ranges: Array<[number, number]> } | null {
  if (terms.length === 0) return null;
  const text = e.folded;
  const phrase = terms.join(" ");
  const ranges: Array<[number, number]> = [];
  let quality = 3;

  const whole = terms.length > 1 ? bestOccurrence(text, phrase) : null;
  if (whole) {
    quality = whole.quality;
    ranges.push([whole.at, phrase.length]);
  } else {
    for (const term of terms) {
      const occ = bestOccurrence(text, term);
      if (!occ) return null;
      quality = Math.min(quality, occ.quality);
      ranges.push([occ.at, term.length]);
    }
    ranges.sort((a, b) => a[0] - b[0]);
  }

  let score = KIND_WEIGHT[e.kind] + quality;
  if (whole) score += 0.5;
  if (text === phrase || (e.kind === "tag" && text === `${phrase.replace(/^\#/ , "")}`)) score += 2;
  return { score, ranges };
}

// -----
// Search and grouping
// -----

function documentOrder(a: SearchEntry, b: SearchEntry): number {
  return (
    (a.pageIndex ?? 0) - (b.pageIndex ?? 0) ||
    (a.y ?? 0) - (b.y ?? 0) ||
    (a.x ?? 0) - (b.x ?? 0)
  );
}

/**
 * All hits for `query` in `scope`, in display order.
 *
 * "This note" is find-in-page: top to bottom, page by page, the order you
 * would step through. The wider scopes go nearest-first, then best match,
 * then most recently touched.
 */
export function searchEntries(
  entries: SearchEntry[],
  query: string,
  scope: SearchScope,
  ctx: SearchContext
): SearchHit[] {
  const terms = parseQuery(query);
  if (terms.length === 0) return [];
  const info = scopeInfo(ctx);
  const hits: SearchHit[] = [];
  for (const e of entries) {
    if (!inScope(e, scope, info)) continue;
    const m = matchEntry(e, terms);
    if (!m) continue;
    hits.push({ entry: e, score: m.score, ranges: m.ranges, nearness: nearnessOf(e, info) });
  }
  return sortHits(hits, scope);
}

```

```

export function sortHits(hits: SearchHit[], scope: SearchScope): SearchHit[] {
  if (scope === "note") return hits.sort((a, b) => documentOrder(a.entry, b.entry));
  const rank = (n: Nearness) => NEARNESS_ORDER.indexOf(n);
  return hits.sort(
    (a, b) =>
      rank(a.nearness) - rank(b.nearness) ||
      b.score - a.score ||
      b.entry.updatedAt - a.entry.updatedAt ||
      documentOrder(a.entry, b.entry)
  );
}

export interface SearchGroup {
  id: Nearness;
  label: string;
  hits: SearchHit[];
}

/** Group headers, so the nearest-first order explains itself. */
export function groupLabel(id: Nearness, scope: SearchScope, ctx: SearchContext): string {
  const folderName = ctx.folderId
    ? ctx.folders.find((f) => f.id === ctx.folderId)?.name || "this folder"
    : "Top level";
  switch (id) {
    case "note":
      return "In this note";
    case "folder":
      return ctx.folderId ? `In ${folderName}` : "Top level";
    case "near":
      return scope === "folder" ? "In sub-folders" : "Sub-folders & parent";
    default:
      return "Elsewhere";
  }
}

/** Splits sorted hits into their nearness groups, keeping the order. */
export function groupHits(hits: SearchHit[], scope: SearchScope, ctx: SearchContext): SearchGroup[] {
  const groups = new Map<Nearness, SearchHit[]>();
  for (const h of hits) {
    const list = groups.get(h.nearness) ?? [];
    list.push(h);
    groups.set(h.nearness, list);
  }
  return NEARNESS_ORDER.filter((id) => groups.has(id)).map((id) => ({
    id,
    label: groupLabel(id, scope, ctx),
    hits: groups.get(id) as SearchHit[],
  }));
}

/** Whether opening a hit lands on a spot in a note (and so can be stepped through). */
export function isLocatable(h: SearchHit): boolean {
  return CONTENT_KINDS.has(h.entry.kind);
}

// -----
// Snippets
// -----

export interface Snippet {
  text: string;
  /** Highlight ranges in `text`. */
  ranges: Array<number, number>;
}

/**
 * A readable window of the text around the first match, with the matches
 * mapped back from folded to written positions.
 */
export function makeSnippet(hit: SearchHit, width = 96): Snippet {
  const text = hit.entry.text;
  let ranges = hit.displayRanges;
  if (!ranges) {
    const { map } = foldWithMap(text);
    ranges = hit.ranges.map(([at, len]) => {
      if (!map) return [at, len] as [number, number];
      const start = map[at] ?? text.length;
      const lastFolded = Math.min(at + len - 1, map.length - 1);

```

```

    const end = (map[lastFolded] ?? start) + 1;
    return [start, Math.max(1, end - start)] as [number, number];
  });
}
if (text.length <= width) return { text, ranges };

const first = ranges[0]?.[0] ?? 0;
// Some context before the match, but mostly after it: people read on.
let start = Math.max(0, first - Math.floor(width * 0.3));
// Start on a word boundary so the snippet does not open mid-word.
if (start > 0) {
  const space = text.indexOf(" ", start);
  if (space !== -1 && space < first) start = space + 1;
}
const end = Math.min(text.length, start + width);
const lead = start > 0 ? "..." : "";
const tail = end < text.length ? "...": "";
const shift = lead.length - start;
return {
  text: lead + text.slice(start, end) + tail,
  ranges: ranges
    .filter(([at]) => at >= start && at < end)
    .map(([at, len]) => [at + shift, Math.min(len, end - at)] as [number, number]),
};
}

/** Splits text into plain and highlighted parts for rendering. */
export function highlightParts(snippet: Snippet): Array<{ text: string; hit: boolean }> {
  const parts: Array<{ text: string; hit: boolean }> = [];
  const ranges = [...snippet.ranges].sort((a, b) => a[0] - b[0]);
  let pos = 0;
  for (const [at, len] of ranges) {
    if (at < pos) continue; // overlapping ranges: the first one wins
    if (at > pos) parts.push({ text: snippet.text.slice(pos, at), hit: false });
    parts.push({ text: snippet.text.slice(at, at + len), hit: true });
    pos = at + len;
  }
  if (pos < snippet.text.length) parts.push({ text: snippet.text.slice(pos), hit: false });
  return parts;
}

// -----
// Document text (PDF) hits
// -----

/**
 * Turns a PDF text hit into a search hit, so it ranks and groups like any
 * other. The snippet is the text layer's own; its match range is trusted for
 * highlighting, and re-matched only to judge quality (whole word or not).
 */
export function pdfHit(
  board: Board,
  raw: { pageIndex: number; pageId?: string; snippet: string; matchStart: number; matchLength: number; ordinal
    ? : number },
  query: string,
  ordinal: number,
  info: ScopeInfo
): SearchHit {
  const pages = getPages(board);
  const e = entry({
    id: `pdf:${board.id}:${raw.pageIndex}:${raw.ordinal}`,
    kind: "pdf",
    text: raw.snippet.replace(/\s+/g, " "),
    boardId: board.id,
    folderId: board.folderId ?? null,
    pageId: raw.pageId ?? pages[raw.pageIndex]?.id,
    pageIndex: raw.pageIndex,
    ordinal: raw.ordinal,
    // Document text sits under the page's ink and notes: list it first on
    // its page, in reading order.
    y: -1e9 + ordinal,
    x: 0,
    updatedAt: board.updatedAt || 0,
  });
  const m = matchEntry(e, parseQuery(query));
  const start = Math.max(0, Math.min(raw.matchStart, e.text.length));
  return {
    entry: e,

```

```

    score: m ? m.score : 1,
    ranges: m ? m.ranges : [[start, raw.matchLength]],
    displayRanges: [[start, Math.max(1, Math.min(raw.matchLength, e.text.length - start))]],
    nearness: nearnessOf(e, info),
  };
}

```

```

=====
// === END FILE: src/lib/whiteboard/search.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/shapeRecognition.ts ===
// File 120 of 194 | Lines: 894 | Size: 34400 bytes
=====

```

```

/**
 * Shape recognition
 * =====
 * Turns a hand-drawn stroke into the shape it was meant to be - a circle, an
 * ellipse, a straight line, a triangle, a rectangle or square, or another
 * polygon - at the same place and the same size. Anything it is not sure of is
 * left exactly as drawn: a wrong guess destroys what you wrote, a missed one
 * costs nothing.
 *
 * How it decides:
 * - A line is an open stroke whose ends are almost as far apart as the whole
 *   path is long, and which never strays far from the straight chord.
 * - A closed stroke (the pen came back near where it started) is cut to one
 *   loop, resampled evenly, and its CORNERS are found: places where the
 *   direction turns sharply within a short stretch. A drawn circle turns
 *   steadily and has none; a square has four.
 * - 3-6 corners with straight sides between them -> a polygon, its corners
 *   placed where the fitted sides meet (not where the pen happened to turn).
 *   Four right angles -> a rectangle, or a square when the sides match.
 * - No corners and a good ellipse fit -> an ellipse, or a circle when the
 *   axes match.
 * - Near-horizontal / vertical lines, rectangles and ellipses snap straight.
 *
 * Points are [x, y, pressure] in the stroke's own coordinates, in and out.
 */

export type ShapeKind =
  | "line"
  | "circle"
  | "ellipse"
  | "triangle"
  | "square"
  | "rectangle"
  | "quadrilateral"
  | "pentagon"
  | "hexagon";

export interface RecognizedShape {
  kind: ShapeKind;
  /** Dense points along the perfect shape, closed shapes ending where they start. */
  points: [number, number, number][];
  /**
   * The shape's key points: a line's two ends, a polygon's corners (none for
   * an ellipse). What the next line you draw can snap onto.
   */
  vertices: [number, number][];
}

type P = [number, number];
type Pt = [number, number, number];

const SHAPE_LABELS: Record<ShapeKind, string> = {
  line: "Line",
  circle: "Circle",
  ellipse: "Ellipse",
  triangle: "Triangle",
  square: "Square",
  rectangle: "Rectangle",
  quadrilateral: "Shape",
  pentagon: "Pentagon",
  hexagon: "Hexagon",

```

```

};

export function shapeLabel(kind: ShapeKind): string {
  return SHAPE_LABELS[kind];
}

// -----
// Geometry helpers
// -----

const dist = (a: P, b: P) => Math.hypot(a[0] - b[0], a[1] - b[1]);

function pathLength(pts: P[]): number {
  let n = 0;
  for (let i = 1; i < pts.length; i++) n += dist(pts[i - 1], pts[i]);
  return n;
}

/** `n` points evenly spaced along the path. */
function resample(pts: P[], n: number): P[] {
  const total = pathLength(pts);
  if (total === 0) return [pts[0]];
  const step = total / (n - 1);
  const out: P[] = [pts[0]];
  let acc = 0;
  let prev = pts[0];
  for (let i = 1; i < pts.length && out.length < n; i++) {
    let cur = pts[i];
    let d = dist(prev, cur);
    while (acc + d >= step && out.length < n) {
      const t = (step - acc) / d;
      const q: P = [prev[0] + (cur[0] - prev[0]) * t, prev[1] + (cur[1] - prev[1]) * t];
      out.push(q);
      prev = q;
      d = dist(prev, cur);
      acc = 0;
    }
    acc += d;
    prev = cur;
    cur = pts[i];
  }
  while (out.length < n) out.push(pts[pts.length - 1]);
  return out;
}

function distToSegment(p: P, a: P, b: P): number {
  const dx = b[0] - a[0];
  const dy = b[1] - a[1];
  const len2 = dx * dx + dy * dy;
  if (len2 === 0) return dist(p, a);
  const t = Math.max(0, Math.min(1, ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy) / len2));
  return dist(p, [a[0] + dx * t, a[1] + dy * t]);
}

function polygonArea(pts: P[]): number {
  let a = 0;
  for (let i = 0; i < pts.length; i++) {
    const [x1, y1] = pts[i];
    const [x2, y2] = pts[(i + 1) % pts.length];
    a += x1 * y2 - x2 * y1;
  }
  return Math.abs(a) / 2;
}

/** Angle between two directions, 0..PI. */
function turn(a: P, b: P, c: P): number {
  const v1x = b[0] - a[0];
  const v1y = b[1] - a[1];
  const v2x = c[0] - b[0];
  const v2y = c[1] - b[1];
  const l1 = Math.hypot(v1x, v1y);
  const l2 = Math.hypot(v2x, v2y);
  if (l1 === 0 || l2 === 0) return 0;
  const cos = Math.max(-1, Math.min(1, (v1x * v2x + v1y * v2y) / (l1 * l2)));
  return Math.acos(cos);
}

/** A best-fit line through points: a point on it and its unit direction. */

```

```

function fitLine(pts: P[]): { c: P; d: P } {
  const n = pts.length;
  const cx = pts.reduce((s, p) => s + p[0], 0) / n;
  const cy = pts.reduce((s, p) => s + p[1], 0) / n;
  let sxx = 0;
  let syx = 0;
  let syy = 0;
  for (const [x, y] of pts) {
    sxx += (x - cx) * (x - cx);
    syx += (x - cx) * (y - cy);
    syy += (y - cy) * (y - cy);
  }
  const angle = 0.5 * Math.atan2(2 * syx, sxx - syy);
  return { c: [cx, cy], d: [Math.cos(angle), Math.sin(angle)] };
}

function intersect(l1: { c: P; d: P }, l2: { c: P; d: P }): P | null {
  const det = l1.d[0] * l2.d[1] - l1.d[1] * l2.d[0];
  if (Math.abs(det) < 1e-9) return null;
  const t = ((l2.c[0] - l1.c[0]) * l2.d[1] - (l2.c[1] - l1.c[1]) * l2.d[0]) / det;
  return [l1.c[0] + l1.d[0] * t, l1.c[1] + l1.d[1] * t];
}

/** Snaps an angle (radians) to the nearest multiple of `step` when within `tol`. */
function snapAngle(a: number, step: number, tol: number): number {
  const k = Math.round(a / step);
  return Math.abs(a - k * step) <= tol ? k * step : a;
}

const DEG = Math.PI / 180;

// -----
// Output builders (dense points, so the pen draws them evenly and the pixel
// eraser can cut them like any stroke)
// -----

function densify(corners: P[], closed: boolean, spacing: number, pressure: number): Pt[] {
  const out: Pt[] = [];
  const ring = closed ? [...corners, corners[0]] : corners;
  for (let i = 0; i < ring.length - 1; i++) {
    const a = ring[i];
    const b = ring[i + 1];
    const n = Math.max(1, Math.ceil(dist(a, b) / spacing));
    for (let k = 0; k < n; k++) {
      const t = k / n;
      out.push([a[0] + (b[0] - a[0]) * t, a[1] + (b[1] - a[1]) * t, pressure]);
    }
  }
  const end = ring[ring.length - 1];
  out.push([end[0], end[1], pressure]);
  return out;
}

function ellipsePoints(c: P, a: number, b: number, rot: number, start: number, pressure: number): Pt[] {
  const n = Math.max(48, Math.min(160, Math.round((Math.PI * (a + b)) / 3)));
  const out: Pt[] = [];
  const cos = Math.cos(rot);
  const sin = Math.sin(rot);
  for (let i = 0; i <= n; i++) {
    const t = start + (i / n) * Math.PI * 2;
    const u = a * Math.cos(t);
    const v = b * Math.sin(t);
    out.push([c[0] + u * cos - v * sin, c[1] + u * sin + v * cos, pressure]);
  }
  return out;
}

// -----
// Recognition
// -----

export interface RecognizeOptions {
  /** Smallest shape (bounding-box diagonal) worth recognising, in the points' units. */
  minSize?: number;
  /** Filled with the measurements behind the decision (tests and tuning). */
  explain?: Record<string, unknown>;
}

```

```

/**
 * The shape a stroke was meant to be, or null when it is not clearly one.
 * `points` are the finished stroke's points (hooks already trimmed).
 */
export function recognizeShape(points: Pt[], opts: RecognizeOptions = {}): RecognizedShape | null {
  if (!points || points.length < 5) return null;
  const raw: P[] = points.map((p) => [p[0], p[1]]);
  const pressure = median(points.map((p) => (typeof p[2] === "number" && p[2] > 0 ? p[2] : 0.5)));

  let minX = Infinity;
  let minY = Infinity;
  let maxX = -Infinity;
  let maxY = -Infinity;
  for (const [x, y] of raw) {
    minX = Math.min(minX, x);
    minY = Math.min(minY, y);
    maxX = Math.max(maxX, x);
    maxY = Math.max(maxY, y);
  }
  const diag = Math.hypot(maxX - minX, maxY - minY);
  if (diag < (opts.minSize ?? 20)) return null;

  const L = pathLength(raw);
  if (L === 0) return null;
  const ex = opts.explain;
  if (ex) Object.assign(ex, { diag, L, gapOverL: dist(raw[0], raw[raw.length - 1]) / L });

  // ---- Line -----
  const line = tryLine(raw, L, pressure) ?? tryLine(...trimHooks(raw), pressure);
  if (ex) ex.line = !!line;
  if (line) return { kind: "line", points: line, vertices: [[line[0][0], line[0][1]], [line[line.length - 1][0], line[line.length - 1][1]]] };

  // ---- One closed loop -----
  // Hands rarely end exactly where they began: they stop short, run on past
  // the start (a tail), trace back over the first side, or cross over it.
  // The loop is the stretch between the pair of points - one early, one
  // late - where the stroke comes back closest to itself.
  const lap = findLoop(raw, L);
  if (ex) Object.assign(ex, { lapGap: lap ? lap.d / diag : null, lapShare: lap ? lap.len / L : null });
  // Stopping short or running just past the start leaves a gap; cutting off
  // a long tail (a retrace, a crossing) is trusted only where the stroke
  // really meets itself - otherwise a spiral's turns would pass as a lap.
  const lapTol = lap && lap.len >= L * 0.92 ? Math.max(diag * 0.22, lap.len * 0.08) : diag * 0.05;
  if (!lap || lap.d > lapTol) return null;
  const loop = raw.slice(lap.i, lap.j + 1);
  const first = loop[0];
  const N = 96;
  const ring = resample(loop, N + 1).slice(0, N); // closed ring, no repeat
  if (ring.length < N) return null;
  const area = polygonArea(ring);
  // Measured on the loop itself: a tail cut off must not make it look bigger.
  const ringDiag = (() => {
    const xs = ring.map((p) => p[0]);
    const ys = ring.map((p) => p[1]);
    return Math.hypot(Math.max(...xs) - Math.min(...xs), Math.max(...ys) - Math.min(...ys));
  })();
  // A scribble back and forth over itself encloses nothing.
  if (area < ringDiag * ringDiag * 0.06) return null;

  // ---- Corners -----
  const W = 4; // samples either side: ~8% of the perimeter
  const turning = ring.map((_, i) => turn(ring[(i - W + N) % N], ring[i], ring[(i + W) % N]));
  // The same over half the stretch. A corner turns nearly all of its angle
  // in that short span; the tip of a thin ellipse spreads its turn out, so
  // there the short span carries only about half.
  const W2 = 2;
  const tight = ring.map((_, i) => turn(ring[(i - W2 + N) % N], ring[i], ring[(i + W2) % N]));
  const findCorners = (sharpness: number): number[] => {
    const found: number[] = [];
    for (let i = 0; i < N; i++) {
      const t = turning[i];
      if (t < 50 * DEG) continue;
      const sharp = Math.max(tight[i], tight[(i + 1) % N], tight[(i - 1 + N) % N]);
      if (sharp < sharpness * t) continue;
      // A local maximum of turning, and not the second peak of the same corner.
      let isMax = true;
      for (let k = -W; k <= W; k++) {

```

```

    if (k === 0) continue;
    const j = (i + k + N) % N;
    if (turning[j] > t || (turning[j] === t && k < 0)) {
      isMax = false;
      break;
    }
  }
  if (isMax) found.push(i);
}
return found;
};
/**
 * The one corner the turning test missed in a box: three corners were
 * found and the fourth was drawn round and wide. The point farthest from
 * the triangle of the three is that corner. Only ever this one step: a
 * heart or a D shows two corners, and "finding" more on its curves would
 * turn it into a polygon it never was.
 */
// Turning over a wider stretch: a corner drawn round still turns like a
// corner here (~90 deg), the middle of an arc (a D's round, a heart's
// lobe) only gently.
const WW = 8;
const wide = ring.map((_, i) => turn(ring[(i - WW + N) % N], ring[i], ring[(i + WW) % N]));
const tryGrowing = (start: number[], ellipseErr: number): RecognizedShape | null => {
  if (start.length !== 3) return null;
  const cs = [...start].sort((p, q) => p - q);
  let added = 0;
  while (cs.length <= 4) {
    if (cs.length >= 3) {
      const r = tryPolygon(cs, ellipseErr);
      if (r) return r;
    }
    if (cs.length === 4) break;
    const outline = cs.length >= 2 ? cs.map((i) => ring[i]) : [ring[0], ring[Math.floor(N / 2)]];
    let far = -1;
    let farD = 0;
    for (let i = 0; i < N; i++) {
      if (cs.some(c => Math.min(Math.abs(c - i), N - Math.abs(c - i)) < 5)) continue;
      let d = Infinity;
      for (let k = 0; k < outline.length; k++) d = Math.min(d, distToSegment(ring[i], outline[k], outline[(k
        + 1) % outline.length]));
      if (d > farD) {
        farD = d;
        far = i;
      }
    }
    if (far < 0 || farD < ringDiag * 0.04) break;
    if (ex) (ex.grow = (ex.grow as unknown[] | undefined) ?? []).push({ k: cs.length, wide: wide[far] / DEG,
      farD: farD / ringDiag });
    // It must still turn like a corner over a wider stretch.
    if (added >= 1 || wide[far] < 40 * DEG) break;
    if (cs.length < 2 && !cs.includes(0)) cs.push(0);
    cs.push(far);
    added++;
    cs.sort((p, q) => p - q);
  }
  return null;
};
const tryPolygon = (cs: number[], ellipseErr: number): RecognizedShape | null => {
  if (cs.length < 3 || cs.length > 6) return null;
  const poly = fitPolygon(ring, cs);
  if (!poly) return null;
  const polyErr = meanDistanceToPolygon(ring, poly) / ringDiag;
  const bow = sideBow(ring, cs, poly);
  // Sides must be straight: a wobbly loop with a few kinks is not a
  // polygon, and neither is a circle that fits better as a circle.
  if (ex) (ex.polyTries = (ex.polyTries as unknown[] | undefined) ?? []).push({ k: cs.length, polyErr, bow,
    angles: interiorAngles(poly).map((x) => Math.round(x / DEG)) });
  if (polyErr >= 0.035 || bow > 0.13) return null;
  if (cs.length >= 5 && ellipseErr < polyErr * 0.8) return null;
  return polygonShape(poly, ringDiag, pressure);
};

const ellipse = fitEllipse(ring);
const ellErr = ellipse ? ellipse.err : Infinity;

// 1. Clear corners: a polygon.
const corners = findCorners(0.68);

```

```

    if (ex) Object.assign(ex, { ellErr, sharpCorners: corners.length, softCorners: findCorners(0).length, area:
      area / (ringDiag * ringDiag) });
    const sharpPoly = tryPolygon(corners, ellErr);
    if (sharpPoly) return sharpPoly;

    // 2. No corners and a good fit: an ellipse. Two gentle kinks (a lemon, an
    //    eye) still read as one when it fits well.
    if (ellipse && ((corners.length <= 1 && ellErr < 0.085) || (corners.length === 2 && ellErr < 0.06))) {
      return ellipseShape(ellipse, first, pressure);
    }

    // 3. Rounded corners - how most hands draw a square - turn over a longer
    //    stretch. Counted too, but only for a loop that is plainly no ellipse.
    if (ellErr > 0.06) {
      const softCorners = findCorners(0);
      const soft = tryPolygon(softCorners, ellErr) ?? tryGrowing(softCorners, ellErr);
      if (soft) return soft;
    }

    // 4. A box drawn loosely - a rounded corner, a bowed side - may not show
    //    four clean corners, but it still fills its tightest bounding
    //    rectangle almost completely (a circle fills 79%, a triangle 50%).
    if (ellErr > 0.07) {
      const box = minAreaRect(convexHull(ring));
      if (ex) ex.boxFill = box ? area / box.area : null;
      if (box && area / box.area >= 0.86) return polygonShape(box.corners, ringDiag, pressure);
    }
    return null;
  }

  /** Convex hull (monotone chain), counter-clockwise. */
  function convexHull(pts: P[]): P[] {
    const s = [...pts].sort((a, b) => a[0] - b[0] || a[1] - b[1]);
    const cross = (o: P, a: P, b: P) => (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0]);
    const lower: P[] = [];
    for (const p of s) {
      while (lower.length >= 2 && cross(lower[lower.length - 2], lower[lower.length - 1], p) <= 0) lower.pop();
      lower.push(p);
    }
    const upper: P[] = [];
    for (let i = s.length - 1; i >= 0; i--) {
      const p = s[i];
      while (upper.length >= 2 && cross(upper[upper.length - 2], upper[upper.length - 1], p) <= 0) upper.pop();
      upper.push(p);
    }
    upper.pop();
    lower.pop();
    return lower.concat(upper);
  }

  /** The smallest rectangle around a convex polygon (one side lies on a hull edge). */
  function minAreaRect(hull: P[]): { corners: P[]; area: number } | null {
    if (hull.length < 3) return null;
    let best: { corners: P[]; area: number } | null = null;
    for (let i = 0; i < hull.length; i++) {
      const a = hull[i];
      const b = hull[(i + 1) % hull.length];
      const len = dist(a, b);
      if (len === 0) continue;
      const ux = (b[0] - a[0]) / len;
      const uy = (b[1] - a[1]) / len;
      let minU = Infinity;
      let maxU = -Infinity;
      let minV = Infinity;
      let maxV = -Infinity;
      for (const p of hull) {
        const u = p[0] * ux + p[1] * uy;
        const v = -p[0] * uy + p[1] * ux;
        minU = Math.min(minU, u);
        maxU = Math.max(maxU, u);
        minV = Math.min(minV, v);
        maxV = Math.max(maxV, v);
      }
      const area = (maxU - minU) * (maxV - minV);
      if (!best || area < best.area) {
        const back = (u: number, v: number): P => [u * ux - v * uy, u * uy + v * ux];
        best = { area, corners: [back(minU, minV), back(maxU, minV), back(maxU, maxV), back(minU, maxV)] };
      }
    }
  }

```

```

    }
    return best;
}

/** Cumulative path length at each point. */
function cumulative(pts: P[]): number[] {
    const out = [0];
    for (let i = 1; i < pts.length; i++) out.push(out[i - 1] + dist(pts[i - 1], pts[i]));
    return out;
}

/**
 * A straight line, read from the middle of the stroke: the pen landing and
 * lifting often add a hook at either end, which must not make a straight
 * line "not straight". Null when it is not a line.
 */
function tryLine(raw: P[], L: number, pressure: number): Pt[] | null {
    const cum = cumulative(raw);
    const mid = raw.filter((_, i) => cum[i] >= L * 0.07 && cum[i] <= L * 0.93);
    const core = mid.length >= 3 ? mid : raw;
    const coreL = pathLength(core);
    const chord = dist(core[0], core[core.length - 1]);
    if (coreL === 0 || chord / coreL < 0.93) return null;
    const fit = fitLine(core);
    const perp = (p: P) => Math.abs((p[0] - fit.c[0]) * fit.d[1] - (p[1] - fit.c[1]) * fit.d[0]);
    let dev = 0;
    for (const p of core) dev = Math.max(dev, perp(p));
    if (dev > Math.max(3, chord * 0.075)) return null;
    // Ends: the outermost points still on the line (a hook is not).
    const tol = Math.max(4, chord * 0.07);
    const along = (p: P) => (p[0] - fit.c[0]) * fit.d[0] + (p[1] - fit.c[1]) * fit.d[1];
    let a = 0;
    while (a < raw.length - 1 && perp(raw[a]) > tol) a++;
    let b = raw.length - 1;
    while (b > 0 && perp(raw[b]) > tol) b--;
    if (b <= a) return null;
    const on = (t: number): P => [fit.c[0] + fit.d[0] * t, fit.c[1] + fit.d[1] * t];
    const start = on(along(raw[a]));
    const end = on(along(raw[b]));
    if (dist(start, end) < chord * 0.6) return null;
    return makeLine(start, end, pressure);
}

/**
 * The stroke without a hook at either end: a sharp turn within the first or
 * last quarter, where the pen landed moving one way and then set off the
 * other (or flicked away on lifting).
 */
function trimHooks(raw: P[]): [P[], number] {
    const n = 64;
    const rs = resample(raw, n);
    let a = 0;
    let b = n - 1;
    for (let i = 2; i < n - 2; i++) {
        const t = turn(rs[i - 2], rs[i], rs[i + 2]);
        if (t < 55 * DEG) continue;
        if (i < n * 0.28) a = i;
        else if (i > n * 0.72 && b === n - 1) b = i;
    }
    const cut = rs.slice(a, b + 1);
    return [cut, pathLength(cut)];
}

/**
 * The closed lap inside a stroke: early point i and late point j (at least
 * half the stroke apart) where it comes back closest to itself. A tiny cost
 * for what is cut off keeps as much of the stroke as possible when several
 * pairs are about equally close (a side traced twice).
 */
function findLoop(pts: P[], L: number): { i: number; j: number; d: number; len: number } | null {
    const n = pts.length;
    const cum = cumulative(pts);
    const step = Math.max(1, Math.floor(n / 220));
    let best: { i: number; j: number; d: number; len: number; cost: number } | null = null;
    for (let i = 0; i < n; i += step) {
        if (cum[i] > L * 0.4) break;
        for (let j = n - 1; j > i; j -= step) {
            const len = cum[j] - cum[i];

```

```

    if (len < L * 0.5) break;
    const d = dist(pts[i], pts[j]);
    const cost = d + (L - len) * 0.03;
    if (!best || cost < best.cost) best = { i, j, d, len, cost };
  }
}
if (!best) return null;
// Settle on the closest pair right around the one found (the coarse
// stride may have stepped over the exact crossing).
const { i: bi, j: bj } = best;
for (let i = Math.max(0, bi - step); i <= Math.min(n - 1, bi + step); i++) {
  for (let j = Math.max(i + 1, bj - step); j <= Math.min(n - 1, bj + step); j++) {
    const len = cum[j] - cum[i];
    if (len < L * 0.5) continue;
    const d = dist(pts[i], pts[j]);
    const cost = d + (L - len) * 0.03;
    if (cost < best.cost) best = { i, j, d, len, cost };
  }
}
return { i: best.i, j: best.j, d: best.d, len: best.len };
}

function median(xs: number[]): number {
  const s = [...xs].sort((a, b) => a - b);
  return s.length ? s[Math.floor(s.length / 2)] : 0.5;
}

/** A straight stroke between two exact points (no levelling). */
export function lineBetween(a: [number, number], b: [number, number], pressure = 0.5): Pt[] {
  return densify([a, b], false, Math.max(2, dist(a, b) / 64), pressure);
}

function makeLine(a: P, b: P, pressure: number): Pt[] {
  // A ruler: horizontal / vertical when nearly so, the first point kept.
  const len = dist(a, b);
  const ang = snapAngle(Math.atan2(b[1] - a[1], b[0] - a[0]), Math.PI / 2, 4 * DEG);
  const end: P = [a[0] + Math.cos(ang) * len, a[1] + Math.sin(ang) * len];
  return densify([a, end], false, Math.max(2, len / 64), pressure);
}

// ---- Ellipse -----

interface EllipseFit {
  c: P;
  a: number;
  b: number;
  rot: number;
  /** Mean |normalised radius - 1|: 0 is a perfect ellipse. */
  err: number;
}

function fitEllipse(ring: P[]): EllipseFit | null {
  const n = ring.length;
  const cx = ring.reduce((s, p) => s + p[0], 0) / n;
  const cy = ring.reduce((s, p) => s + p[1], 0) / n;
  const { d } = fitLine(ring);
  const rot = Math.atan2(d[1], d[0]);
  const cos = Math.cos(rot);
  const sin = Math.sin(rot);
  const uv: P[] = ring.map(([x, y]) => {
    const dx = x - cx;
    const dy = y - cy;
    return [dx * cos + dy * sin, -dx * sin + dy * cos];
  });
  // Its size and centre are where the drawing actually reaches along each
  // axis (the mean of the few most extreme points, so one stray wobble does
  // not stretch it): "the same size, in the same place" as drawn.
  const ext = (vals: number[]) => {
    const s = [...vals].sort((p, q) => p - q);
    const k = Math.max(1, Math.round(s.length * 0.03));
    const lo = s.slice(0, k).reduce((t, x) => t + x, 0) / k;
    const hi = s.slice(-k).reduce((t, x) => t + x, 0) / k;
    return [lo, hi];
  };
  const [u0, u1] = ext(uv.map((p) => p[0]));
  const [v0, v1] = ext(uv.map((p) => p[1]));
  const a = (u1 - u0) / 2;
  const b = (v1 - v0) / 2;

```

```

    if (!(a > 0) || !(b > 0)) return null;
    const mu = (u0 + u1) / 2;
    const mv = (v0 + v1) / 2;
    let err = 0;
    for (const [u, v] of uv) err += Math.abs(Math.hypot((u - mu) / a, (v - mv) / b) - 1);
    const c: P = [cx + mu * cos - mv * sin, cy + mu * sin + mv * cos];
    return { c, a, b, rot, err: err / n };
}

function ellipseShape(e: EllipseFit, penStart: P, pressure: number): RecognizedShape {
    let { a, b, rot } = e;
    const round = Math.abs(a - b) / Math.max(a, b) < 0.14;
    if (round) {
        const r = (a + b) / 2;
        a = r;
        b = r;
        rot = 0;
    } else {
        // Upright when nearly so.
        rot = snapAngle(rot, Math.PI / 2, 8 * DEG);
    }
    // Start where the pen started, so the seam is where you began drawing.
    const cos = Math.cos(rot);
    const sin = Math.sin(rot);
    const dx = penStart[0] - e.c[0];
    const dy = penStart[1] - e.c[1];
    const start = Math.atan2((-dx * sin + dy * cos) / b, (dx * cos + dy * sin) / a);
    return { kind: round ? "circle" : "ellipse", points: ellipsePoints(e.c, a, b, rot, start, pressure), vertice
        s: [] };
}

// ---- Polygon -----

/** Corners where the fitted straight sides meet. */
function fitPolygon(ring: P[], cornerIdx: number[]): P[] | null {
    const N = ring.length;
    const k = cornerIdx.length;
    const lines: Array<{ c: P; d: P }> = [];
    for (let s = 0; s < k; s++) {
        const i0 = cornerIdx[s];
        const i1 = cornerIdx[(s + 1) % k];
        const span = (i1 - i0 + N) % N;
        if (span < 4) return null;
        // The middle of the side: the rounding at each corner would bend the fit.
        const trim = Math.max(1, Math.floor(span * 0.18));
        const side: P[] = [];
        for (let j = trim; j <= span - trim; j++) side.push(ring[(i0 + j) % N]);
        if (side.length < 2) return null;
        lines.push(fitLine(side));
    }
    const out: P[] = [];
    for (let s = 0; s < k; s++) {
        const p = intersect(lines[(s - 1 + k) % k], lines[s]);
        if (!p) return null;
        // A corner far from where the pen turned means two nearly parallel sides.
        if (dist(p, ring[cornerIdx[s]]) > 0.35 * dist(ring[0], ring[Math.floor(N / 2)]) + 1) return null;
        out.push(p);
    }
    return out;
}

/**
 * How far the drawn sides bow away from straight: for each side, the largest
 * gap between the drawing and the fitted side, over the side's length. A
 * hand-drawn straight side stays small; an arc (a heart's lobe, the round of
 * a D) does not, however well a few segments happen to follow it.
 */
function sideBow(ring: P[], cs: number[], poly: P[]): number {
    const N = ring.length;
    const k = cs.length;
    let worst = 0;
    for (let s = 0; s < k; s++) {
        const a = poly[s];
        const b = poly[(s + 1) % k];
        const len = dist(a, b);
        if (len === 0) return Infinity;
        const i0 = cs[s];
        const span = (cs[(s + 1) % k] - i0 + N) % N;

```

```

    let m = 0;
    for (let j = Math.floor(span * 0.15); j <= span - Math.floor(span * 0.15); j++) {
        m = Math.max(m, distToSegment(ring[(i0 + j) % N], a, b));
    }
    worst = Math.max(worst, m / len);
}
return worst;
}

function meanDistanceToPolygon(ring: P[], poly: P[]): number {
    let total = 0;
    for (const p of ring) {
        let best = Infinity;
        for (let i = 0; i < poly.length; i++) best = Math.min(best, distToSegment(p, poly[i], poly[(i + 1) % poly.length]));
        total += best;
    }
    return total / ring.length;
}

function interiorAngles(poly: P[]): number[] {
    return poly.map((p, i) => {
        const a = poly[(i - 1 + poly.length) % poly.length];
        const b = poly[(i + 1) % poly.length];
        const v1: P = [a[0] - p[0], a[1] - p[1]];
        const v2: P = [b[0] - p[0], b[1] - p[1]];
        const cos = (v1[0] * v2[0] + v1[1] * v2[1]) / (Math.hypot(...v1) * Math.hypot(...v2));
        return Math.acos(Math.max(-1, Math.min(1, cos)));
    });
}

function polygonShape(poly: P[], diag: number, pressure: number): RecognizedShape {
    const spacing = Math.max(2, diag / 80);
    if (poly.length === 4) {
        const angles = interiorAngles(poly);
        // Meant as a rectangle: every corner near square, or three clearly
        // square and one drawn round and wide (its fitted sides lean in).
        const offs = angles.map((a) => Math.abs(a - Math.PI / 2)).sort((p, q) => p - q);
        const boxy = offs[3] < 20 * DEG || (offs[2] < 12 * DEG && offs[3] < 30 * DEG);
        if (boxy) {
            // A rectangle: its sides' mean direction, straightened when nearly
            // upright, and its extent measured in that frame.
            const dirs = poly.map((p, i) => {
                const q = poly[(i + 1) % 4];
                return Math.atan2(q[1] - p[1], q[0] - p[0]);
            });
            // Average the four directions folded onto a quarter turn.
            let sx = 0;
            let sy = 0;
            for (const d of dirs) {
                sx += Math.cos(4 * d);
                sy += Math.sin(4 * d);
            }
            let rot = Math.atan2(sy, sx) / 4;
            rot = snapAngle(rot, Math.PI / 2, 10 * DEG);
            const cos = Math.cos(rot);
            const sin = Math.sin(rot);
            const uv = poly.map(([x, y]) => [x * cos + y * sin, -x * sin + y * cos] as P);
            const us = uv.map((p) => p[0]).sort((a, b) => a - b);
            const vs = uv.map((p) => p[1]).sort((a, b) => a - b);
            let u0 = (us[0] + us[1]) / 2;
            let u1 = (us[2] + us[3]) / 2;
            let v0 = (vs[0] + vs[1]) / 2;
            let v1 = (vs[2] + vs[3]) / 2;
            let kind: ShapeKind = "rectangle";
            const w = u1 - u0;
            const h = v1 - v0;
            if (Math.abs(w - h) / Math.max(w, h) < 0.12) {
                kind = "square";
                const s = (w + h) / 2;
                const cu = (u0 + u1) / 2;
                const cv = (v0 + v1) / 2;
                u0 = cu - s / 2;
                u1 = cu + s / 2;
                v0 = cv - s / 2;
                v1 = cv + s / 2;
            }
            // Corners in the order the pen drew them round (keeps the seam where

```

```

    // the stroke began, and the winding the same).
    const back = (u: number, v: number): P => [u * cos - v * sin, u * sin + v * cos];
    let rect: P[] = [back(u0, v0), back(u1, v0), back(u1, v1), back(u0, v1)];
    const drawnClockwise = signedArea(poly) > 0;
    if (signedArea(rect) > 0 !== drawnClockwise) rect = [rect[0], rect[3], rect[2], rect[1]];
    const startAt = nearestIndex(rect, poly[0]);
    rect = [...rect.slice(startAt), ...rect.slice(0, startAt)];
    return { kind, points: densify(rect, true, spacing, pressure), vertices: rect };
}
return { kind: "quadrilateral", points: densify(poly, true, spacing, pressure), vertices: poly };
}
if (poly.length === 3) {
    // A near-level side is levelled, turning the triangle about its centre.
    const cx = (poly[0][0] + poly[1][0] + poly[2][0]) / 3;
    const cy = (poly[0][1] + poly[1][1] + poly[2][1]) / 3;
    let fix = 0;
    for (let i = 0; i < 3; i++) {
        const a = poly[i];
        const b = poly[(i + 1) % 3];
        const ang = Math.atan2(b[1] - a[1], b[0] - a[0]);
        const snapped = snapAngle(ang, Math.PI / 2, 5 * DEG);
        if (snapped !== ang && Math.abs(snapped - ang) > Math.abs(fix)) fix = snapped - ang;
    }
    const tri = fix
        ? poly.map(([x, y]) => {
            const dx = x - cx;
            const dy = y - cy;
            return [cx + dx * Math.cos(fix) - dy * Math.sin(fix), cy + dx * Math.sin(fix) + dy * Math.cos(fix)]
                as P;
        })
        : poly;
    return { kind: "triangle", points: densify(tri, true, spacing, pressure), vertices: tri };
}
return { kind: poly.length === 5 ? "pentagon" : "hexagon", points: densify(poly, true, spacing, pressure), vertices: poly };
}

function signedArea(pts: P[]): number {
    let a = 0;
    for (let i = 0; i < pts.length; i++) {
        const [x1, y1] = pts[i];
        const [x2, y2] = pts[(i + 1) % pts.length];
        a += x1 * y2 - x2 * y1;
    }
    return a / 2;
}

function nearestIndex(pts: P[], q: P): number {
    let best = 0;
    for (let i = 1; i < pts.length; i++) if (dist(pts[i], q) < dist(pts[best], q)) best = i;
    return best;
}

// -----
// Joining lines drawn one after another
// -----

export interface JoinTarget {
    id: string;
    /** A straight line's two ends, when the target is a line. */
    line?: [[number, number], [number, number]];
    /** A shape's corners. */
    corners?: [number, number][];
}

export interface JoinResult {
    start: [number, number];
    end: [number, number];
    /** Earlier lines whose end moves to the shared corner (0 = its first end). */
    adjust: Array<{ id: string; end: 0 | 1; to: [number, number] }>;
}

/**
 * A new straight line's ends, joined to what is nearby: drawn right up to
 * (or a little past, or short of) the end of an earlier line, both lines are
 * cut or extended to meet exactly where they cross - so four quick lines make
 * a box with clean corners. Near a shape's corner, the end lands on it.
 * `reach` is how far an end may be from what it joins.

```

```

*/
export function joinLine(
  start: [number, number],
  end: [number, number],
  targets: JoinTarget[],
  reach: number
): JoinResult {
  const res: JoinResult = { start, end, adjust: [] };
  const newDir: P = [end[0] - start[0], end[1] - start[1]];
  const newLen = Math.hypot(newDir[0], newDir[1]);
  if (newLen === 0) return res;
  const used = new Set<string>();

  const joinEnd = (e: P): { to: P; adjust?: { id: string; end: 0 | 1; to: P } } | null => {
    let best: { to: P; adjust?: { id: string; end: 0 | 1; to: P }; d: number } | null = null;
    for (const t of targets) {
      if (t.line) {
        const [a, b] = t.line;
        for (const k of [0, 1] as const) {
          if (used.has(`${t.id}:${k}`)) continue;
          const q = k === 0 ? a : b;
          const d = dist(e, q);
          if (d > reach || (best && d >= best.d)) continue;
          const tDir: P = [b[0] - a[0], b[1] - a[1]];
          const tLen = Math.hypot(tDir[0], tDir[1]);
          const sin = tLen ? Math.abs(newDir[0] * tDir[1] - newDir[1] * tDir[0]) / (newLen * tLen) : 0;
          // Lines at a real angle meet where they cross; nearly parallel
          // ones simply continue from the earlier end.
          if (sin > Math.sin(20 * DEG)) {
            const x = intersect({ c: start, d: [newDir[0] / newLen, newDir[1] / newLen] }, { c: a, d: [tDir[0] / tLen, tDir[1] / tLen] });
            if (x && dist(x, e) <= reach * 1.5 && dist(x, q) <= reach * 1.5) {
              best = { to: x, adjust: { id: t.id, end: k, to: x }, d };
              continue;
            }
          }
        }
        best = { to: q, d };
      }
    }
    for (const c of t.corners ?? []) {
      const d = dist(e, c);
      if (d <= reach && (!best || d < best.d)) best = { to: c, d };
    }
  }
  return best;
};

const s = joinEnd(start);
if (s) {
  res.start = s.to;
  if (s.adjust) {
    res.adjust.push(s.adjust);
    used.add(`${s.adjust.id}:${s.adjust.end}`);
  }
}
const t = joinEnd(end);
// Both ends onto one point would fold the line to nothing.
if (t && dist(t.to, res.start) > newLen * 0.3) {
  res.end = t.to;
  if (t.adjust) res.adjust.push(t.adjust);
}
return res;
}

```

```

=====
// === END FILE: src/lib/whiteboard/shapeRecognition.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/share.ts ===
// File 121 of 194 | Lines: 193 | Size: 7449 bytes
=====

```

```

/**
 * Delivering an exported file
 * =====
 * One question, asked three ways depending on where ProNotes is running:

```

```

* "you have a PDF in memory - how does it reach WhatsApp, Drive, Gmail, or
* the user's Downloads folder?"
*
*   Android APK -> write to the app cache, hand the URI to the OS share
*                  sheet (@capacitor/share). This is the ONLY route that
*                  works: the WebView has no download manager behind it, so
*                  an '<a download>' click is swallowed with no file and no
*                  error at all.
*   Chrome/Edge -> navigator.share({ files }) opens the Windows share flyout,
*   on Windows    which is the same "choose an app" experience. Guarded by
*                  navigator.canShare, because Firefox exposes share() but
*                  refuses files, and the API needs HTTPS.
*   Everywhere    -> an ordinary browser download.
*   else
*
* Two things make this harder than it looks, and both are handled below:
*
* 1. TRANSIENT USER ACTIVATION. navigator.share may only be called shortly
*    after a click. Rasterising a 30-page notebook takes longer than that
*    window, so share() rejects with NotAllowedError on a file the user very
*    much wanted. That is not an error to show - it is a fallback to download.
* 2. CANCELLING IS NOT FAILING. Dismissing either share sheet rejects the
*    promise. Reporting "export failed" for "I changed my mind" trains people
*    to distrust the message that matters.
*/

import { Capacitor } from "@capacitor/core";

/** How a file left the app, so the UI can say something true afterwards. */
export type ShareOutcome = "shared" | "downloaded" | "cancelled";

export type ShareRoute = "native" | "webshare" | "download";

/**
 * True when running inside the Capacitor APK. Capacitor's own runtime flag is
 * used rather than a user-agent test, because Chrome on the same Android
 * tablet is a real browser and must keep the normal download path.
 */
export function isNativeApp(): boolean {
  return Capacitor.isNativePlatform();
}

/**
 * Whether this build can show an OS app chooser at all.
 *
 * `canShare` is probed with a real File: Chrome on Windows returns true for
 * `{ files }` while some builds return true for text and then throw on files.
 * A one-byte probe file costs nothing and answers the question honestly.
 */
export function canChooseApp(): boolean {
  if (isNativeApp()) return true;
  if (typeof navigator === "undefined" || typeof navigator.share !== "function") return false;
  if (typeof navigator.canShare !== "function" || typeof File === "undefined") return false;
  try {
    return navigator.canShare({ files: [new File([new Uint8Array(1)], "probe.png", { type: "image/png" })] });
  } catch {
    return false;
  }
}

/** Which route `shareBlob` would take right now. */
export function shareRoute(): ShareRoute {
  if (isNativeApp()) return "native";
  return canChooseApp() ? "webshare" : "download";
}

/**
 * Dismissing a share sheet rejects the call. That is the user changing their
 * mind, not a failure, so it must not surface as an error.
 *
 * AbortError is the Web Share API's own name for it; the Android plugin sends
 * a plain message instead, hence the text test as well.
 */
export function isShareCancellation(err: unknown): boolean {
  const e = err as { name?: string; message?: string } | null;
  if (e?.name === "AbortError") return true;
  return /cancel|abort|dismiss/i.test(e?.message ?? "");
}

```

```

/** The browser refused because the click that started this has expired. */
function isActivationExpiry(err: unknown): boolean {
  const e = err as { name?: string; message?: string } | null;
  if (e?.name === "NotAllowedError") return true;
  return /user activation|user gesture|transient/i.test(e?.message ?? "");
}

/**
 * Reads a Blob as a data URL. Capacitor's Filesystem bridge only carries
 * JSON, so binary has to round-trip through one to cross it.
 */
export function blobToDataURL(blob: Blob): Promise<string> {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => resolve(String(reader.result));
    reader.onerror = () => reject(new Error("Could not read the exported file data"));
    reader.readAsDataURL(blob);
  });
}

async function blobToBase64(blob: Blob): Promise<string> {
  const result = await blobToDataURL(blob);
  const comma = result.indexOf(",");
  return comma >= 0 ? result.slice(comma + 1) : result;
}

/**
 * Hands a file to the Android share sheet.
 *
 * Cache, not Documents: the file only has to outlive the share sheet, and the
 * cache directory needs no storage permission on any Android version. The
 * plugins are imported lazily so the web and Electron bundles never load
 * native code they cannot use.
 */
async function shareNative(blob: Blob, filename: string, title: string): Promise<ShareOutcome> {
  const [{ Filesystem, Directory }, { Share }] = await Promise.all([
    import("@capacitor/filesystem"),
    import("@capacitor/share"),
  ]);

  const data = await blobToBase64(blob);
  const written = await Filesystem.writeFile({
    path: filename,
    data,
    directory: Directory.Cache,
    recursive: true,
  });

  try {
    await Share.share({ title, files: [written.uri] });
    return "shared";
  } catch (err) {
    if (isShareCancellation(err)) return "cancelled";
    throw err;
  }
}

/** Triggers an ordinary browser download for any Blob. */
export function downloadBlob(blob: Blob, filename: string): void {
  if (typeof window === "undefined") return;
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  setTimeout(() => URL.revokeObjectURL(url), 1000);
}

/**
 * Opens the OS app chooser for this file, falling back to a download whenever
 * the chooser is unavailable or the click that started the export has aged
 * out. Returns how it actually left, never guessing.
 */
export async function shareBlob(blob: Blob, filename: string, title = filename): Promise<ShareOutcome> {
  if (isNativeApp()) return shareNative(blob, filename, title);

```

```

    if (canChooseApp()) {
      try {
        const file = new File([blob], filename, { type: blob.type || "application/octet-stream" });
        if (navigator.canShare({ files: [file] })) {
          await navigator.share({ files: [file], title });
          return "shared";
        }
      } catch (err) {
        if (isShareCancellation(err)) return "cancelled";
        // Anything else - an expired click, a share target that blew up - still
        // leaves the user wanting their file. Falling through to the download is
        // always better than an error dialog and no file.
        if (!isActivationExpiry(err)) {
          console.warn("[share] The system share sheet failed; downloading instead.", err);
        }
      }
    }

    downloadBlob(blob, filename);
    return "downloaded";
  }

  /**
   * Saves a file without offering an app chooser: a plain download on the web
   * and desktop. On Android there is no such thing - the share sheet IS how a
   * file is saved, and it offers "Save to Files" among the apps.
   */
  export async function saveBlob(blob: Blob, filename: string): Promise<ShareOutcome> {
    if (isNativeApp()) return shareNative(blob, filename, filename);
    downloadBlob(blob, filename);
    return "downloaded";
  }

  =====
  // === END FILE: src/lib/whiteboard/share.ts ===
  =====

  =====
  // === START FILE: src/lib/whiteboard/shortcuts.ts ===
  // File 122 of 194 | Lines: 32 | Size: 1059 bytes
  =====

  /**
   * Keyboard combo predicates.
   *
   * These live outside the component so they can be unit-tested. They match on
   * `event.code` (the PHYSICAL key) rather than `event.key` (the produced
   * character), because `event.key` carries the shifted/caps form:
   *
   * Ctrl+Shift+Z -> event.key === "Z" (uppercase!)
   * Ctrl+Z + CapsLock -> event.key === "Z"
   *
   * so a check for `event.key === "z"` silently misses both. `event.code` is
   * also layout-stable, which matters for non-QWERTY keyboards.
   */

  export interface ComboEvent {
    code: string;
    ctrlKey: boolean;
    metaKey: boolean;
    shiftKey: boolean;
  }

  const withModifier = (e: ComboEvent) => e.ctrlKey || e.metaKey;

  /** Ctrl/Cmd + Z (without Shift) — step back. */
  export function isUndoCombo(e: ComboEvent): boolean {
    return withModifier(e) && e.code === "KeyZ" && !e.shiftKey;
  }

  /** Ctrl/Cmd + Y, or Ctrl/Cmd + Shift + Z — step forward again. */
  export function isRedoCombo(e: ComboEvent): boolean {
    return withModifier(e) && (e.code === "KeyY" || (e.code === "KeyZ" && e.shiftKey));
  }

  =====

```

```
// === END FILE: src/lib/whiteboard/shortcuts.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/sort.ts ===
// File 123 of 194 | Lines: 25 | Size: 895 bytes
=====

/**
 * Universal comparator for folders and boards.
 * Sorts by "updated" (most recent first / oldest first), "created", or natural "name" (A-Z / Z-A).
 */
export function sortItems<T> extends { name: string; createdAt?: number; updatedAt?: number }>({
  items: T[],
  sortBy: "updated" | "created" | "name",
  sortOrder: "asc" | "desc"
}): T[] {
  return [...items].sort((a, b) => {
    if (sortBy === "name") {
      const cmp = a.name.localeCompare(b.name, undefined, { numeric: true, sensitivity: "base" });
      return sortOrder === "asc" ? cmp : -cmp;
    }
    if (sortBy === "created") {
      const tA = a.createdAt || 0;
      const tB = b.createdAt || 0;
      return sortOrder === "asc" ? tA - tB : tB - tA;
    }
    // updated
    const tA = a.updatedAt || a.createdAt || 0;
    const tB = b.updatedAt || b.createdAt || 0;
    return sortOrder === "asc" ? tA - tB : tB - tA;
  });
}
```

```
// === END FILE: src/lib/whiteboard/sort.ts ===
=====
```

```
// === START FILE: src/lib/whiteboard/speechRecognition.ts ===
// File 124 of 194 | Lines: 866 | Size: 32496 bytes
=====

/**
 * Speech-to-Text & Live Voice Dictation Controller
 * =====
 * Provides continuous speech recognition and real-time transcription
 * streaming directly into ProNotes text boxes and sticky notes.
 *
 * Supports Web Speech API (Chrome, Edge, Safari, Android WebView)
 * with graceful fallback and user feedback.
 *
 * Key design patterns (from react-speech-recognition & annyang):
 * - Auto-restart on natural `onend` (Android WebView kills recognition
 *   after ~10-60s of silence even with continuous:true)
 * - `manualStop` flag to distinguish user-initiated stop from engine timeout
 * - Restart debounce to prevent rapid onend→start() crash loops
 * - Max restart limit to prevent infinite restart loops on broken sessions
 * - Clearing interim text when final results arrive or session ends
 * - Filtering "aborted" errors triggered by our own abort() cleanup calls
 */
```

```
// Web Speech API Types
interface SpeechRecognitionErrorEvent extends Event {
  readonly error: string;
  readonly message?: string;
}
```

```
interface SpeechRecognitionAlternative {
  readonly transcript: string;
  readonly confidence: number;
}
```

```
interface SpeechRecognitionResult {
  readonly isFinal: boolean;
  readonly length: number;
}
```

```

    item(index: number): SpeechRecognitionAlternative;
    [index: number]: SpeechRecognitionAlternative;
}

interface SpeechRecognitionResultList {
    readonly length: number;
    item(index: number): SpeechRecognitionResult;
    [index: number]: SpeechRecognitionResult;
}

interface SpeechRecognitionEvent extends Event {
    readonly resultIndex: number;
    readonly results: SpeechRecognitionResultList;
}

interface ISpeechRecognition extends EventTarget {
    continuous: boolean;
    interimResults: boolean;
    lang: string;
    maxAlternatives: number;
    start(): void;
    stop(): void;
    abort(): void;
    onaudiostart: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onsoundstart: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onspeechstart: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onspeechend: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onsoundend: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onaudioend: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onresult: ((this: ISpeechRecognition, ev: SpeechRecognitionEvent) => void) | null;
    onnomatch: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onerror: ((this: ISpeechRecognition, ev: SpeechRecognitionErrorEvent) => void) | null;
    onstart: ((this: ISpeechRecognition, ev: Event) => void) | null;
    onend: ((this: ISpeechRecognition, ev: Event) => void) | null;
}

type SpeechRecognitionConstructor = new () => ISpeechRecognition;

export type SpeechStatus =
    | "idle"
    | "requesting-permission"
    | "listening"
    | "processing"
    | "unsupported"
    | "error";

export interface FinalOpts {
    /**
     * True when this final CONTINUES the previous one (the engine re-sent the
     * sentence-so-far with more words: "how" then "how to"). The receiver
     * should revise the previously inserted chunk in place, not append.
     */
    revisesPrevious: boolean;
}

export interface SpeechListenerOptions {
    onInterim?: (interim: string) => void;
    onFinal?: (finalText: string, opts?: FinalOpts) => void;
    onError?: (error: string) => void;
    onStatusChange?: (status: SpeechStatus) => void;
    lang?: string;
}

/** Languages the dictation picker offers. BCP-47 ids, as Web Speech expects. */
export const SUPPORTED_DICTATION_LANGS: Array<{ id: string; label: string }> = [
    { id: "en-US", label: "English" },
    { id: "he-IL", label: "עברית" },
    { id: "ar-SA", label: "العربية" },
    { id: "ru-RU", label: "русский" },
    { id: "es-ES", label: "Español" },
    { id: "fr-FR", label: "Français" },
    { id: "de-DE", label: "Deutsch" },
    { id: "it-IT", label: "Italiano" },
];

export const DEFAULT_DICTATION_LANG = "en-US";

/**

```

```

* Resolves a preferred language to one the picker supports: exact id wins,
* then a bare base language ("he" -> "he-IL"), else the default.
*/
export function resolveDictationLang(pref: string | undefined | null): string {
  if (pref) {
    const exact = SUPPORTED_DICTATION_LANGS.find((l) => l.id.toLowerCase() === pref.toLowerCase());
    if (exact) return exact.id;
    const base = pref.split(/[-_]/)[0]?.toLowerCase();
    if (base) {
      const byBase = SUPPORTED_DICTATION_LANGS.find(
        (l) => l.id.split("-")[0]?.toLowerCase() === base
      );
      if (byBase) return byBase.id;
    }
  }
  return DEFAULT_DICTATION_LANG;
}

export function isSpeechRecognitionSupported(): boolean {
  if (typeof window === "undefined") return false;
  const w = window as unknown as {
    SpeechRecognition?: SpeechRecognitionConstructor;
    webkitSpeechRecognition?: SpeechRecognitionConstructor;
  };
  return !!((w.SpeechRecognition || w.webkitSpeechRecognition));
}

/**
 * Explicitly requests microphone audio permission via getUserMedia.
 * This triggers the browser or Android system permission dialog
 * ("Allow ProNotes to record audio?").
 */
/** Set once the microphone has been granted in this app session. */
let micGranted = false;

export async function requestMicrophonePermission(): Promise<{ granted: boolean; error?: string }> {
  if (typeof navigator === "undefined" || !navigator.mediaDevices?.getUserMedia) {
    // If mediaDevices is unavailable, fallback directly to Web Speech API
    return { granted: true };
  }
  // Opening the mic just to close it again costs ~0.3-1 s on the tablet before
  // a single word is heard. Once granted, every later Dictate starts at once.
  if (micGranted) return { granted: true };

  try {
    // Processing flags help the engine "hear well" on noisy tablets: echo
    // cancellation, background-noise suppression and auto gain are all free
    // on Chrome/Edge/Android WebView and ignored where unsupported.
    const stream = await navigator.mediaDevices.getUserMedia({
      audio: { echoCancellation: true, noiseSuppression: true, autoGainControl: true },
    });
    // Stop all audio tracks immediately so the microphone is released for speech recognition
    stream.getTracks().forEach((track) => track.stop());
    micGranted = true;
    return { granted: true };
  } catch (err: unknown) {
    const error = err as { name?: string; message?: string };
    if (error?.name === "NotAllowedError" || error?.name === "PermissionDeniedError") {
      return {
        granted: false,
        error: "Microphone permission was denied. Please allow microphone access in your device or browser settings.",
      };
    }
    return {
      granted: false,
      error: error?.message || "Could not access microphone.",
    };
  }
}

/**
 * Max auto-restarts in a row where the engine died almost at once (under
 * QUICK_END_MS, nothing heard). That is a broken engine, not a pause.
 *
 * It used to cap EVERY restart at 5 - but Android ends a session after a few
 * seconds of quiet, so thinking for ~30 s silently switched dictation off.
 * Pauses are now limited by time instead (SILENCE_LIMIT_MS).

```

```

*/
const MAX_QUICK_FAILS = 4;
const QUICK_END_MS = 1500;

/** Dictation turns itself off after this long with nothing heard. */
export const SILENCE_LIMIT_MS = 90_000;

/** After Stop, finals still arriving within this window are kept. */
const STOP_DRAIN_MS = 1200;

/** A "network" error is retried this many times in a row before giving up. */
const MAX_NETWORK_RETRIES = 3;

/**
 * Minimum ms between auto-restart attempts.
 * annyang uses 1000ms based on years of production use – rapid restarts cause
 * runaway CPU usage and browser rate-limits on mobile WebView.
 */
const RESTART_DEBOUNCE_MS = 1000;

/** Errors that should NOT trigger the error UI (they are expected/internal) */
const SILENT_ERRORS = new Set(["no-speech", "aborted"]);

/** Errors that should NOT trigger auto-restart (terminal conditions) */
const FATAL_ERRORS = new Set(["not-allowed", "service-not-allowed", "audio-capture", "language-not-supported"]);

/**
 * A small ring of recent engine events, readable on the tablet as
 * `window.__dictationLog` (e.g. over remote DevTools) - so "it missed my
 * words" can be checked against what the engine actually sent.
 */
function logDictation(ev: string, detail?: unknown) {
  if (typeof window === "undefined") return;
  const w = window as unknown as { __dictationLog?: Array<{ t: number; ev: string; detail?: unknown }> };
  const log = (w.__dictationLog ??= []);
  log.push({ t: Date.now(), ev, detail });
  if (log.length > 300) log.splice(0, log.length - 300);
}

/** How long a repeated final phrase still counts as the engine's echo. */
export const REPEAT_FINAL_WINDOW_MS = 3000;

/**
 * True when `text` is the engine repeating the phrase it just delivered.
 *
 * Android's recogniser sometimes reports one phrase twice. The old guard
 * dropped the second of ANY two final-only results in a row, without looking
 * at the words - but Android delivers most phrases final-only, so every other
 * sentence you spoke was silently thrown away. Only a true repeat - same
 * words, moments later - is dropped now.
 */
export function isRepeatedFinal(
  prevText: string,
  prevAt: number,
  text: string,
  now: number,
  ignoreWindow = false
): boolean {
  const norm = normTranscript(text);
  return norm !== "" && norm === prevText && (ignoreWindow || now - prevAt < REPEAT_FINAL_WINDOW_MS);
}

/** Lowercase, trimmed, inner whitespace collapsed – the comparison form. */
export function normTranscript(text: string): string {
  return text.trim().toLowerCase().replace(/\s+/g, " ");
}

/** One word, comparison form: lowercase, no surrounding punctuation. */
function normWord(w: string): string {
  return w.toLowerCase().replace(/^[^p{L}\p{N}]+|^[^p{L}\p{N}]+$|$/gu, "");
}

/**
 * True when `text` is the engine re-sending the sentence-so-far – grown
 * ("how" -> "how to") OR with an earlier word revised ("are you want to
 * fix the" -> "are you going to fix the problem") – rather than a new
 * phrase. The two must share at least one leading word and the new final

```

```

* must not be shorter (a shrink is treated as a new phrase: revising would
* delete confirmed words). A repeat outside the echo window is a new
* phrase, and "yes" -> "yesterday" shares no whole word, so it is not one.
*
* Known trade-off, same class as the exact-echo drop: two deliberate
* utterances sharing leading words inside 3s ("I want tea" ... "I want
* coffee") merge into one. Continuous speech revises far more often than
* people repeat sentence starts, so revising wins.
*
* `prevText` must already be in `normTranscript` form (as stored below).
*/
export function isContinuationOfFinal(
  prevText: string,
  prevAt: number,
  text: string,
  now: number,
  ignoreWindow = false
): boolean {
  const norm = normTranscript(text);
  if (norm === "" || prevText === "") return false;
  if (!ignoreWindow && now - prevAt >= REPEAT_FINAL_WINDOW_MS) return false;
  if (norm === prevText) return false;
  const prevWords = prevText.split(" ");
  const nextWords = norm.split(" ");
  if (nextWords.length < prevWords.length) return false;
  let common = 0;
  while (common < prevWords.length && normWord(nextWords[common]) === normWord(prevWords[common])) {
    // Guard a word that normalises to nothing (e.g. "--"): it must not
    // count as a shared word, or "--" -> "-- x" would revise on punctuation.
    if (normWord(nextWords[common]) === "") break;
    common++;
  }
  return common >= 1;
}

/** Where the last dictated chunk was inserted (editor coordinates). */
export interface DictationRange {
  start: number;
  length: number;
}

/**
* Splices one dictated chunk into the text, pure and unit-tested.
*
* Append is the default. When `revisesPrevious` is set and the cursor is
* still parked exactly where the last chunk landed (the user did not type
* or tap elsewhere in between), the previous chunk is first removed and the
* fuller text inserted at the same spot – so "how" + "how to" becomes
* "how to ", never "how how to ". Anything else falls back to a plain
* append at the cursor/selection.
*/
export function spliceDictationChunk(
  current: string,
  selStart: number,
  selEnd: number,
  chunk: string,
  last: DictationRange | null,
  revisesPrevious: boolean
): { text: string; cursor: number; range: DictationRange } {
  let base = current;
  let at = selStart;
  let end = selEnd;
  if (
    revisesPrevious &&
    last &&
    selEnd === selStart &&
    selStart === last.start + last.length &&
    last.start >= 0 &&
    last.start + last.length <= current.length
  ) {
    base = current.slice(0, last.start) + current.slice(last.start + last.length);
    at = last.start;
    end = last.start;
  }
  const before = base.slice(0, at);
  const after = base.slice(end);
  const trimmed = chunk.trim();
  const addSpace = before.length > 0 && !before.endsWith(" ") && !before.endsWith("\n") ? " " : "";

```

```

const insertion = `${addSpace}${trimmed}`;
return { text: before + insertion + after, cursor: at + insertion.length, range: { start: at, length: insertion.length } };
}

/**
 * Takes the live (not yet final) words back out of the text. They sit in
 * `range`; the caret goes to where they started, which is exactly where the
 * final – or the next interim – belongs.
 */
export function stripInterim(
  value: string,
  selStart: number,
  selEnd: number,
  range: DictationRange | null
): { value: string; selStart: number; selEnd: number } {
  if (!range || range.start < 0 || range.start + range.length > value.length) {
    return { value, selStart, selEnd };
  }
  return {
    value: value.slice(0, range.start) + value.slice(range.start + range.length),
    selStart: range.start,
    selEnd: range.start,
  };
}

/**
 * Shows the words being heard right now IN the text, replacing the previous
 * interim. Returns the new text, caret and range (null when nothing is heard).
 */
export function placeInterim(
  value: string,
  selStart: number,
  selEnd: number,
  prev: DictationRange | null,
  interim: string
): { text: string; cursor: number; range: DictationRange | null } {
  const base = stripInterim(value, selStart, selEnd, prev);
  const words = interim.trim();
  if (!words) return { text: base.value, cursor: base.selStart, range: null };
  const before = base.value.slice(0, base.selStart);
  const after = base.value.slice(base.selEnd);
  const lead = before.length > 0 && !/\s$/.test(before) ? " " : "";
  const insertion = lead + words;
  return {
    text: before + insertion + after,
    cursor: base.selStart + insertion.length,
    range: { start: base.selStart, length: insertion.length },
  };
}

export class VoiceDictationController {
  private recognition: ISpeechRecognition | null = null;
  private status: SpeechStatus = "idle";
  private options: SpeechListenerOptions = {};
  private errorMessage: string = "";

  // ■■ Auto-restart state ■■
  /** True when user explicitly called stop()/abort() – prevents auto-restart */
  private manualStop: boolean = false;
  /** Timer handle for debounced restart */
  private restartTimer: ReturnType<typeof setTimeout> | null = null;
  /** Whether the last onerror was a fatal (non-restartable) error */
  private lastErrorWasFatal: boolean = false;
  /** Timestamp of the last .start() call (for debounce calculation) */
  private lastStartedAt: number = 0;

  // ■■ Duplicate detection (see isRepeatedFinal) ■■
  private lastFinalText: string = "";
  private lastFinalAt: number = 0;
  /**
   * Engine session counter: every startRecognitionEngine() (including an
   * auto-restart after a pause) opens a new one. Result indices restart at
   * 0 in each session, so emission memory is keyed `${session}:${index}`.
   */
  private sessionSeq: number = 0;
  /** Norm text already emitted per session+result-index: re-sends are skipped. */
  private emittedFinals: Map<string, string> = new Map();

```

```

/**
 * True right after an auto-restart, until the first genuinely-new final
 * arrives. A pause ends the engine session; the new session routinely
 * re-sends what was already written (exact or grown), NO MATTER how long
 * the pause was – so the time window is ignored while this is set.
 */
private catchUp: boolean = false;
/** Norms of recently emitted chunks (capped): the catch-up echo check. */
private recentChunks: string[] = [];
/** The words heard but not yet final: written as final if you stop mid-phrase. */
private pendingInterim: string = "";
/** Last time anything was heard (speech, interim or final). */
private lastHeardAt: number = 0;
/** Quick-death restarts in a row (see MAX_QUICK_FAILS). */
private quickFails: number = 0;
private networkRetries: number = 0;
/** After stop(): the old engine may still deliver the last final. */
private draining: boolean = false;
private drainTimer: ReturnType<typeof setTimeout> | null = null;
private heardInSession: boolean = false;

constructor(options: SpeechListenerOptions = {}) {
  this.options = options;
}

public getStatus(): SpeechStatus {
  return this.status;
}

public getErrorMessage(): string {
  return this.errorMessage;
}

private setStatus(s: SpeechStatus, errorMsg: string = "") {
  this.status = s;
  this.errorMessage = errorMsg;
  this.options.onStatusChange?.(s);
  if (errorMsg) {
    this.options.onError?.(errorMsg);
  }
}

private clearRestartTimer() {
  if (this.restartTimer !== null) {
    clearTimeout(this.restartTimer);
    this.restartTimer = null;
  }
}

/**
 * Detach all event handlers from the current recognition instance
 * and null it out. Prevents stale callbacks firing on old objects.
 */
private detachRecognition() {
  if (this.recognition) {
    this.recognition.onstart = null;
    this.recognition.onresult = null;
    this.recognition.onerror = null;
    this.recognition.onend = null;
    this.recognition.onspeechstart = null;
    this.recognition.onspeechend = null;
    this.recognition.onaudiostart = null;
    this.recognition.onaudioend = null;
    this.recognition.onsoundstart = null;
    this.recognition.onsoundend = null;
    this.recognition.onnomatch = null;
    this.recognition = null;
  }
}

public async start(): Promise<boolean> {
  if (typeof window === "undefined") return false;

  // Reset restart state for a fresh session
  this.manualStop = false;
  this.lastErrorWasFatal = false;
  this.lastFinalText = "";
  this.lastFinalAt = 0;

```

```

    this.sessionSeq = 0;
    this.emittedFinals.clear();
    this.catchUp = false;
    this.recentChunks = [];
    this.pendingInterim = "";
    this.lastHeardAt = Date.now();
    this.quickFails = 0;
    this.networkRetries = 0;
    this.endDrain();
    this.clearRestartTimer();
    logDictation("start", { lang: this.options.lang });

    // 1. Explicitly prompt / verify microphone permission
    this.setStatus("requesting-permission");
    const perm = await requestMicrophonePermission();
    if (!perm.granted) {
        this.setStatus(
            "error",
            perm.error || "Microphone permission is required to record speech."
        );
        return false;
    }

    return this.startRecognitionEngine();
}

/**
 * Internal: creates and starts the SpeechRecognition engine.
 * Separated from start() so auto-restart can call it without
 * re-requesting microphone permission. `isAutoRestart` marks a session
 * that follows a pause: its first finals are engine catch-up until proven
 * otherwise (see catchUp).
 */
private startRecognitionEngine(isAutoRestart = false): boolean {
    const w = window as unknown as {
        SpeechRecognition?: SpeechRecognitionConstructor;
        webkitSpeechRecognition?: SpeechRecognitionConstructor;
    };

    const SpeechRec = w.SpeechRecognition || w.webkitSpeechRecognition;
    if (!SpeechRec) {
        this.setStatus(
            "unsupported",
            "Speech recognition is not available in this browser. Tip: Tap the microphone key (🎤) on your keyboard to dictate directly."
        );
        return false;
    }

    try {
        // Clean up any previous instance
        if (this.recognition) {
            try {
                this.recognition.abort();
            } catch {
                // ignore
            }
            this.detachRecognition();
        }

        const rec = new SpeechRec();
        this.recognition = rec;
        // A new engine session: indices restart at 0, so last session's
        // emission memory must not leak into this one (cross-session repeats
        // are the catch-up check's job, below).
        this.sessionSeq++;
        this.emittedFinals.clear();
        this.catchUp = isAutoRestart;
        this.heardInSession = false;
        rec.continuous = true;
        rec.interimResults = true;
        rec.maxAlternatives = 1;
        rec.lang =
            this.options.lang ||
            (typeof navigator !== "undefined" && navigator.language) ||
            "en-US";

        rec.onstart = () => {

```

```

    logDictation("onstart", { restart: isAutoRestart });
    if (!this.draining) this.setStatus("listening");
  };
  rec.onspeechstart = () => {
    this.lastHeardAt = Date.now();
    this.heardInSession = true;
  };

  rec.onresult = (e: SpeechRecognitionEvent) => {
    // Successfully received results - reset the failure counters
    this.quickFails = 0;
    this.networkRetries = 0;
    this.lastHeardAt = Date.now();
    this.heardInSession = true;
    logDictation("onresult", {
      i: e.resultIndex,
      r: Array.from({ length: e.results.length }, (_, k) => [
        e.results[k].isFinal ? "F" : "I",
        e.results[k][0]?.transcript,
      ]),
    });

    let interim = "";
    // Finals in index order, each with its result index: the same
    // segment re-sent is the same (session, index), a new phrase never is.
    const finals: Array<{ index: number; text: string }> = [];

    for (let i = e.resultIndex; i < e.results.length; ++i) {
      const res = e.results[i];
      if (res.isFinal) {
        // Confidence is NOT used to filter: Android reports 0 for many real
        // phrases, and those were being dropped. Echoes are caught below.
        finals.push({ index: i, text: res[0].transcript });
      } else {
        interim += res[0].transcript;
      }
    }

    if (finals.length > 0) {
      const now = Date.now();
      for (const f of finals) {
        const norm = normTranscript(f.text);
        if (norm === "") continue;
        const key = `${this.sessionSeq}:${f.index}`;
        // Same segment re-sent (the engine repeats whole result lists,
        // e.g. resultIndex 0 after a long pause): never new content.
        if (this.emittedFinals.get(key) === norm) {
          continue;
        }
        if (this.catchUp) {
          // First finals after a pause/restart: an exact re-send of
          // anything recent is the engine catching up – drop it, no
          // matter how long the pause was.
          if (this.recentChunks.includes(norm)) {
            this.options.onInterim?.("");
            continue;
          }
          // A grown re-send revises the previous chunk in place.
          if (isContinuationOfFinal(this.lastFinalText, this.lastFinalAt, f.text, now, true)) {
            this.emitFinal(f.text, now, true, key, norm);
            continue;
          }
          // Genuinely new speech: append, and catch-up is over.
          this.emitFinal(f.text, now, false, key, norm);
          this.catchUp = false;
          continue;
        }
        if (isRepeatedFinal(this.lastFinalText, this.lastFinalAt, f.text, now)) {
          this.options.onInterim?.("");
          continue;
        }
        // A cumulative re-send ("how" -> "how to") revises the previous
        // chunk in place; anything else appends as a new phrase.
        const revisesPrevious = isContinuationOfFinal(this.lastFinalText, this.lastFinalAt, f.text, now);
        this.emitFinal(f.text, now, revisesPrevious, key, norm);
      }
    }
    // Interim is always replaced by final (react-speech-recognition
    // pattern). Words AFTER the final in the same event are the next

```

```

    // phrase already starting: they stay as the new interim.
    this.pendingInterim = interim;
    if (!this.draining) this.options.onInterim?.(interim);
  } else if (interim && !this.draining) {
    this.pendingInterim = interim;
    this.options.onInterim?.(interim);
  }
};

rec.onerror = (e: SpeechRecognitionErrorEvent) => {
  logDictation("onerror", e.error);
  // "no-speech" and "aborted" are expected/internal – don't show UI errors
  if (SILENT_ERRORS.has(e.error) || this.draining) {
    return;
  }
  // A dropped connection mid-sentence is usually momentary: the restart
  // after onend retries it, and only repeated failures are reported.
  if (e.error === "network" && this.networkRetries < MAX_NETWORK_RETRIES) {
    this.networkRetries++;
    return;
  }

  // Track fatal errors to prevent auto-restart
  if (FATAL_ERRORS.has(e.error)) {
    this.lastErrorWasFatal = true;
  }

  let userMsg = "Speech recognition error: " + (e.error || "unknown");
  if (e.error === "not-allowed") {
    userMsg = "Microphone access was denied. Please allow microphone permission in device settings.";
  } else if (e.error === "audio-capture") {
    userMsg = "Microphone is unavailable or busy with another application.";
  } else if (e.error === "language-not-supported") {
    userMsg = "This device can't dictate in the chosen language. Pick another one from the language menu
    next to Dictate.";
  } else if (e.error === "network" || e.error === "service-not-allowed") {
    userMsg = "Speech service is temporarily unavailable. Tip: Tap the microphone key (🎤) on your keybo
    ard to speak into notes.";
  }

  this.setStatus("error", userMsg);
};

rec.onend = () => {
  logDictation("onend", { heard: this.heardInSession });
  if (this.draining) {
    this.endDrain();
    return;
  }
  // A session ending mid-phrase: what was heard is kept, not lost.
  this.flushInterim();
  // Always clear interim text when recognition session ends
  this.options.onInterim?.("");

  const now = Date.now();
  const quick = !this.heardInSession && now - this.lastStartedAt < QUICK_END_MS;
  this.quickFails = quick ? this.quickFails + 1 : 0;
  const tooQuiet = now - this.lastHeardAt > SILENCE_LIMIT_MS;
  if (!this.manualStop && tooQuiet && this.status === "listening") {
    this.detachRecognition();
    this.setStatus("idle");
    this.options.onError?.("Dictation stopped after 1.5 minutes of silence. Tap Dictate to continue.");
    return;
  }

  // 🎤 Auto-restart logic 🎤
  // If the user manually stopped, or we hit a fatal error, or we've
  // exceeded the restart limit – go idle. Otherwise, auto-restart
  // with debounce (Android WebView regularly kills continuous sessions
  // after silence timeouts).
  if (
    this.manualStop ||
    this.lastErrorWasFatal ||
    this.status === "error" ||
    this.status === "unsupported" ||
    this.quickFails >= MAX_QUICK_FAILS
  ) {
    if (this.quickFails >= MAX_QUICK_FAILS && !this.manualStop && this.status === "listening") {

```

```

        this.options.onError?.("The speech engine keeps stopping. Tap Dictate to try again.");
    }
    if (this.status === "listening" || this.status === "processing") {
        this.setStatus("idle");
    }
    this.detachRecognition();
    return;
}

// Debounced auto-restart (annyang pattern: wait at least RESTART_DEBOUNCE_MS
// since the last start, so very short sessions don't cause rapid-fire restarts)
this.clearRestartTimer();
const elapsed = Date.now() - this.lastStartedAt;
const delay = Math.max(0, RESTART_DEBOUNCE_MS - elapsed);
this.restartTimer = setTimeout(() => {
    this.restartTimer = null;
    // Double-check we weren't manually stopped during the debounce window
    if (!this.manualStop && this.status !== "error" && this.status !== "idle") {
        this.startRecognitionEngine(true);
    }
}, delay);
};

this.lastStartedAt = Date.now();
rec.start();
return true;
} catch (err: unknown) {
    // DOMException indicates a redundant microphone start – safe to swallow
    // (Chrome throws when recognition.start() is called while already started)
    if (typeof DOMException !== "undefined" && err instanceof DOMException) {
        return true;
    }
    const msg = (err as Error)?.message || "Failed to start speech recognition.";
    this.setStatus("error", msg);
    return false;
}
}

/**
 * Writes the words heard so far as a final (used when the session ends or
 * you tap Stop mid-phrase). If the engine's real final follows, it revises
 * this in place (or is dropped as a repeat), so nothing is written twice.
 */
private flushInterim() {
    const text = this.pendingInterim;
    this.pendingInterim = "";
    const norm = normTranscript(text);
    if (!norm) return;
    const now = Date.now();
    if (isRepeatedFinal(this.lastFinalText, this.lastFinalAt, text, now)) return;
    logDictation("flush", text);
    const revises = isContinuationOfFinal(this.lastFinalText, this.lastFinalAt, text, now);
    this.emitFinal(text, now, revises, `flush:${now}`, norm);
}

private endDrain() {
    if (this.drainTimer !== null) {
        clearTimeout(this.drainTimer);
        this.drainTimer = null;
    }
    if (this.draining) {
        this.draining = false;
        this.detachRecognition();
    }
}

/** Records one emitted final and forwards it to the editor. */
private emitFinal(text: string, now: number, revisesPrevious: boolean, key: string, norm: string) {
    this.emittedFinals.set(key, norm);
    this.lastFinalText = norm;
    this.lastFinalAt = now;
    this.recentChunks.push(norm);
    if (this.recentChunks.length > 8) this.recentChunks.splice(0, this.recentChunks.length - 8);
    this.options.onFinal?.(text, { revisesPrevious });
}

/**
 * Switches the recognition language. Takes effect on the next session;

```

```

    * also applied live when the engine allows it.
    */
    public setLang(lang: string) {
        const resolved = resolveDictationLang(lang);
        this.options.lang = resolved;
        try {
            if (this.recognition) this.recognition.lang = resolved;
        } catch {
            // ignore – the next startRecognitionEngine() applies it
        }
    }

    /**
     * Stop listening, KEEPING what was being said. The words heard so far are
     * written straight away; the engine's own final for them, which arrives a
     * moment after stop(), still revises them for STOP_DRAIN_MS. (Stop used to
     * unhook the engine at once, and the last sentence was simply lost.)
     */
    public stop() {
        this.manualStop = true;
        this.clearRestartTimer();
        logDictation("stop");
        this.flushInterim();

        if (this.recognition) {
            try {
                this.recognition.stop();
            } catch {
                // ignore
            }
            this.draining = true;
            if (this.drainTimer !== null) clearTimeout(this.drainTimer);
            this.drainTimer = setTimeout(() => this.endDrain(), STOP_DRAIN_MS);
        }
        // Clear any lingering interim text
        this.options.onInterim?.("");
        this.setStatus("idle");
    }

    /** Stop at once and discard anything not yet final. */
    public abort() {
        this.manualStop = true;
        this.clearRestartTimer();
        this.pendingInterim = "";
        this.endDrain();

        if (this.recognition) {
            try {
                this.recognition.abort();
            } catch {
                // ignore
            }
            this.detachRecognition();
        }
        // Clear any lingering interim text
        this.options.onInterim?.("");
        this.setStatus("idle");
    }
}

```

```

=====
// === END FILE: src/lib/whiteboard/speechRecognition.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/split.ts ===
// File 125 of 194 | Lines: 16 | Size: 587 bytes
=====

```

```

/**
 * Split view geometry, kept free of React so it can be unit-tested.
 */

/**
 * Each pane keeps at least this much room: the live pane carries the tool
 * dock, the page bar and the top bar, and below ~260px they overlap.
 */

```

```

export const MIN_PANE_PX = 260;

/** The ratio for a pointer at `pos` px along a `length` px row, kept usable. */
export function dividerRatio(pos: number, length: number): number {
  if (!(length > 0)) return 0.5;
  const min = Math.min(0.45, MIN_PANE_PX / length);
  return Math.max(Math.max(0.2, min), Math.min(1 - Math.max(0.2, min), pos / length));
}

=====
// === END FILE: src/lib/whiteboard/split.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/stroke.ts ===
// File 126 of 194 | Lines: 760 | Size: 26481 bytes
=====

/**
 * Whiteboard Stroke Rendering (v4.6 – Pixel-Perfect Path & Anti-Wedge Engine)
 * =====
 * Features:
 * - Mathematical fix for getSvgPathFromStroke: Smoothly connects polygon perimeter
 *   to start point without creating sharp triangular cross-over flap wedges!
 * - Single point dots (periods, letter dots, niqqud, dagesh, i/j dots)
 * - Anti-Hook lift-off filter for touchscreens
 * - Smooth freehand spline with perfect-freehand
 */

import { getStroke, getStrokePoints } from "perfect-freehand";
import type { StrokeElement, PressureCurve } from "../types";
import { PEN_TYPE_PRESETS } from "../types";

export interface StrokeOptions {
  size: number;
  thinning: number;
  smoothing: number;
  streamline: number;
  easing: (t: number) => number;
  start: { cap: boolean; taper: number };
  end: { cap: boolean; taper: number };
  simulatePressure: boolean;
  last: boolean;
}

/**
 * Calibrate raw hardware digitizer pressure (0..1) through a user-selected
 * response curve and dynamic sensitivity range.
 */
export function calibratePressure(
  rawPressure: number,
  sensitivity: number = 50,
  curve: PressureCurve = "standard"
): number {
  if (curve === "off" || sensitivity <= 0) return 0.5;

  let p = Math.max(0.01, Math.min(1.0, rawPressure));

  // Apply response curve
  if (curve === "soft") {
    // Concave: reaches expressive thick ink faster with gentle S Pen touch
    p = Math.pow(p, 0.7);
  } else if (curve === "firm") {
    // Convex: requires deliberate, firm pressure for broad strokes
    p = Math.pow(p, 1.4);
  }

  // Dynamic sensitivity scaling around midpoint 0.5
  // sensitivity = 50 -> scale = 1.0 (exact curve)
  // sensitivity = 100 -> maximum dynamic thickness variation
  // sensitivity = 0 -> completely uniform line
  const scale = sensitivity / 50;
  const delta = (p - 0.5) * scale;
  return Math.max(0.02, Math.min(1.0, 0.5 + delta));
}

export function getStrokeOptionsForStyle(

```

```

    style: string,
    baseSize: number,
    isLast = true,
    smoothingLevel: number = 50
): Partial<StrokeOptions> {
  const preset = PEN_TYPE_PRESETS[style as keyof typeof PEN_TYPE_PRESETS] || PEN_TYPE_PRESETS.gel;
  // Line weight is the single most visible difference between writing pens.
  const sizeMultiplier = preset.sizeScale ?? 1;
  const roundCap = preset.cap !== "flat";

  // When smoothingLevel is 50 (default), scale is 1.0 – exactly matches preset defaults.
  // 0% gives raw digitizer input; 100% gives maximum stroke stabilization.
  const scale = Math.max(0, smoothingLevel) / 50;
  const smoothing = Math.min(0.98, Math.max(0.02, preset.smoothing * scale));
  const streamline = Math.min(0.95, Math.max(0.02, preset.streamline * scale));

  return {
    size: baseSize * sizeMultiplier,
    thinning: preset.thinning,
    smoothing,
    streamline,
    simulatePressure: preset.simulatePressure,
    // A flat cap is what gives a marker/highlighter its squared-off chisel end;
    // a round cap is a pen. This is visible at both ends of every stroke.
    start: { cap: roundCap, taper: preset.startTaper },
    end: { cap: roundCap, taper: preset.endTaper },
    last: isLast,
  };
}

/**
 * Re-weight a stroke's per-point pressure to express the pen's character.
 *
 * perfect-freehand only varies width via `pressure` x `thinning`. With a mouse
 * every point arrives at ~0.5, so ANY preset that merely tweaks `thinning`
 * renders identically – which is why the old pen, marker, highlighter and
 * pencil were indistinguishable. These two modifiers give each pen a real,
 * measurable geometry:
 *
 * - **speedResponse** – fast strokes thin out, as a real ballpoint or fountain
 *   pen does. Computed from the distance between consecutive points.
 * - **nibAngle** – a chisel nib is wide across its edge and narrow along it, so
 *   width becomes a function of the DIRECTION of travel. This is what makes a
 *   fountain pen calligraphic instead of merely thin.
 */
export function applyPenDynamics(
  points: Array<[number, number, number]>,
  style: string
): Array<[number, number, number]> {
  const preset = PEN_TYPE_PRESETS[style as keyof typeof PEN_TYPE_PRESETS];
  if (!preset) return points;
  const { speedResponse, nibAngle } = preset;
  if (!speedResponse && nibAngle === undefined) return points;

  const nibRad = nibAngle !== undefined ? (nibAngle * Math.PI) / 180 : null;
  // Speed at which a stroke is considered "fast"; above this the width stops
  // thinning further, so a flick cannot collapse the line to nothing.
  const FAST = 22;

  return points.map((pt, i) => {
    const prev = points[i - 1] ?? points[i + 1] ?? pt;
    const dx = pt[0] - prev[0];
    const dy = pt[1] - prev[1];
    const dist = Math.hypot(dx, dy);

    // The stored pressure is used as is. The pressure curve is applied ONCE,
    // when the point is captured (see calibratePressure): applying it here as
    // well calibrated every new stroke twice, and re-drew every stroke already
    // in the notebook with a curve that did not exist when it was written.
    let pressure = pt[2];

    if (speedResponse) {
      const fast = Math.min(1, dist / FAST);
      pressure *= 1 - fast * speedResponse;
    }

    if (nibRad !== null && dist > 0.001) {
      // |sin| of the angle between travel and the nib: 1 across the nib

```

```

    // (widest), 0 along it (thinnest). Floored so the line never vanishes.
    const travel = Math.atan2(dy, dx);
    const across = Math.abs(Math.sin(travel - nibRad));
    pressure *= 0.22 + 0.78 * across;
  }

  return [pt[0], pt[1], Math.max(0.02, Math.min(1, pressure))];
});
}

/**
 * Filter out spurious lift-off hooks (the sharp trailing tail caused by lifting
 * finger/stylus off capacitive glass screens).
 */
export function pruneTerminalHooks(points: [number, number, number][]): [number, number, number][] {
  if (!points || points.length < 4) return points || [];

  const pts = [...points];

  // Pass 1: Check the very last point
  const last = pts[pts.length - 1];
  const prev = pts[pts.length - 2];
  const prev2 = pts[pts.length - 3];

  const dx1 = prev[0] - prev2[0];
  const dy1 = prev[1] - prev2[1];
  const len1 = Math.hypot(dx1, dy1);

  const dx2 = last[0] - prev[0];
  const dy2 = last[1] - prev[1];
  const len2 = Math.hypot(dx2, dy2);

  if (len1 > 1.5 && len2 > 1.5) {
    const dot = (dx1 * dx2 + dy1 * dy2) / (len1 * len2);
    // If angle > ~95 degrees reversal and tail is relatively short (< 40px)
    if (dot < -0.1 && len2 < 45) {
      pts.pop();
    }
  }

  // Pass 2: Check the new last point in case of a 2-step hook
  if (pts.length >= 4) {
    const l1 = pts[pts.length - 1];
    const p1 = pts[pts.length - 2];
    const p2 = pts[pts.length - 3];
    const dx1 = p1[0] - p2[0];
    const dy1 = p1[1] - p2[1];
    const L1 = Math.hypot(dx1, dy1);
    const dx2 = l1[0] - p1[0];
    const dy2 = l1[1] - p1[1];
    const L2 = Math.hypot(dx2, dy2);
    if (L1 > 1.5 && L2 > 1.5) {
      const dot2 = (dx1 * dx2 + dy1 * dy2) / (L1 * L2);
      if (dot2 < -0.15 && L2 < 35) {
        pts.pop();
      }
    }
  }

  return pts;
}

/**
 * Standard perfect-freehand SVG path builder.
 * Connects the outline polygon cleanly with true closed perimeter midpoint B-splines
 * without creating sharp triangular cross-over flap wedges at endpoints.
 */
export function getSvgPathFromStroke(
  inputPoints: number[][],
  closed = true
): string {
  let points = inputPoints;
  let len = points.length;
  if (len === 0) return "";
  if (len === 1) {
    const [x, y] = points[0];
    return `M ${x - 1}.toFixed(2)},${y.toFixed(2)} a 1,1 0 1,0 2,0 a 1,1 0 1,0 -2,0 Z`;
  }
}

```

```

if (len === 2) {
  const [p0, p1] = points;
  return `M ${p0[0].toFixed(2)},${p0[1].toFixed(2)} L ${p1[0].toFixed(2)},${p1[1].toFixed(2)} Z`;
}

// Collapse consecutive coincident vertices FIRST.
//
// perfect-freehand converges a tapered end to a point by emitting several
// vertices at the same coordinate (measured: three vertices all 0px from the
// last input point, with the next real vertices 17.9px away). Those
// zero-length edges have no defined angle, so the sharp-corner test bailed
// out and the tip was smoothed away – the stroke ended in a flat chisel cut
// instead of a point. With duplicates removed, the tip becomes a single
// vertex between two long edges and is correctly seen as sharp.
const deduped: number[][] = [points[0]];
for (let i = 1; i < len; i++) {
  const prev = deduped[deduped.length - 1];
  if (Math.hypot(points[i][0] - prev[0], points[i][1] - prev[1]) > 0.01) {
    deduped.push(points[i]);
  }
}
// A closed outline also wraps: drop a final point coincident with the first.
if (
  deduped.length > 1 &&
  Math.hypot(
    deduped[deduped.length - 1][0] - deduped[0][0],
    deduped[deduped.length - 1][1] - deduped[0][1]
  ) <= 0.01
) {
  deduped.pop();
}
if (deduped.length >= 3) {
  points = deduped;
  len = deduped.length;
}

if (closed) {
  const startMidX = (points[0][0] + points[len - 1][0]) / 2;
  const startMidY = (points[0][1] + points[len - 1][1]) / 2;
  let d = `M ${startMidX.toFixed(2)},${startMidY.toFixed(2)}`;

  for (let i = 0; i < len; i++) {
    const prev = points[(i - 1 + len) % len];
    const curr = points[i];
    const next = points[(i + 1) % len];
    const midX = (curr[0] + next[0]) / 2;
    const midY = (curr[1] + next[1]) / 2;

    if (isSharpCorner(prev, curr, next)) {
      // Route THROUGH the vertex. A midpoint quadratic only ever passes
      // through edge midpoints, so at a sharp vertex it cuts the corner off –
      // measured at 17% of the stroke width at a pen tip, which is what made
      // the end of a stroke look chopped instead of pointed.
      d += ` L ${curr[0].toFixed(2)},${curr[1].toFixed(2)}`;
      d += ` L ${midX.toFixed(2)},${midY.toFixed(2)}`;
    } else {
      d += ` Q ${curr[0].toFixed(2)},${curr[1].toFixed(2)} ${midX.toFixed(2)},${midY.toFixed(2)}`;
    }
  }
  d += " Z";
  return d;
} else {
  let d = `M ${points[0][0].toFixed(2)},${points[0][1].toFixed(2)}`;
  for (let i = 0; i < len - 1; i++) {
    const prev = points[i === 0 ? 0 : i - 1];
    const curr = points[i];
    const next = points[i + 1];
    const midX = (curr[0] + next[0]) / 2;
    const midY = (curr[1] + next[1]) / 2;
    if (i > 0 && isSharpCorner(prev, curr, next)) {
      d += ` L ${curr[0].toFixed(2)},${curr[1].toFixed(2)}`;
      d += ` L ${midX.toFixed(2)},${midY.toFixed(2)}`;
    } else {
      d += ` Q ${curr[0].toFixed(2)},${curr[1].toFixed(2)} ${midX.toFixed(2)},${midY.toFixed(2)}`;
    }
  }
  const last = points[len - 1];
  d += ` L ${last[0].toFixed(2)},${last[1].toFixed(2)}`;
}

```

```

    return d;
  }
}

/**
 * Is the turn at `curr` sharp enough that rounding it would visibly blunt the
 * shape? Measured across real stroke outlines, tips and true corners fall at
 * 112-125° while ordinary curvature stays at 137° and above, so 130° separates
 * them cleanly: tips stay pointed, curves stay smooth.
 */
const SHARP_CORNER_DEG = 130;

function isSharpCorner(
  prev: number[],
  curr: number[],
  next: number[]
): boolean {
  const v1x = prev[0] - curr[0];
  const v1y = prev[1] - curr[1];
  const v2x = next[0] - curr[0];
  const v2y = next[1] - curr[1];
  const l1 = Math.hypot(v1x, v1y);
  const l2 = Math.hypot(v2x, v2y);
  // Degenerate/duplicate points have no meaningful angle; treat them as smooth
  // so they keep the existing behaviour instead of injecting stray corners.
  if (l1 < 0.001 || l2 < 0.001) return false;
  const cos = (v1x * v2x + v1y * v2y) / (l1 * l2);
  const deg = (Math.acos(Math.max(-1, Math.min(1, cos))) * 180) / Math.PI;
  return deg < SHARP_CORNER_DEG;
}

/**
 * Build the SVG `d` attribute for a StrokeElement.
 */
export function buildStrokePath(
  el: StrokeElement,
  isDraft = false,
  smoothingOverride?: number
): string {
  if (!el.points || el.points.length === 0) return "";

  const strokeWidth = Math.max(el.strokeWidth || 2, 1);
  const dotRadius = Math.max(strokeWidth / 2, 2);

  // Clean points of any trailing hooks if committing. A settled stroke's
  // points are the finished line (and its ends may be eraser cuts, which
  // are not pen hooks).
  const cleanPoints = isDraft || el.settled ? el.points : pruneTerminalHooks(el.points);

  // Single point -> render exact solid circular dot with r = max(strokeWidth / 2, 2)
  if (cleanPoints.length === 1) {
    const [x, y] = cleanPoints[0];
    const r = dotRadius;
    return `M ${x - r}.toFixed(2)},${y.toFixed(2)} a ${r.toFixed(2)},${r.toFixed(2)} 0 1,0 ${r * 2}.toFixed(
      2)},0 a ${r.toFixed(2)},${r.toFixed(2)} 0 1,0 ${(-r * 2).toFixed(2)},0 Z`;
  }

  // Calculate bounding diagonal span of points to detect micro-movements on tap
  let minX = Infinity;
  let minY = Infinity;
  let maxX = -Infinity;
  let maxY = -Infinity;
  for (const [px, py] of cleanPoints) {
    if (px < minX) minX = px;
    if (py < minY) minY = py;
    if (px > maxX) maxX = px;
    if (py > maxY) maxY = py;
  }
  const span = Math.hypot(maxX - minX, maxY - minY);
  const dotThreshold = Math.max(4, strokeWidth * 0.6);

  if (span <= dotThreshold || (cleanPoints.length <= 2 && span < 6)) {
    const avgX = (minX + maxX) / 2;
    const avgY = (minY + maxY) / 2;
    const r = dotRadius;
    return `M ${avgX - r}.toFixed(2)},${avgY.toFixed(2)} a ${r.toFixed(2)},${r.toFixed(2)} 0 1,0 ${r * 2}.to
      Fixed(2)},0 a ${r.toFixed(2)},${r.toFixed(2)} 0 1,0 ${(-r * 2).toFixed(2)},0 Z`;
  }
}

```

```

// Settled points were smoothed once already; smoothing them again would
// move them.
const smoothingLevel = el.settled ? 0 : smoothingOverride ?? el.smoothing ?? 50;
const options = getStrokeOptionsForStyle(el.strokeStyle, strokeWidth, !isDraft, smoothingLevel);
if (el.cutStart && options.start) options.start = { ...options.start, taper: 0 };
if (el.cutEnd && options.end) options.end = { ...options.end, taper: 0 };
// Pen character is applied to the POINTS, not the options: perfect-freehand
// derives width from pressure, so this is the only place a preset can make a
// visible difference when the input pressure is flat (mouse, or a stylus at
// steady force).
const dynamic = applyPenDynamics(
  cleanPoints as Array<[number, number, number]>,
  el.strokeStyle
);
const inputPoints = dynamic.map([x, y, p]) => [x, y, p];
const outline = getStroke(inputPoints, options);

if (!outline || outline.length < 3) {
  const avgX = (minX + maxX) / 2;
  const avgY = (minY + maxY) / 2;
  const r = dotRadius;
  return `M ${avgX - r}.toFixed(2)},${avgY.toFixed(2)} a ${r.toFixed(2)},${r.toFixed(2)} 0 1,0 ${r * 2}.toFixed(2)},0 a ${r.toFixed(2)},${r.toFixed(2)} 0 1,0 ${-r * 2}.toFixed(2)},0 Z`;
}

return getSvgPathFromStroke(outline, true);
}

/**
 * The line a stroke is actually drawn along, as points: its raw points after
 * the same hook trimming and smoothing the renderer applies. The eraser cuts
 * THIS - what you see - and its pieces keep it, so nothing beside a cut moves.
 */
export function settledStrokePoints(el: StrokeElement): [number, number, number][] {
  if (el.settled || !el.points || el.points.length < 3) return el.points;
  const strokeWidth = Math.max(el.strokeWidth || 2, 1);
  const options = getStrokeOptionsForStyle(el.strokeStyle, strokeWidth, true, el.smoothing ?? 50);
  const clean = pruneTerminalHooks(el.points);
  const sp = getStrokePoints(
    clean.map([x, y, p]) => [x, y, p]),
    options
  );
  if (sp.length < 2) return el.points;
  return sp.map((s) => [s.point[0], s.point[1], s.pressure] as [number, number, number]);
}

export function computePressureFromSpeed(
  points: Array<[number, number, number]>
): number {
  if (points.length < 2) return 0.5;
  const prev = points[points.length - 2];
  const curr = points[points.length - 1];
  if (!prev || !curr) return 0.5;
  const dx = curr[0] - prev[0];
  const dy = curr[1] - prev[1];
  const dist = Math.hypot(dx, dy);
  const maxSpeed = 10;
  const t = Math.min(1, dist / maxSpeed);
  return 0.52 - t * 0.08;
}

export function getArrowheadPath(
  tipX: number,
  tipY: number,
  angle: number,
  size: number
): string {
  const cos = Math.cos(angle);
  const sin = Math.sin(angle);

  const bx1 = tipX - cos * size + sin * (size * 0.4);
  const by1 = tipY - sin * size - cos * (size * 0.4);
  const bx2 = tipX - cos * size - sin * (size * 0.4);
  const by2 = tipY - sin * size + cos * (size * 0.4);

  return `M ${tipX},${tipY} L ${bx1},${by1} L ${bx2},${by2} Z`;
}

```

```

/**
 * Clip one segment against the eraser capsule, returning the covered parameter
 * range(s) in [0,1], or null when it misses.
 *
 * The capsule (segment A->B thickened by `r`) is the union of a rectangle and
 * the two end discs, so each piece is clipped analytically and the results
 * unioned. This is O(1) per segment. The previous version approximated the
 * capsule by sampling circles along it, which cost O(samples) per segment and
 * blew the frame budget on a fast drag (measured 8.2ms for one event).
 */
function clipSegmentToCapsule(
  x0: number,
  y0: number,
  x1: number,
  y1: number,
  ax: number,
  ay: number,
  bx: number,
  by: number,
  r: number
): [number, number] | null {
  const ex = bx - ax;
  const ey = by - ay;
  const len = Math.hypot(ex, ey);

  // Work in the capsule's frame: origin A, x along A->B, y perpendicular.
  let ux = 1;
  let uy = 0;
  if (len > 1e-9) {
    ux = ex / len;
    uy = ey / len;
  }
  const toLocal = (px: number, py: number): [number, number] => {
    const dx = px - ax;
    const dy = py - ay;
    return [dx * ux + dy * uy, -dx * uy + dy * ux];
  };

  const [lx0, ly0] = toLocal(x0, y0);
  const [lx1, ly1] = toLocal(x1, y1);
  const dx = lx1 - lx0;
  const dy = ly1 - ly0;

  let lo = Infinity;
  let hi = -Infinity;
  const add = (a: number, b: number) => {
    if (b < a) [a, b] = [b, a];
    a = Math.max(0, a);
    b = Math.min(1, b);
    if (b >= a) {
      lo = Math.min(lo, a);
      hi = Math.max(hi, b);
    }
  };

  // 1. The rectangular body: |y| <= r AND 0 <= x <= len.
  if (len > 1e-9) {
    let yLo = 0;
    let yHi = 1;
    if (Math.abs(dy) < 1e-9) {
      if (Math.abs(ly0) > r) {
        yLo = 1;
        yHi = 0; // never inside the slab
      }
    } else {
      yLo = (-r - ly0) / dy;
      yHi = (r - ly0) / dy;
      if (yLo > yHi) [yLo, yHi] = [yHi, yLo];
    }
    let xLo = 0;
    let xHi = 1;
    if (Math.abs(dx) < 1e-9) {
      if (lx0 < 0 || lx0 > len) {
        xLo = 1;
        xHi = 0;
      }
    } else {

```

```

    xLo = (0 - lx0) / dx;
    xHi = (len - lx0) / dx;
    if (xLo > xHi) [xLo, xHi] = [xHi, xLo];
  }
  const a = Math.max(yLo, xLo);
  const b = Math.min(yHi, xHi);
  if (b >= a) add(a, b);
}

// 2. The two end discs.
const disc = (cx: number, cy: number) => {
  const fx = lx0 - cx;
  const fy = ly0 - cy;
  const A = dx * dx + dy * dy;
  if (A < 1e-9) {
    if (fx * fx + fy * fy <= r * r) add(0, 1);
    return;
  }
  const B = 2 * (fx * dx + fy * dy);
  const C = fx * fx + fy * fy - r * r;
  const disc2 = B * B - 4 * A * C;
  if (disc2 < 0) return;
  const sq = Math.sqrt(disc2);
  add((-B - sq) / (2 * A), (-B + sq) / (2 * A));
};
disc(0, 0);
if (len > 1e-9) disc(len, 0);

return hi >= lo ? [lo, hi] : null;
}

/**
 * Pixel eraser: remove the part of a stroke swept by the eraser and return the
 * surviving pieces.
 *
 * The eraser is a CAPSULE, not a point: `(ax, ay)` is where it was at the last
 * pointer event and `(bx, by)` where it is now. Testing only the current
 * position leaves un-erased gaps whenever the user drags faster than the event
 * rate.
 *
 * Each stroke SEGMENT is clipped analytically against the eraser, rather than
 * testing the recorded points. Point-sampling has two failure modes that make
 * the eraser feel broken, both measured on a stroke with 30px point spacing:
 * - a small eraser passing BETWEEN two recorded points erased nothing at all;
 * - a 24px eraser removed 60px of stroke, because the cut could only land on
 *   an existing point.
 * Clipping cuts exactly at the eraser's edge whatever the point spacing.
 *
 * `points` and the eraser coordinates are all in the stroke's local space.
 *
 * Returns `null` when nothing was touched (so the caller can skip the write -
 * this runs on every pointermove of a drag), `[]` when the stroke is gone, or
 * the surviving runs.
 */
export function eraseFromStroke(
  points: Array<number, number, number>[],
  ax: number,
  ay: number,
  bx: number,
  by: number,
  radius: number
): Array<Array<number, number, number>> | null {
  if (!points || points.length < 2) {
    // A dot has no segments: fall back to a distance test.
    if (points && points.length === 1) {
      const [px, py] = points[0];
      return distanceToSegment(px, py, ax, ay, bx, by) <= radius ? [] : null;
    }
    return null;
  }

  // Bounding box of the swept eraser, used to reject segments cheaply. Without
  // this every segment of every stroke paid for the full clip on every
  // pointermove, which is what made a fast drag stutter.
  const sweepMinX = Math.min(ax, bx) - radius;
  const sweepMaxX = Math.max(ax, bx) + radius;
  const sweepMinY = Math.min(ay, by) - radius;
  const sweepMaxY = Math.max(ay, by) + radius;

```

```

let touched = false;
const covered: Array<[number, number] | null> = [];

for (let i = 0; i < points.length - 1; i++) {
  const [x0, y0] = points[i];
  const [x1, y1] = points[i + 1];

  if (
    Math.max(x0, x1) < sweepMinX ||
    Math.min(x0, x1) > sweepMaxX ||
    Math.max(y0, y1) < sweepMinY ||
    Math.min(y0, y1) > sweepMaxY
  ) {
    covered.push(null);
    continue;
  }

  const span = clipSegmentToCapsule(x0, y0, x1, y1, ax, ay, bx, by, radius);
  if (span) touched = true;
  covered.push(span);
}

if (!touched) return null;

const lerp = (i: number, t: number): [number, number, number] => {
  const [x0, y0, p0] = points[i];
  const [x1, y1, p1] = points[i + 1];
  return [x0 + (x1 - x0) * t, y0 + (y1 - y0) * t, p0 + (p1 - p0) * t];
};

const runs: Array<Array<[number, number, number]>> = [];
let current: Array<[number, number, number]> = [];
const EPS = 1e-6;

for (let i = 0; i < points.length - 1; i++) {
  const span = covered[i];
  if (!span) {
    // Untouched segment: the run continues through it.
    if (current.length === 0) current.push(points[i]);
    current.push(points[i + 1]);
    continue;
  }
  const [lo, hi] = span;
  if (lo > EPS) {
    if (current.length === 0) current.push(points[i]);
    current.push(lerp(i, lo));
  }
  if (current.length > 0) {
    runs.push(current);
    current = [];
  }
  if (hi < 1 - EPS) {
    current.push(lerp(i, hi));
    current.push(points[i + 1]);
  }
}
if (current.length > 0) runs.push(current);

// Drop crumbs. A fragment shorter than the eraser itself is a bead or a
// sliver left behind mid-rub – it reads as debris, not as ink the user chose
// to keep, and each one is a full element with its own round caps.
const minLength = Math.max(radius, 1);
return runs.filter((run) => {
  if (run.length < 2) return false;
  let length = 0;
  for (let i = 1; i < run.length; i++) {
    length += Math.hypot(run[i][0] - run[i - 1][0], run[i][1] - run[i - 1][1]);
  }
  return length >= minLength;
});
}

/** Shortest distance from a point to a segment – used for dot strokes. */
function distanceToSegment(
  px: number,
  py: number,
  ax: number,

```

```

    ay: number,
    bx: number,
    by: number
  ): number {
    const dx = bx - ax;
    const dy = by - ay;
    const len2 = dx * dx + dy * dy;
    if (len2 < 1e-9) return Math.hypot(px - ax, py - ay);
    let t = ((px - ax) * dx + (py - ay) * dy) / len2;
    t = Math.max(0, Math.min(1, t));
    return Math.hypot(px - (ax + dx * t), py - (ay + dy * t));
  }

  /**
   * Determine whether a freehand stroke approximates an intentional straight line.
   * Used for "Draw and Hold" auto-straightening (e.g. underlining, arrows, highlighters).
   */
  export function isStrokeStraight(
    points: Array<number, number, number>|>,
    toleranceRatio: number = 0.88
  ): boolean {
    if (!points || points.length < 5) return false;
    const first = points[0];
    const last = points[points.length - 1];
    const chord = Math.hypot(last[0] - first[0], last[1] - first[1]);
    if (chord < 28) return false; // too short to classify as intentional line

    let totalLength = 0;
    for (let i = 1; i < points.length; i++) {
      totalLength += Math.hypot(
        points[i][0] - points[i - 1][0],
        points[i][1] - points[i - 1][1]
      );
    }
    if (totalLength <= 0) return false;
    return chord / totalLength >= toleranceRatio;
  }

  /**
   * The stroke without the points recorded while the pen was held still at its
   * end. A held pen keeps reporting tiny moves; left in, their scribble adds
   * path length the line never had, and a ruler-straight line followed by a
   * one-second hold measured as a curve (length ratio ~0.6), so draw-and-hold
   * never straightened anything drawn by a real hand.
   */
  export function trimHoldTail(
    points: Array<number, number, number>|>,
    radius: number
  ): Array<number, number, number>|> {
    if (points.length < 3) return points;
    const last = points[points.length - 1];
    let i = points.length - 1;
    while (i > 0 && Math.hypot(points[i - 1][0] - last[0], points[i - 1][1] - last[1]) <= radius) i--;
    // Keep the first point of the hold so the line still ends where the pen is.
    return [...points.slice(0, i), last];
  }

  /**
   * Replace a jagged freehand stroke with an evenly interpolated straight line
   * between the start and end point, preserving average stroke pressure.
   */
  export function straightenStrokePoints(
    points: Array<number, number, number>|>,
    segments: number = 8
  ): Array<number, number, number>|> {
    if (!points || points.length < 2) return points;
    const first = points[0];
    const last = points[points.length - 1];
    const avgPressure =
      points.reduce((acc, pt) => acc + (pt[2] ?? 0.5), 0) / points.length;

    const result: Array<number, number, number>|> = [];
    for (let i = 0; i <= segments; i++) {
      const t = i / segments;
      const x = first[0] + (last[0] - first[0]) * t;
      const y = first[1] + (last[1] - first[1]) * t;
      result.push([x, y, avgPressure]);
    }
  }

```

```

    return result;
}

=====
// === END FILE: src/lib/whiteboard/stroke.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/textParser.ts ===
// File 127 of 194 | Lines: 550 | Size: 17455 bytes
=====

/**
 * Rich Markdown & Typography Tokenizer for ProNotes Whiteboard Cards & Stickies
 * =====
 * Fast, robust, dependency-free parser for:
 * - Headings (#, ##, ###)
 * - Interactive checklists (- [ ], - [x])
 * - Bullet lists (- , * , + ) and numbered lists (1. , 2. )
 * - Blockquotes (> )
 * - Code blocks (```)
 * - Inline formatting: bold, italic, <u>underline</u>, ~~strikethrough~~, `code`, ==highlight==
 * - LaTeX math:  $...$  and  $...$ 
 * - Obsidian [[WikiLinks]] and #tags
 */

export interface InlineToken {
  type:
    | "text"
    | "bold"
    | "italic"
    | "underline"
    | "strike"
    | "code"
    | "highlight"
    | "math"
    | "link"
    | "tag"
    | "url";
  text: string;
  value?: string; // target for links / math formula
}

export interface BlockToken {
  type:
    | "h1"
    | "h2"
    | "h3"
    | "checklist"
    | "bullet"
    | "numbered"
    | "quote"
    | "codeblock"
    | "paragraph";
  lineIndex: number; // for interactive checklist toggles
  checked?: boolean; // for checklist
  number?: number; // for numbered list
  tokens: InlineToken[];
  rawText: string;
}

/**
 * Tokenize a single line's inline markdown: bold, italic, underline, strike, code, highlight, math, wikilinks
 * , tags.
 */
export function tokenizeInline(text: string): InlineToken[] {
  if (!text) return [];

  const tokens: InlineToken[] = [];
  const remaining = text;

  // Pattern matching precedence
  // 1. Math:  $...$  or  $...$ 
  // 2. WikiLink: [[Target|Alias]] or [[Target]]
  // 3. Markdown Link: [label](url)
  // 4. Raw URL: https://...
  // 5. Underline: <u>...</u>

```

```
// 6. Bold: **...**
// 7. Italic: *...*
// 8. Strikethrough: ~~...~~
// 9. Highlight: ==...==
// 10. Code: `...`
// 11. Tag: #tag
//
// Maths needs no space inside its $...$ and no digit after it, so prices
// ("5 and $10") stay text; italics need non-word characters around
// them, so "2*3*4" stays text; a tag starts with a letter, so "#42" does
// not become one.
const inlineRegex =
  /(\$\$(.+?)\$\$|\\$([^\n]*?[^\\$])?)\$(?:\d|\[[^\]]+\)(?:\|([^\|]+)?\]|\\$)|\[[^\]]+\)\|((^)+)\|https?:\/\/[^\s]+|<u>(.*?)</u>|\\*(.*?)*\|(?![\w*])\*(?![\s*])(.+(?![\s*])\*(?![\w*])|~(.*?))~|=|(.*?)=|^([^\+]+)'(?![\w&/])#([a-zA-Z\u0590-\u05FF\u0600-\u06FF][a-zA-Z\u0590-\u05FF\u0600-\u06FF-]*)/g;

let lastIndex = 0;
let match: RegExpExecArray | null;

while ((match = inlineRegex.exec(remaining)) !== null) {
  if (match.index > lastIndex) {
    tokens.push({
      type: "text",
      text: remaining.slice(lastIndex, match.index),
    });
  }

  const fullMatch = match[0];

  if (fullMatch.startsWith("$") && fullMatch.endsWith("$")) {
    tokens.push({ type: "math", text: match[2], value: match[2] });
  } else if (fullMatch.startsWith("$") && fullMatch.endsWith("$") && fullMatch.length > 2) {
    tokens.push({ type: "math", text: match[3], value: match[3] });
  } else if (fullMatch.startsWith("[[")]) {
    const target = match[4].trim();
    const alias = match[5]??.trim() || target;
    tokens.push({ type: "link", text: alias, value: target });
  } else if (match[6] && match[7]) {
    tokens.push({ type: "url", text: match[6], value: match[7] });
  } else if (fullMatch.startsWith("http://") || fullMatch.startsWith("https://")) {
    tokens.push({ type: "url", text: fullMatch, value: fullMatch });
  } else if (fullMatch.startsWith("<u>")) {
    tokens.push({ type: "underline", text: match[8] || "" });
  } else if (fullMatch.startsWith("***")) {
    tokens.push({ type: "bold", text: match[9] || "" });
  } else if (fullMatch.startsWith("*")) {
    tokens.push({ type: "italic", text: match[10] || "" });
  } else if (fullMatch.startsWith("~~")) {
    tokens.push({ type: "strike", text: match[11] || "" });
  } else if (fullMatch.startsWith("==")) {
    tokens.push({ type: "highlight", text: match[12] || "" });
  } else if (fullMatch.startsWith("`")) {
    tokens.push({ type: "code", text: match[13] || "" });
  } else if (fullMatch.startsWith("#")) {
    tokens.push({ type: "tag", text: fullMatch, value: match[14] });
  } else {
    tokens.push({ type: "text", text: fullMatch });
  }

  lastIndex = inlineRegex.lastIndex;
}

if (lastIndex < remaining.length) {
  tokens.push({
    type: "text",
    text: remaining.slice(lastIndex),
  });
}

return tokens;
}

/**
 * Parse full markdown text into blocks for rendering.
 */
export function parseMarkdownBlocks(text: string): BlockToken[] {
  if (!text) return [];
}
```

```

const lines = text.split("\n");
const blocks: BlockToken[] = [];
let inCodeBlock = false;
let codeBuffer: string[] = [];
let codeStartLine = 0;

for (let i = 0; i < lines.length; i++) {
  const rawLine = lines[i];
  const trimmed = rawLine.trim();

  // Code block toggle: ```
  if (trimmed.startsWith("```")) {
    if (inCodeBlock) {
      // End code block
      blocks.push({
        type: "codeblock",
        lineIndex: codeStartLine,
        rawText: codeBuffer.join("\n"),
        tokens: [{ type: "code", text: codeBuffer.join("\n") }],
      });
      codeBuffer = [];
      inCodeBlock = false;
    } else {
      // Start code block
      inCodeBlock = true;
      codeStartLine = i;
      codeBuffer = [];
    }
    continue;
  }

  if (inCodeBlock) {
    codeBuffer.push(rawLine);
    continue;
  }

  // Heading 1: #
  if (rawLine.startsWith("# ") || rawLine === "#") {
    const content = rawLine.replace(/^#\s*/, "");
    blocks.push({
      type: "h1",
      lineIndex: i,
      rawText: rawLine,
      tokens: tokenizeInline(content),
    });
    continue;
  }

  // Heading 2: ##
  if (rawLine.startsWith("## ") || rawLine === "##") {
    const content = rawLine.replace(/^#\s*/, "");
    blocks.push({
      type: "h2",
      lineIndex: i,
      rawText: rawLine,
      tokens: tokenizeInline(content),
    });
    continue;
  }

  // Heading 3: ###
  if (rawLine.startsWith("### ") || rawLine === "###") {
    const content = rawLine.replace(/^#\s*/, "");
    blocks.push({
      type: "h3",
      lineIndex: i,
      rawText: rawLine,
      tokens: tokenizeInline(content),
    });
    continue;
  }

  // Checklist: - [ ] or - [x]
  const checkMatch = rawLine.match(/^(\\s*[-]\\s*\\([ xX])\\)\\s*)(.*)$/);
  if (checkMatch) {
    const isChecked = checkMatch[2].toLowerCase() === "x";
    const content = checkMatch[3];
  }
}

```

```
        blocks.push({
          type: "checklist",
          lineIndex: i,
          checked: isChecked,
          rawText: rawLine,
          tokens: tokenizeInline(content),
        });
        continue;
      }

      // Bullet list: - or * or +
      const bulletMatch = rawLine.match(/^(\s*[-*+]\s+)(.*)$/);
      if (bulletMatch) {
        blocks.push({
          type: "bullet",
          lineIndex: i,
          rawText: rawLine,
          tokens: tokenizeInline(bulletMatch[2]),
        });
        continue;
      }

      // Numbered list: 1. or 2.
      const numMatch = rawLine.match(/^(\s*(\d+)\. \s+)(.*)$/);
      if (numMatch) {
        blocks.push({
          type: "numbered",
          lineIndex: i,
          number: parseInt(numMatch[2], 10),
          rawText: rawLine,
          tokens: tokenizeInline(numMatch[3]),
        });
        continue;
      }

      // Blockquote: >
      if (rawLine.startsWith("> ") || rawLine === ">") {
        const content = rawLine.replace(/^>\s*/, "");
        blocks.push({
          type: "quote",
          lineIndex: i,
          rawText: rawLine,
          tokens: tokenizeInline(content),
        });
        continue;
      }

      // Regular paragraph / line
      blocks.push({
        type: "paragraph",
        lineIndex: i,
        rawText: rawLine,
        tokens: tokenizeInline(rawLine),
      });
    }

    // Handle unclosed code block
    if (inCodeBlock && codeBuffer.length > 0) {
      blocks.push({
        type: "codeblock",
        lineIndex: codeStartLine,
        rawText: codeBuffer.join("\n"),
        tokens: [{ type: "code", text: codeBuffer.join("\n") }],
      });
    }
  }

  return blocks;
}

export interface PlainLine {
  text: string;
  /** Font size relative to the element's (headings are larger). */
  scale: number;
  /** Font weight override (headings are bold). */
  weight?: number;
  /** Source line index, so exports can honour per-line direction. */
  lineIndex: number;
}
```

```

const HEADING_STYLE = {
  h1: { scale: 1.45, weight: 700 },
  h2: { scale: 1.25, weight: 600 },
  h3: { scale: 1.12, weight: 600 },
} as const;

/**
 * The text as the canvas SHOWS it, one entry per rendered line - for exports,
 * which draw plain SVG text. Without this a PDF showed the raw markup the
 * canvas hides: "- [ ] **Buy milk**" instead of "■ Buy milk".
 */
export function markdownToPlainLines(text: string): PlainLine[] {
  const out: PlainLine[] = [];
  const plain = (b: BlockToken) => b.tokens.map((t) => t.text).join("");
  const indent = (b: BlockToken) => b.rawText.match(/^\\s*/)?.[0] ?? "";
  for (const b of parseMarkdownBlocks(text)) {
    switch (b.type) {
      case "h1":
      case "h2":
      case "h3":
        out.push({ text: plain(b), ...HEADING_STYLE[b.type], lineIndex: b.lineIndex });
        break;
      case "checkboxlist":
        out.push({ text: `${indent(b)}${b.checked ? "■" : "■"} ${plain(b)}`, scale: 1, lineIndex: b.lineIndex });
        break;
      case "bullet":
        out.push({ text: `${indent(b)}• ${plain(b)}`, scale: 1, lineIndex: b.lineIndex });
        break;
      case "numbered":
        out.push({ text: `${indent(b)}${b.number}. ${plain(b)}`, scale: 1, lineIndex: b.lineIndex });
        break;
      case "quote":
        out.push({ text: `■ ${plain(b)}`, scale: 1, lineIndex: b.lineIndex });
        break;
      case "codeblock":
        b.rawText.split("\\n").forEach((line, k) => out.push({ text: line, scale: 1, lineIndex: b.lineIndex + k }));
        break;
      default:
        out.push({ text: plain(b), scale: 1, lineIndex: b.lineIndex });
    }
  }
  return out;
}

/**
 * Toggles a checklist item at a specific line index in full text.
 * Switches between "- [ ]" and "- [x]".
 */
export function toggleChecklistAtLine(text: string, targetLineIndex: number): string {
  const lines = text.split("\\n");
  if (targetLineIndex < 0 || targetLineIndex >= lines.length) return text;

  const line = lines[targetLineIndex];
  const uncheckedMatch = line.match(/^(\\s*[-]\\s*\\[\\s\\]\\s*)(.*)$/);
  if (uncheckedMatch) {
    lines[targetLineIndex] = line.replace(/\\[\\s\\]/, "[x]");
    return lines.join("\\n");
  }

  const checkedMatch = line.match(/^(\\s*[-]\\s*\\[[x]\\s\\]\\s*)(.*)$/);
  if (checkedMatch) {
    lines[targetLineIndex] = line.replace(/\\[[x]\\s\\]/, "[ ]");
    return lines.join("\\n");
  }

  return text;
}

/**
 * Intelligent list auto-continuation for the Enter key.
 * If user is on an empty list item (e.g. "- [ ]" or "- `"), returns shouldExit: true
 * so the editor clears the prefix and creates a regular newline.
 * Otherwise returns the appropriate prefix for the next line (incrementing numbers).
 */
export function getNextLinePrefix(currentLine: string): { prefix: string; shouldExit: boolean } {

```

```

// Empty checklist: line is just whitespace + - [ ] or - [x]
const emptyCheckMatch = currentLine.match(/^\s*[-*\[\] \s*\s*$/);
if (emptyCheckMatch) {
  return { prefix: "", shouldExit: true };
}

// Checklist with content: copy prefix to next line
const checkMatch = currentLine.match(/^\s*[-*\[\] \s*\s*$/);
if (checkMatch) {
  return { prefix: "- [ ] ", shouldExit: false };
}

// Empty bullet: line is just whitespace + - or * or +
const emptyBulletMatch = currentLine.match(/^\s*[-*+]\s*$/);
if (emptyBulletMatch) {
  return { prefix: "", shouldExit: true };
}

// Bullet with content: copy bullet prefix
const bulletMatch = currentLine.match(/^\s*[-*+]\s+/);
if (bulletMatch) {
  return { prefix: bulletMatch[1], shouldExit: false };
}

// Empty numbered: line is just whitespace + digits + .
const emptyNumMatch = currentLine.match(/^\s*\d+\.\s*$/);
if (emptyNumMatch) {
  return { prefix: "", shouldExit: true };
}

// Numbered list with content: increment number
const numMatch = currentLine.match(/^\s*(\d+)\.\s+/);
if (numMatch) {
  const nextNum = parseInt(numMatch[2], 10) + 1;
  return { prefix: `${numMatch[1]}${nextNum}.`, shouldExit: false };
}

// Blockquote with content: continue quote
const quoteMatch = currentLine.match(/^\s*>\s+/);
if (quoteMatch) {
  if (currentLine.trim() === ">") {
    return { prefix: "", shouldExit: true };
  }
  return { prefix: "> ", shouldExit: false };
}

return { prefix: "", shouldExit: false };
}

const BLOCK_PREFIX_RE = /^(s*[-*\[\] \s*|s*[-*+]\s*|s*\d+\.\s*|#+s*>\s*)/;

/**
 * First strong directional character wins; digits/punctuation look deeper. */
const RTL_STRONG_RE = /[\u0590-\u08FF\uFB1D-\uFDFF\uFE70-\uFEFC]/;
const LTR_STRONG_RE = /[A-Za-z\u00C0-\u024F\u0370-\u03FF\u0400-\u04FF\u4E00-\u9FFF]/;

/**
 * Auto-detects a line's direction from its first strong character, the same
 * rule browsers apply for dir="auto": Hebrew/Arabic lead RTL, Latin/Cyrillic/
 * Greek/CJK lead LTR, and a line with no letters at all reads LTR.
 */
export function detectLineDir(line: string): "ltr" | "rtl" {
  for (const ch of line) {
    if (RTL_STRONG_RE.test(ch)) return "rtl";
    if (LTR_STRONG_RE.test(ch)) return "ltr";
  }
  return "ltr";
}

/**
 * The direction a line renders with: an explicit ltr/rtl override wins,
 * anything else (missing, "auto") leaves it to the browser via dir="auto".
 */
export function resolveLineDir(
  lineDirs: Array<string | undefined> | undefined,
  lineIndex: number
): "ltr" | "rtl" | "auto" {
  const d = lineDirs?.[lineIndex];
  return d === "ltr" || d === "rtl" ? d : "auto";
}

```

```

}

/** Which source lines a selection touches (shared by prefix and direction). */
export function selectedLineRange(
  text: string,
  start: number,
  end: number
): { first: number; last: number } {
  const lines = text.split("\n");
  const starts: number[] = [];
  let off = 0;
  for (const ln of lines) {
    starts.push(off);
    off += ln.length + 1;
  }
  const lineOf = (pos: number): number => {
    let li = 0;
    for (let k = 0; k < lines.length; k++) {
      if (starts[k] <= pos) li = k;
    }
    return li;
  };
  const first = lineOf(start);
  // A selection ending exactly at a line start does not claim that line.
  const lastPos = end > start && text[end - 1] === "\n" ? end - 1 : Math.min(end, text.length);
  return { first, last: Math.max(first, lineOf(lastPos)) };
}

/**
 * Sets the direction override for every selected line. "auto" clears back
 * to detection. Trailing autos are trimmed so the array never grows past
 * the lines it describes.
 */
export function setLineDirs(
  lineDirs: Array<"ltr" | "rtl" | "auto"> | undefined,
  text: string,
  start: number,
  end: number,
  dir: "ltr" | "rtl" | "auto"
): Array<"ltr" | "rtl" | "auto"> {
  const lineCount = text.split("\n").length;
  const { first, last } = selectedLineRange(text, start, end);
  const out: Array<"ltr" | "rtl" | "auto"> = [];
  for (let i = 0; i < lineCount; i++) {
    if (i >= first && i <= last) out.push(dir);
    else out.push(lineDirs?.[i] ?? "auto");
  }
  while (out.length > 0 && out[out.length - 1] === "auto") out.pop();
  return out;
}

/**
 * Applies a line prefix (lists, headings, quotes) to every line the
 * selection touches. A collapsed cursor keeps the old single-line toggle
 * exactly; a real selection prefixes each intersected line (a selection
 * ending right at a line start does not claim that line). Toggles off only
 * when EVERY touched line already carries the prefix.
 */
export function prefixSelectedLines(
  text: string,
  start: number,
  end: number,
  prefix: string
): { updated: string; cursor: number } {
  const lines = text.split("\n");
  const { first, last } = selectedLineRange(text, start, end);
  const allHave = lines.slice(first, last + 1).every((l) => l.startsWith(prefix));
  const out = lines.map((l, idx) => {
    if (idx < first || idx > last) return l;
    if (allHave) return l.startsWith(prefix) ? l.slice(prefix.length) : l;
    if (l.startsWith(prefix)) return l;
    return `${prefix}${l.replace(BLOCK_PREFIX_RE, "")}`;
  });
  let cursor = 0;
  for (let k = 0; k < last; k++) cursor += out[k].length + 1;
  return { updated: out.join("\n"), cursor: cursor + out[last].length };
}

```

```

/**
 * Wraps the selection in a marker pair (Bold, Italic, code, `$_$, `[...]`).
 * With NO selection it inserts the empty pair and parks the cursor between
 * the markers, so typing starts inside immediately – no placeholder word to
 * delete first. A real selection wraps exactly as before.
 */
export function spliceFormatMarkers(
  text: string,
  start: number,
  end: number,
  prefix: string,
  suffix: string
): { updated: string; cursor: number } {
  if (start === end) {
    return { updated: text.slice(0, start) + prefix + suffix + text.slice(end), cursor: start + prefix.length
    };
  }
  const sel = text.slice(start, end);
  const replacement = `${prefix}${sel}${suffix}`;
  return {
    updated: text.slice(0, start) + replacement + text.slice(end),
    cursor: start + prefix.length + sel.length,
  };
}

```

```

=====
// === END FILE: src/lib/whiteboard/textParser.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/textbox.ts ===
// File 128 of 194 | Lines: 25 | Size: 1377 bytes
=====

```

```

/**
 * Text boxes that were started but never written in.
 *
 * Tapping with the text tool creates a box immediately so you can start
 * typing. If you tap away without typing, the box used to stay on the page as
 * an invisible element that only showed a "Type thoughts..." placeholder while
 * selected - and they piled up, because nothing ever removed them. GoodNotes,
 * Notability and OneNote all throw such a box away, and so do we: the store
 * discards it when editing ends (see `discardAbandonedText` in the store), and
 * the loader drops ones that older builds left behind (persistence.ts).
 *
 * ONE definition, used by both, so they can never disagree.
 */
import type { WhiteboardElement } from "../types";

export function isAbandonedTextBox(el: WhiteboardElement | null | undefined): boolean {
  if (!el || el.type !== "text") return false;
  const t = el as { text?: unknown; backgroundColor?: unknown; borderColor?: unknown };
  if (typeof t.text === "string" && t.text.trim() !== "") return false;
  // An empty box that was deliberately given a fill or border is being used
  // as a shape, not abandoned. Keep it.
  if (typeof t.backgroundColor === "string" && t.backgroundColor && t.backgroundColor !== "transparent") return
    false;
  if (typeof t.borderColor === "string" && t.borderColor && t.borderColor !== "transparent") return false;
  return true;
}

```

```

=====
// === END FILE: src/lib/whiteboard/textbox.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/types.ts ===
// File 129 of 194 | Lines: 1407 | Size: 45512 bytes
=====

```

```

/**
 * Whiteboard Type System (v3 – Professional Obsidian Canvas Grade)
 * =====
 * Core type definitions for whiteboard elements, connections, styling, and boards.
 */

```

```
// =====
// Element Types
// =====

export interface BaseElement {
  id: string;
  type: ElementType;
  x: number;
  y: number;
  rotation: number;
  opacity: number;
  locked?: boolean;
  zIndex?: number;
  createdAt: number;
  updatedAt: number;
}

export type ElementType =
  | "stroke" // freehand pen, pencil, marker, highlighter, brush, calligraphy
  | "rectangle"
  | "ellipse"
  | "diamond"
  | "triangle"
  | "star"
  | "cloud"
  | "line"
  | "arrow"
  | "text"
  | "sticky"
  | "image"
  | "connection"; // Obsidian Canvas-grade semantic directed link between nodes

// =====
// Stroke Styles & Pen Types
// =====

/**
 * Pen families. These are modelled on what note apps actually ship (GoodNotes:
 * fountain / ball / brush; Samsung Notes: standard / fountain / slanted / ink
 * brush) plus the universal marker, highlighter and pencil.
 *
 * Each one must differ in HOW IT RENDERS, not just in its label. The previous
 * set did not: pen, marker, highlighter and pencil all had simulatePressure
 * off, thinning ~0 and no taper, so they produced the same geometry and
 * differed only in opacity, and brush vs calligraphy differed by 0.05 thinning.
 */
export type StrokeStyle =
  | "ballpoint"
  | "gel"
  | "fineliner"
  | "fountain"
  | "brush"
  | "marker"
  | "highlighter"
  | "pencil"
  // Legacy ids kept so strokes drawn before the pen rework still render.
  | "pen"
  | "calligraphy";

export interface PenPreset {
  thinning: number;
  smoothing: number;
  streamline: number;
  simulatePressure: boolean;
  startTaper: number;
  endTaper: number;
  opacity: number;
  mixBlendMode: "normal" | "multiply";
  label: string;
  /** Short description shown in the pen picker. */
  hint: string;
  /**
   * Round nib or flat chisel. A marker and a highlighter have a squared-off
   * tip; a pen does not. This is `cap` in perfect-freehand and is a visible
   * difference at both ends of every stroke.
   */
  cap: "round" | "flat";
  /**
```

```

    * How strongly speed changes the width, 0 = never. Real ballpoints and
    * fountain pens thin out when moved fast; a fineliner does not. Applies even
    * with a mouse, which is why the old presets all looked alike on desktop.
    */
speedResponse: number;
/**
    * Chisel-nib angle in degrees. When set, width depends on the DIRECTION of
    * travel – thick across the nib, thin along it. This is what makes a fountain
    * pen look calligraphic rather than just thin.
    */
nibAngle?: number;
/** Pencil tooth. Rendered as a grain filter over the stroke. */
grain?: boolean;
/**
    * Nominal line weight relative to the chosen size. Real pens differ mostly by
    * WEIGHT – a 0.3mm fineliner against a 0.7mm ballpoint – and without this the
    * everyday writing pens were within 8% of each other's ink area, i.e.
    * indistinguishable at handwriting size.
    */
sizeScale: number;
}

export const PEN_TYPE_PRESETS: Record<StrokeStyle, PenPreset> = {
  ballpoint: {
    thinning: 0.28, smoothing: 0.45, streamline: 0.42, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 1, mixBlendMode: "normal",
    cap: "round", speedResponse: 0.55, sizeScale: 0.7,
    label: "Ballpoint", hint: "Thin, crisp, thins slightly when you write fast",
  },
  gel: {
    thinning: 0.08, smoothing: 0.62, streamline: 0.52, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 1, mixBlendMode: "normal",
    cap: "round", speedResponse: 0.15, sizeScale: 1,
    label: "Pen", hint: "Smooth, even line – everyday writing",
  },
  fineliner: {
    thinning: 0, smoothing: 0.75, streamline: 0.62, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 1, mixBlendMode: "normal",
    cap: "round", speedResponse: 0, sizeScale: 0.5,
    label: "Fineliner", hint: "Perfectly even line – technical drawing",
  },
  fountain: {
    thinning: 0.5, smoothing: 0.6, streamline: 0.5, simulatePressure: false,
    startTaper: 0, endTaper: 6, opacity: 1, mixBlendMode: "normal",
    cap: "round", speedResponse: 0.35, nibAngle: 40, sizeScale: 1.05,
    label: "Fountain", hint: "Thick across the nib, thin along it – calligraphic",
  },
  brush: {
    thinning: 0.62, smoothing: 0.5, streamline: 0.4, simulatePressure: true,
    startTaper: 6, endTaper: 14, opacity: 0.95, mixBlendMode: "normal",
    cap: "round", speedResponse: 0.7, sizeScale: 1.25,
    label: "Brush", hint: "Pressure and speed sensitive, tapers to a point",
  },
  marker: {
    thinning: 0, smoothing: 0.5, streamline: 0.4, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 0.92, mixBlendMode: "normal",
    cap: "flat", speedResponse: 0, sizeScale: 1.6,
    label: "Marker", hint: "Thick, opaque, squared-off chisel tip",
  },
  highlighter: {
    thinning: 0, smoothing: 0.4, streamline: 0.3, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 0.35, mixBlendMode: "multiply",
    cap: "flat", speedResponse: 0, sizeScale: 3,
    label: "Highlighter", hint: "Wide, translucent, multiplies over text",
  },
  pencil: {
    thinning: 0.22, smoothing: 0.32, streamline: 0.22, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 0.85, mixBlendMode: "multiply",
    cap: "round", speedResponse: 0.4, grain: true, sizeScale: 0.85,
    label: "Pencil", hint: "Grainy graphite tooth, slightly translucent",
  },
  // ---- legacy ids -> nearest new pen, so old strokes keep rendering ----
  pen: {
    thinning: 0.08, smoothing: 0.62, streamline: 0.52, simulatePressure: false,
    startTaper: 0, endTaper: 0, opacity: 1, mixBlendMode: "normal",
    cap: "round", speedResponse: 0.15, sizeScale: 1,
    label: "Gel pen", hint: "Smooth, near-constant width",
  },

```

```

    },
    calligraphy: {
      thinning: 0.5, smoothing: 0.6, streamline: 0.5, simulatePressure: false,
      startTaper: 0, endTaper: 6, opacity: 1, mixBlendMode: "normal",
      cap: "round", speedResponse: 0.35, nibAngle: 40, sizeScale: 1.05,
      label: "Fountain", hint: "Thick across the nib, thin along it",
    },
  },
};

/** Every id that draws a freehand stroke, including legacy ones. */
export const STROKE_TOOL_IDS: string[] = [
  "ballpoint",
  "gel",
  "fineliner",
  "fountain",
  "brush",
  "marker",
  "highlighter",
  "pencil",
  "pen",
  "calligraphy",
];

export function isStrokeTool(tool: string): boolean {
  return STROKE_TOOL_IDS.includes(tool);
}

/**
 * Default ink per pen. Sizes differ because the pens differ: a fineliner is a
 * 0.3mm line and a highlighter is a broad chisel, and starting them both at the
 * same width makes half the picker feel identical.
 */
export const DEFAULT_TOOL_INK: Record<string, { color: string; size: number }> = {
  ballpoint: { color: "#1e293b", size: 3 },
  gel: { color: "#1e293b", size: 4 },
  fineliner: { color: "#1e293b", size: 2 },
  fountain: { color: "#1e293b", size: 5 },
  brush: { color: "#1e293b", size: 12 },
  marker: { color: "#3b82f6", size: 10 },
  highlighter: { color: "#fde047", size: 22 },
  pencil: { color: "#64748b", size: 4 },
  // legacy ids
  pen: { color: "#1e293b", size: 4 },
  calligraphy: { color: "#1e293b", size: 5 },
};

/** Legacy per-pen fields, for migrating a save written before `toolInk`. */
const LEGACY_INK_FIELDS: Record<string, { color: keyof StyleState; size: keyof StyleState }> = {
  pen: { color: "penColor", size: "penSize" },
  highlighter: { color: "highlighterColor", size: "highlighterSize" },
  marker: { color: "markerColor", size: "markerSize" },
  brush: { color: "brushColor", size: "brushSize" },
  calligraphy: { color: "calligraphyColor", size: "calligraphySize" },
  pencil: { color: "pencilColor", size: "pencilSize" },
};

/** Build a complete ink map, filling gaps from legacy fields then defaults. */
export function buildToolInk(
  partial?: Record<string, { color: string; size: number }>,
  legacy?: Partial<StyleState>
): Record<string, { color: string; size: number }> {
  const out: Record<string, { color: string; size: number }> = {};
  for (const id of STROKE_TOOL_IDS) {
    const saved = partial?.[id];
    if (saved && typeof saved.color === "string" && Number.isFinite(saved.size)) {
      out[id] = { color: saved.color, size: saved.size };
      continue;
    }
    const legacyField = LEGACY_INK_FIELDS[id];
    const fallback = DEFAULT_TOOL_INK[id] ?? { color: "#1e293b", size: 4 };
    out[id] = {
      color:
        (legacyField && (legacy?.[legacyField.color] as string | undefined)) || fallback.color,
      size:
        (legacyField && (legacy?.[legacyField.size] as number | undefined)) || fallback.size,
    };
  }
  return out;
}

```

```

}

/**
 * The pens offered in the picker: only pens that LOOK different. Ballpoint
 * and fineliner were near-duplicates of the gel pen (they differ mainly in
 * width, which the thickness setting already covers), so the picker offers
 * one "Pen". Their presets stay, so strokes written with them keep their look.
 */
export const PEN_TYPES: StrokeStyle[] = [
  "gel",
  "fountain",
  "brush",
  "pencil",
  "marker",
  "highlighter",
];

/** Whether pressing harder changes this pen's line at all. */
export function penUsesPressure(style: StrokeStyle): boolean {
  const p = PEN_TYPE_PRESETS[style];
  // simulatePressure: the width comes from writing SPEED, and real pressure
  // is ignored; thinning 0: the width never changes.
  return !!p && !p.simulatePressure && p.thinning > 0;
}

export interface StrokeElement extends BaseElement {
  type: "stroke";
  points: [number, number, number][]; // [x, y, pressure]
  color: string;
  strokeWidth: number;
  strokeStyle: StrokeStyle;
  path?: string;
  smoothing?: number;
  /**
   * `points` ARE the drawn line: already smoothed and hook-trimmed, so they
   * are drawn as they are. Set when the eraser first cuts a stroke - its
   * pieces must keep the exact shape you saw, and re-smoothing each piece
   * from its new start bent the writing beside every cut.
   */
  settled?: boolean;
  /** Ends made by the eraser: drawn blunt, never with the pen's taper. */
  cutStart?: boolean;
  cutEnd?: boolean;
}

// =====
// Shape Fill & Border Styles
// =====

export type EraserMode = "pixel" | "object";

export type FillStyle = "transparent" | "solid" | "semi" | "hatch";
export type DashStyle = "solid" | "dashed" | "dotted";

export interface RectangleElement extends BaseElement {
  type: "rectangle";
  width: number;
  height: number;
  cornerRadius?: number;
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;
  dashStyle: DashStyle;
  strokeWidth: number;
}

export interface EllipseElement extends BaseElement {
  type: "ellipse";
  width: number;
  height: number;
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;
  dashStyle: DashStyle;
  strokeWidth: number;
}

```

```
export interface DiamondElement extends BaseElement {
  type: "diamond";
  width: number;
  height: number;
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;
  dashStyle: DashStyle;
  strokeWidth: number;
}

export interface TriangleElement extends BaseElement {
  type: "triangle";
  width: number;
  height: number;
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;
  dashStyle: DashStyle;
  strokeWidth: number;
}

export interface StarElement extends BaseElement {
  type: "star";
  width: number;
  height: number;
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;
  dashStyle: DashStyle;
  strokeWidth: number;
}

export interface CloudElement extends BaseElement {
  type: "cloud";
  width: number;
  height: number;
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;
  dashStyle: DashStyle;
  strokeWidth: number;
}

// =====
// Lines & Arrows
// =====

export type ArrowheadType = "none" | "arrow" | "triangle" | "stealth" | "diamond" | "circle";

export interface LineElement extends BaseElement {
  type: "line";
  x2: number;
  y2: number;
  strokeColor: string;
  strokeWidth: number;
  dashStyle: DashStyle;
}

export interface ArrowElement extends BaseElement {
  type: "arrow";
  x2: number;
  y2: number;
  strokeColor: string;
  strokeWidth: number;
  dashStyle: DashStyle;
  startArrowhead: ArrowheadType;
  endArrowhead: ArrowheadType;
  arrowheadSize: number;
}

// =====
// Text & Sticky Notes & Typography
// =====

export type TextAlign = "left" | "center" | "right";

/**
```

```

* Per-line writing direction. Missing entries (and "auto") mean auto-detect
* from the line's first strong character, so one box holds RTL Hebrew and
* LTR English lines with no setup. Old saves have no array at all and read
* as all-auto.
*/
export type LineDir = "ltr" | "rtl" | "auto";

export const FONT_FAMILIES = [
  { id: "inter", label: "Clean (Inter)", value: "var(--font-inter), Inter, system-ui, sans-serif" },
  { id: "hand", label: "Handwritten (Caveat)", value: "var(--font-caveat), 'Caveat', cursive, sans-serif" },
  { id: "mono", label: "Code (JetBrains)", value: "var(--font-mono), 'JetBrains Mono', monospace" },
  { id: "serif", label: "Editorial (Playfair)", value: "var(--font-serif), 'Playfair Display', Georgia, serif" },
  { id: "display", label: "Display (Plus Jakarta)", value: "var(--font-display), 'Plus Jakarta Sans', sans-serif" },
];

export interface TextElement extends BaseElement {
  type: "text";
  width: number;
  height: number;
  text: string;
  color: string;
  fontSize: number;
  fontFamily: string;
  fontWeight: number;
  align: TextAlign;
  /** Per-line direction overrides, parallel to text lines (see LineDir). */
  lineDir?: LineDir;
  lineHeight?: number;
  backgroundColor?: string;
  borderColor?: string;
  cardMode?: boolean;
  borderRadius?: number;
  padding?: number;
  highlightColor?: string;
}

export const CARD_BACKGROUNDS = [
  { id: "transparent", label: "Transparent", bg: "transparent", border: "transparent" },
  { id: "white", label: "Paper White", bg: "ffffff", border: "e2e8f0" },
  { id: "yellow", label: "Soft Yellow", bg: "fef9c3", border: "fde047" },
  { id: "blue", label: "Soft Blue", bg: "eff6ff", border: "bfbfde" },
  { id: "green", label: "Soft Green", bg: "f0fdf4", border: "bbf7d0" },
  { id: "purple", label: "Soft Purple", bg: "faf5ff", border: "e9d5ff" },
  { id: "rose", label: "Soft Rose", bg: "fff1f2", border: "fecdd3" },
  { id: "dark", label: "Card Dark", bg: "18181b", border: "3f3f46" },
] as const;

export const TEXT_HIGHLIGHT_COLORS = [
  "transparent",
  "fef08a",
  "bae6fd",
  "bbf7d0",
  "fbcfe8",
  "fed7aa",
  "e9d5ff",
] as const;

export type StickyColor = "yellow" | "pink" | "blue" | "green" | "purple" | "orange" | "zinc";

export const STICKY_COLORS: Record<StickyColor, { bg: string; border: string; text: string; label: string }> =
{
  yellow: { bg: "fef9c3", border: "fde047", text: "713f12", label: "Yellow" },
  pink: { bg: "fce7f3", border: "f472b6", text: "831843", label: "Pink" },
  blue: { bg: "e0f2fe", border: "38bdf8", text: "0c4a6e", label: "Blue" },
  green: { bg: "dcfce7", border: "4ade80", text: "14532d", label: "Green" },
  purple: { bg: "f3e8ff", border: "c084fc", text: "581c87", label: "Purple" },
  orange: { bg: "ffedd5", border: "fb923c", text: "7c2d12", label: "Orange" },
  zinc: { bg: "f4f4f5", border: "d4d4d8", text: "18181b", label: "Zinc" },
};

export interface StickyElement extends BaseElement {
  type: "sticky";
  width: number;
  height: number;
  text: string;
  color: StickyColor;
}

```

```

    fontSize: number;
    align: TextAlign;
    /** Per-line direction overrides, parallel to text lines (see LineDir). */
    lineDirs?: LineDir[];
}

export interface ImageElement extends BaseElement {
  type: "image";
  width: number;
  height: number;
  src: string;
  borderColor?: string;
  borderWidth?: number;
  naturalWidth: number;
  naturalHeight: number;
  /**
   * Non-destructive edits (see lib/whiteboard/imageEdit.ts). The stored
   * pixels never change, so every edit can be undone or reset later.
   */
  crop?: ImageCrop;
  flipX?: boolean;
  flipY?: boolean;
  filter?: ImageFilter;
  frame?: ImageFrame;
}

/** The kept part of the picture, as fractions (0..1) of the flipped picture. */
export interface ImageCrop {
  x: number;
  y: number;
  w: number;
  h: number;
}

/** Split view layout: side by side (left | right) or stacked (top / bottom). */
export type SplitOrientation = "side" | "stacked";

export type ImageFilter = "none" | "document" | "mono" | "vivid" | "faded";
export type ImageFrame = "square" | "rounded" | "circle";

// =====
// Obsidian Canvas-Grade Semantic Directed Connections
// =====

export type NodeAnchor = "top" | "right" | "bottom" | "left" | "center" | "auto";
export type ConnectionRouting = "curved" | "orthogonal" | "straight";

export interface ConnectionEndpoint {
  boardId: string;
  elementId: string;
  anchor: NodeAnchor;
}

export type BuiltinLinkType =
  | "relates"
  | "depends"
  | "causes"
  | "contradicts"
  | "leads_to"
  | "supports"
  | "flow";

export const BUILTIN_LINK_TYPES: BuiltinLinkType[] = [
  "relates",
  "depends",
  "causes",
  "contradicts",
  "leads_to",
  "supports",
  "flow",
];

export const LINK_TYPE_COLORS: Record<BuiltinLinkType, string> = {
  relates: "#3b82f6", // Blue
  depends: "#ef4444", // Red
  causes: "#f97316", // Orange
  contradicts: "#dc2626", // Crimson
  leads_to: "#8b5cf6", // Purple

```

```

    supports: "#10b981",    // Green
    flow: "#06b6d4",       // Cyan
  };

export const LINK_TYPE_LABELS: Record<BuiltinLinkType, string> = {
  relates: "Relates to",
  depends: "Depends on",
  causes: "Causes",
  contradicts: "Contradicts",
  leads_to: "Leads to",
  supports: "Supports",
  flow: "Flows into",
};

export function getLinkTypeColor(
  linkType: string,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): string {
  if (linkType in LINK_TYPE_COLORS) {
    return LINK_TYPE_COLORS[linkType as BuiltinLinkType];
  }
  const custom = customLinkTypes.find((t) => t.id === linkType);
  return custom?.color || "#6b7280";
}

export function getLinkTypeLabel(
  linkType: string,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): string {
  if (linkType in LINK_TYPE_LABELS) {
    return LINK_TYPE_LABELS[linkType as BuiltinLinkType];
  }
  const custom = customLinkTypes.find((t) => t.id === linkType);
  return custom?.name || linkType || "Link";
}

export interface ConnectionElement extends BaseElement {
  type: "connection";
  source: ConnectionEndpoint;
  target: ConnectionEndpoint;
  linkType: string;
  label?: string;
  routing: ConnectionRouting;
  dashStyle: DashStyle;
  startArrowhead: ArrowheadType;
  endArrowhead: ArrowheadType;
  strokeWidth: number;
}

// Union of all whiteboard elements
export type WhiteboardElement =
  | StrokeElement
  | RectangleElement
  | EllipseElement
  | DiamondElement
  | TriangleElement
  | StarElement
  | CloudElement
  | LineElement
  | ArrowElement
  | TextElement
  | StickyElement
  | ImageElement
  | ConnectionElement;

// =====
// Tool & Style Types
// =====

export type ToolType =
  | "select"
  | "lasso"
  | "pan"
  | "ballpoint"
  | "gel"
  | "fineliner"
  | "fountain"
  | "brush"

```

```

| "marker"
| "highlighter"
| "pencil"
// legacy tool ids, still selectable from old saved state
| "pen"
| "calligraphy"
| "eraser"
| "rectangle"
| "ellipse"
| "diamond"
| "triangle"
| "star"
| "cloud"
| "line"
| "arrow"
| "text"
| "sticky"
| "image"
| "connect"
// Points without writing: nothing it draws is saved or undoable.
| "laser"
// Selects the text of PDF / Word / PowerPoint pages to copy or mark up.
| "textselect";

/**
 * How the laser shows where you point.
 * - "dot": a glowing dot only - the classic presenter pointer.
 * - "trail": the dot drags a short comet tail that fades within a moment.
 * - "draw": lines you can circle and underline with; they stay while you keep
 *           drawing and all fade together shortly after the pen lifts.
 */
export type LaserMode = "dot" | "trail" | "draw";

export const LASER_COLORS = ["#ef4444", "#22c55e", "#3b82f6", "#a855f7", "#f97316", "#ec4899"];

// =====
// Customisable tool bar
// =====

/** One button of the tool bar. "pen" and "shapes" each stand for a family. */
export type ToolbarItemId =
| "select"
| "pan"
| "textselect"
| "pen"
| "eraser"
| "laser"
| "autoshape"
| "shapes"
| "connect"
| "text"
| "sticky"
| "image";

/**
 * Every button the tool bar can hold, in the default order. `group` decides
 * where the thin dividers go: one is drawn wherever two neighbours differ.
 */
export const TOOLBAR_ITEMS: Array<{ id: ToolbarItemId; label: string; group: "move" | "ink" | "insert" }> = [
  { id: "select", label: "Select", group: "move" },
  { id: "pan", label: "Hand (move the page)", group: "move" },
  { id: "textselect", label: "Select text", group: "move" },
  { id: "pen", label: "Pens", group: "ink" },
  { id: "eraser", label: "Eraser", group: "ink" },
  { id: "laser", label: "Laser pointer", group: "ink" },
  { id: "autoshape", label: "Shape recognition", group: "ink" },
  { id: "shapes", label: "Shapes & lines", group: "insert" },
  { id: "connect", label: "Connector", group: "insert" },
  { id: "text", label: "Text", group: "insert" },
  { id: "sticky", label: "Sticky note", group: "insert" },
  { id: "image", label: "Picture", group: "insert" },
];

/** Every button there is, in their natural order. */
export const ALL_TOOLBAR_ITEMS: ToolbarItemId[] = TOOLBAR_ITEMS.map((t) => t.id);

/**
 * What a new tool bar holds. The hand is left out: fingers already move the

```

```

    * page, so on the tablet it only took space. It can still be added back.
    */
export const DEFAULT_TOOLBAR_ITEMS: ToolbarItemId[] = ALL_TOOLBAR_ITEMS.filter((id) => id !== "pan");

/**
 * A saved tool bar, kept only if it is a list of known, distinct buttons with
 * at least one in it; anything else means "the default".
 */
export function cleanToolbarItems(v: unknown): ToolbarItemId[] | undefined {
  if (!Array.isArray(v)) return undefined;
  const known = new Set<string>(ALL_TOOLBAR_ITEMS);
  const out: ToolbarItemId[] = [];
  for (const id of v) {
    if (typeof id === "string" && known.has(id) && !out.includes(id as ToolbarItemId)) out.push(id as ToolbarItemId);
  }
  return out.length > 0 ? out : undefined;
}

/**
 * The tool bar to show, from what was saved. A tool that did not exist when
 * the bar was saved appears at its default place - unless the user removed
 * it ('hidden'). Saved bars from before 'hidden' was kept count only the
 * hand as removed, the one tool that was ever left out by default.
 */
export function mergeToolbarItems(
  saved: ToolbarItemId[] | undefined,
  hidden: ToolbarItemId[] | undefined
): ToolbarItemId[] {
  if (!saved || saved.length === 0) return [...DEFAULT_TOOLBAR_ITEMS];
  const removed = new Set<ToolbarItemId>(hidden ?? ["pan"]);
  const out = [...saved];
  DEFAULT_TOOLBAR_ITEMS.forEach((id, i) => {
    if (out.includes(id) || removed.has(id)) return;
    const before = DEFAULT_TOOLBAR_ITEMS.slice(0, i).reverse().find((x) => out.includes(x));
    out.splice(before ? out.indexOf(before) + 1 : 0, 0, id);
  });
  return out;
}

export interface StyleState {
  // Freehand
  /**
   * Per-tool ink, keyed by tool id. This replaced a hand-written switch that
   * had to be edited every time a pen was added – and was not, so ballpoint /
   * gel / fineliner / fountain silently fell through to `strokeColor` and
   * shared one colour and size with EACH OTHER and with shape borders.
   * A map cannot fall through, so a new pen can never again inherit that bug.
   */
  toolInk: Record<string, { color: string; size: number }>;

  // Legacy per-pen fields. Kept so an older save still migrates; the map above
  // is the source of truth.
  penColor: string;
  penSize: number;
  highlighterColor: string;
  highlighterSize: number;
  markerColor: string;
  markerSize: number;
  brushColor: string;
  brushSize: number;
  calligraphyColor: string;
  calligraphySize: number;
  pencilColor: string;
  pencilSize: number;
  eraserSize: number;
  /**
   * "pixel" – erase only the part of a stroke under the eraser, splitting it.
   * "object" – delete whole elements on contact (the old behaviour).
   * Shapes, text, stickies and images have no partial form, so they are always
   * deleted whole regardless of this setting.
   */
  eraserMode: EraserMode;

  // Shapes & Lines
  strokeColor: string;
  fillColor: string;
  fillStyle: FillStyle;

```

```

dashStyle: DashStyle;
strokeWidth: number;
shapeStrokeColor: string;
shapeFillColor: string;

// Lines & Arrows
lineColor: string;
linewidth: number;
startArrowhead: ArrowheadType;
endArrowhead: ArrowheadType;

// Text & Stickies
textColor: string;
fontSize: number;
fontFamily: string;
fontWeight: number;
textAlign: TextAlign;
stickyColor: StickyColor;

// Connections
connectionLinkType: string;
connectionRouting: ConnectionRouting;
connectionArrowStart: ArrowheadType;
connectionArrowEnd: ArrowheadType;

// Laser pointer. Size is in SCREEN pixels: a pointer is for the audience,
// and it should look the same at every zoom.
laserColor: string;
laserSize: number;
laserMode: LaserMode;
/** "draw" mode: how long the lines stay after the pen lifts. */
laserFadeMs: number;
}

/**
 * A pinned pen on the Favorites bar. Each pin is a full (pen type + color +
 * size) triple, so the same pen type can be pinned many times in different
 * colors — that is the whole point of the bar.
 */
export interface PinnedPen {
  id: string;
  tool: StrokeStyle;
  color: string;
  size: number;
}

/** Screen position of the floating Favorites bar (px from top-left). */
/**
 * Which edge the tool dock is attached to.
 *
 * Fixing it to the bottom wastes the widest dimension on a landscape tablet
 * and puts the tools under your writing hand. Every serious note app lets the
 * bar move; this is that setting.
 */
export type DockPosition = "bottom" | "top" | "left" | "right";

export const DOCK_POSITIONS: Array<{ id: DockPosition; label: string }> = [
  { id: "top", label: "Top" },
  { id: "bottom", label: "Bottom" },
  { id: "left", label: "Left" },
  { id: "right", label: "Right" },
];

/** Left and right docks stack vertically and need different layout. */
export function isVerticalDock(pos: DockPosition): boolean {
  return pos === "left" || pos === "right";
}

export interface PinnedBarPosition {
  x: number;
  y: number;
}

export const PEN_COLORS = [
  "#1e293b", // Slate / Black
  "#ef4444", // Red
  "#f97316", // Orange
  "#eab308", // Amber

```